

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal	ocean_proximity
0	2.344766	41.0	6.984127	1.023810	322.0	-0.049597	37.88	-122.23	4.526	2
1	2.332238	21.0	6.238137	0.971880	2401.0	-0.092512	37.86	-122.22	3.585	2
2	1.782699	52.0	8.288136	1.073446	496.0	-0.025843	37.85	-122.24	3.521	1
3	0.932968	52.0	5.817352	1.073059	558.0	-0.050329	37.85	-122.25	3.413	0
4	-0.012881	52.0	6.281853	1.081081	565.0	-0.085616	37.85	-122.25	3.422	0

```
# Tabela das classes criadas
# Com base nela é possível decodificar a coluna ocean_proximity

encoder.classes_
array([0, 1, 2])
```

Exercício 1: Normalização

Normalize a coluna **AveRooms** utilizando o Min-Max Scaling.

```
# Exercício resolvido
# Normalizar a coluna AveRooms utilizando Min-Max Scaling
scaler = MinMaxScaler()
df['AveRooms'] = scaler.fit_transform(df[['AveRooms']])
print(df[['AveRooms']].head())
```

```
AveRooms
0    0.043512
1    0.038224
2    0.052756
3    0.035241
4    0.038534
```

Conjuntos de Treino, Validação e Teste

Objetivos

- » Compreender a importância de dividir os dados em conjuntos de treino, validação e teste.
- » Aprender a dividir um conjunto de dados utilizando bibliotecas populares do Python.
- » Implementar e avaliar modelos de machine learning utilizando esses conjuntos de dados.

1. Introdução

Dividir os dados em conjuntos de treino, validação e teste é uma prática essencial em *machine learning*. Essa divisão permite avaliar o desempenho do modelo de forma objetiva e evitar problemas como o *overfitting*.

Importância da Divisão dos Dados

- » **Treino:** Usado para ajustar os parâmetros do modelo.

- » **Validação:** Usado para ajustar os hiperparâmetros do modelo e selecionar o melhor modelo.
- » **Teste:** Usado para avaliar o desempenho final do modelo.

Exemplo de aplicação da divisão dos dados

Nesse exemplo utilizaremos o dataset do Titanic. O dataset do Titanic é um conjunto de dados bastante popular na comunidade de ciência de dados, especialmente para aqueles que estão iniciando seus estudos em machine learning. Ele contém informações sobre os passageiros do Titanic, incluindo:

- » **Informações demográficas:** Idade, sexo, classe social.
- » **Informações sobre a viagem:** Porto de embarque, número de familiares a bordo.
- » **Resultado:** Se o passageiro sobreviveu ou não ao naufrágio.

Nesse parte, o objetivo é dividir esse dataset em treino, validação e teste utilizando duas técnicas distintas Holdout e K-fold.

Importação dos dados

```
# Importa-se as bibliotecas
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

# Carregar o conjunto de dados do Titanic para trabalhar como exemplo
url = 'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv'
df = pd.read_csv(url)

# Visualizar os primeiros dados
df.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

Holdout

Aqui será apresentado a divisão dos dados em Treino, Validação e Teste. E o treino de um modelo simples para acompanhar a importância dessa divisão.

Algumas etapas importante são:

- » **Seleciona as features relevantes:** Idade (Age) e tarifa (fare), como tentativa para prever se o passageiro sobreviveu ou não.
- » **Remove dados faltantes:** Garante que o modelo seja treinado com dados completos.
- » **Divide os dados:** Cria conjuntos de treinamento (60%), validação (20%) e teste (20%) para avaliar o desempenho do modelo e evitar overfitting.
- » **Verifica as dimensões:** Confirma que os conjuntos foram divididos corretamente.

```
# Definir features e target

# Remove as linhas com valores faltantes (NaN) das colunas 'age', 'fare', 'survived'
utilizando o método dropna()
df_model = df[['age', 'fare', 'survived']].dropna()
# As colunas 'age' e 'fare' serão utilizadas como features (variáveis explicativas)
para o modelo.
X = df_model[['age', 'fare']]
# A coluna 'survived' será o target (variável a ser prevista), indicando se o
passageiro sobreviveu ou não.
y = df_model['survived']

# Dividir os dados em treino (60%), validação (20%) e teste (20%)
# O random_state é importante para garantir a reprodutibilidade dos resultados,
# especialmente quando se deseja comparar diferentes modelos ou realizar experimentos.
# Essa primeira chamada divide os dados em 60% para treinamento e 40% para validação e
teste.
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.4, random_
state=42)

# Essa segunda chamada divide os 40% restantes em partes iguais para validação e
teste.
# O parâmetro random_state=42 garante que a divisão seja sempre a mesma, permitindo
reprodutibilidade dos resultados.
# X_train, X_val, X_test: Contêm as features (idade e tarifa) para os conjuntos de
treinamento, validação e teste, respectivamente.
# y_train, y_val, y_test: Contêm os valores correspondentes da variável target
(sobreviveu ou não) para os conjuntos de treinamento, validação e teste,
respectivamente.
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_
state=42)

# Impressão do tamanhos dos conjuntos de dados
# Note que os tamanhos obedecem 60% para treino, 20% para validação e 20% para teste
print(f"Tamanho do conjunto de treino: {X_train.shape}")
print(f"Tamanho do conjunto de validação: {X_val.shape}")
print(f"Tamanho do conjunto de teste: {X_test.shape}")
```

```
Tamanho do conjunto de treino: (428, 2)
Tamanho do conjunto de validação: (143, 2)
Tamanho do conjunto de teste: (143, 2)
```

Treinando o Modelo

O código apresentado define uma função **treina_modelo** que encapsula o processo de treinamento e avaliação de um modelo de regressão logística.

A função **treina_modelo** recebe como entrada os dados de treinamento e validação, treina um modelo de regressão logística e avalia seu desempenho nos conjuntos de validação e teste, fornecendo como saída as respectivas acurácias.

Entradas:

- » **X_train**: Um DataFrame contendo as features (nesse caso, 'age' e 'fare') dos passageiros do conjunto de treinamento. Essas features serão utilizadas pelo modelo para aprender a prever se um passageiro sobreviveria ao naufrágio do Titanic.
- » **y_train**: Um Series contendo os valores da variável target (sobreviveu ou não) correspondentes aos dados de treinamento.
- » **X_val**: Um DataFrame contendo as features do conjunto de validação, utilizado para ajustar os hiperparâmetros do modelo e avaliar seu desempenho em dados não vistos durante o treinamento.
- » **y_val**: Um Series contendo os valores da variável target correspondentes aos dados de validação.

Saída:

A função não retorna nenhum valor explicitamente. Em vez disso, ela imprime na tela duas métricas de desempenho:

- » **Acurácia no conjunto de validação**: Indica a proporção de exemplos do conjunto de validação que foram classificados corretamente pelo modelo.
- » **Acurácia no conjunto de teste**: Indica a proporção de exemplos do conjunto de teste que foram classificados corretamente pelo modelo. Essa métrica fornece uma estimativa mais confiável do desempenho do modelo em dados

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

def treina_modelo(X_train, y_train, X_val, y_val):
    # Treinar o modelo no conjunto de treino
    model = LogisticRegression(max_iter=1000)
    model.fit(X_train, y_train)

    # Avaliar o modelo no conjunto de validação
    y_val_pred = model.predict(X_val)
    val_accuracy = accuracy_score(y_val, y_val_pred)
    print(f"Acurácia no conjunto de validação: {val_accuracy:.2f}")

    # Avaliar o modelo no conjunto de teste
    y_test_pred = model.predict(X_test)
```

continua

```
test_accuracy = accuracy_score(y_test, y_test_pred)
print(f"Acurácia no conjunto de teste: {test_accuracy:.2f}")
```

```
treina_modelo(X_train, y_train, X_val, y_val)
```

```
Acurácia no conjunto de validação: 0.64
```

```
Acurácia no conjunto de teste: 0.61
```

Validação Cruzada: K-Fold Cross-Validation

Validação cruzada é vantajosa porque pode retirar o fator "sorte" e de ter realizado uma divisão vantajosa para o modelo.

No K-fold dividimos os dados de treino em **k** grupos e comparamos entre eles.

Abaixo é a aplicação da validação cruzada k-fold (com k=5) para avaliar o desempenho do modelo de regressão logística.

Entradas:

- » **X_train**: Um DataFrame contendo as features (nesse caso, 'age' e 'fare') do conjunto de treinamento completo.
- » **y_train**: Um Series contendo os valores da variável target (sobreviveu ou não) correspondentes aos dados de treinamento completo.
- » **kf**: Um objeto KFold que define a estratégia de divisão dos dados em 5 folds.

Saída:

A cada iteração do loop, o código:

- » **Divide os dados**: Separa os dados de treinamento em um conjunto de treinamento e um conjunto de validação, de acordo com os índices definidos pelo objeto KFold.
- » **Treina o modelo**: Chama a função `treina_modelo` com os conjuntos de treinamento e validação atuais, treinando o modelo de regressão logística.
- » **Avalia o modelo**: A função `treina_modelo` calcula e imprime a acurácia do modelo nos conjuntos de validação e teste.

O objetivo principal da validação cruzada k-fold é obter uma estimativa mais robusta e confiável do desempenho do modelo, reduzindo o viés que poderia ocorrer devido a uma única divisão aleatória dos dados em treinamento e teste. Ao repetir o processo de treinamento e validação 5 vezes, utilizando diferentes combinações dos dados como conjunto de treinamento e validação, obtém-se diferentes métricas de desempenho, o que proporciona uma visão mais precisa do desempenho geral do modelo.

```
# Importa a classe KFold da biblioteca sklearn
from sklearn.model_selection import KFold

# Define o número de 5 folds para a validação cruzada
kf = KFold(n_splits=5)

# Itera sobre cada fold
for train_index, val_index in kf.split(X_train):
    # Divide os dados em conjuntos de treino e validação para o fold atual
    X_train_fold, X_val_fold = X.iloc[train_index], X.iloc[val_index]
    y_train_fold, y_val_fold = y.iloc[train_index], y.iloc[val_index]

    # Aqui chama-se a função de treinamento anteriormente definida
    # verifica-se as métricas a cada fold
    treina_modelo(X_train_fold, y_train_fold, X_val_fold, y_val_fold)
```

```
Acurácia no conjunto de validação: 0.58
Acurácia no conjunto de teste: 0.61
Acurácia no conjunto de validação: 0.64
Acurácia no conjunto de teste: 0.61
Acurácia no conjunto de validação: 0.66
Acurácia no conjunto de teste: 0.58
Acurácia no conjunto de validação: 0.64
Acurácia no conjunto de teste: 0.62
Acurácia no conjunto de validação: 0.56
Acurácia no conjunto de teste: 0.59
```

Exercício 1

Divida o conjunto de dados do titanic em conjuntos de treino (70%), validação (15%) e teste (15%).

OBS. Para questões de comparação, utilize o **random_state** igual a 42

```
# Exercício resolvido

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

# Carregar um conjunto de dados de exemplo
url = 'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv'
df = pd.read_csv(url)

# Visualizar os primeiros dados
df.head()

# Dividir os dados
# O parâmetro random_state=42 garante que a divisão seja sempre a mesma, permitindo
reprodutibilidade dos resultados.
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_
state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_
state=42)

print(f"Tamanho do conjunto de treino: {X_train.shape}")
```

continua

```
print(f"Tamanho do conjunto de validação: {X_val.shape}")
print(f"Tamanho do conjunto de teste: {X_test.shape}")
```

```
Tamanho do conjunto de treino: (499, 2)
Tamanho do conjunto de validação: (107, 2)
Tamanho do conjunto de teste: (108, 2)
```

Dados balanceados e desbalanceados

Objetivos

- » Compreender a importância de dados balanceados em modelos de machine learning.
- » Identificar técnicas para lidar com dados desbalanceados.
- » Implementar essas técnicas utilizando bibliotecas populares do Python.

1. Introdução

Em muitos problemas de *machine learning*, especialmente aqueles envolvendo classificação, os dados podem estar desbalanceados, significando que algumas classes têm muito mais exemplos do que outras. Isso pode afetar o desempenho do modelo, levando a previsões tendenciosas.

2. Explicação Teórica

Dados Balanceados vs. Desbalanceados

- » **Dados Balanceados:** Quando as classes no conjunto de dados têm aproximadamente o mesmo número de exemplos.
- » **Dados Desbalanceados:** Quando uma ou mais classes têm significativamente mais exemplos do que outras.

3. Exemplos Práticos

A exemplo do que foi utilizado anteriormente, o dataset do Titanic agora será explorado as questões de desbalanceamento.

Teremos algumas etapas a serem seguidas:

- » **Seleção das colunas relevantes para a análise de sobrevivência:** 'pclass' (classe social), 'age' (idade), 'sibsp' (número de irmãos/cônjuges a bordo) e 'fare' (tarifa paga).
- » **Remoção das linhas com valores ausentes (NaN)** para evitar erros no processamento posterior.
- » **Divisão em features e target:** Separa o DataFrame em features (X) contendo

as colunas selecionadas e a target (y) contendo a informação de sobrevivência (0 - não sobreviveu, 1 - sobreviveu).

- » **Divisão treino e teste:** Divide os dados em conjuntos de treinamento (X_train, y_train) e teste (X_test, y_test) utilizando a função train_test_split com uma proporção de 70% para treinamento e 30% para teste. Define um random_state para garantir reprodutibilidade dos resultados.
- » **Visualização da distribuição da classe target:** Imprime a contagem de valores da variável target no conjunto de treinamento, permitindo verificar a distribuição entre as classes "sobreviveu" e "não sobreviveu" (valor idealmente balanceado para um modelo de classificação).

Carregando Bibliotecas e Dados

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler

# Carregar um conjunto de dados de exemplo
url = 'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv'
df = pd.read_csv(url)

# Filtrar colunas relevantes e remover valores ausentes
df = df[['survived', 'pclass', 'age', 'sibsp', 'fare']].dropna()

# Dividir features e target
X = df.drop(columns=['survived'])
y = df['survived']

# Dividir os dados em treino (70%) e teste (30%)
# O parâmetro random_state=42 garante que a divisão seja sempre a mesma, permitindo
# reprodutibilidade dos resultados.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Visualizar distribuição da classe target
print(y_train.value_counts())
```

survived	
0	298
1	201

Name: count, dtype: int64

Treina o modelo

Essa é a mesma função apresentada no item de Conjuntos de Treino, Validação e Teste. Caso tenha dúvida sobre seu funcionamento, volte na célula <https://colab.research.google.com/drive/1s0wK-XDQOdUubutjqhJVQL-6q6jKAfy9#scrollTo=urlttD4B61SW>

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

def treina_modelo(X_train, y_train, X_val, y_val):
    # Treinar o modelo no conjunto de treino
    model = LogisticRegression(max_iter=1000)
    model.fit(X_train, y_train)

    # Avaliar o modelo no conjunto de validação
    y_val_pred = model.predict(X_val)
    val_accuracy = accuracy_score(y_val, y_val_pred)
    print(f"Acurácia no conjunto de validação: {val_accuracy:.2f}")

    # Avaliar o modelo no conjunto de teste
    y_test_pred = model.predict(X_test)
    test_accuracy = accuracy_score(y_test, y_test_pred)
    print(f"Acurácia no conjunto de teste: {test_accuracy:.2f}")

```

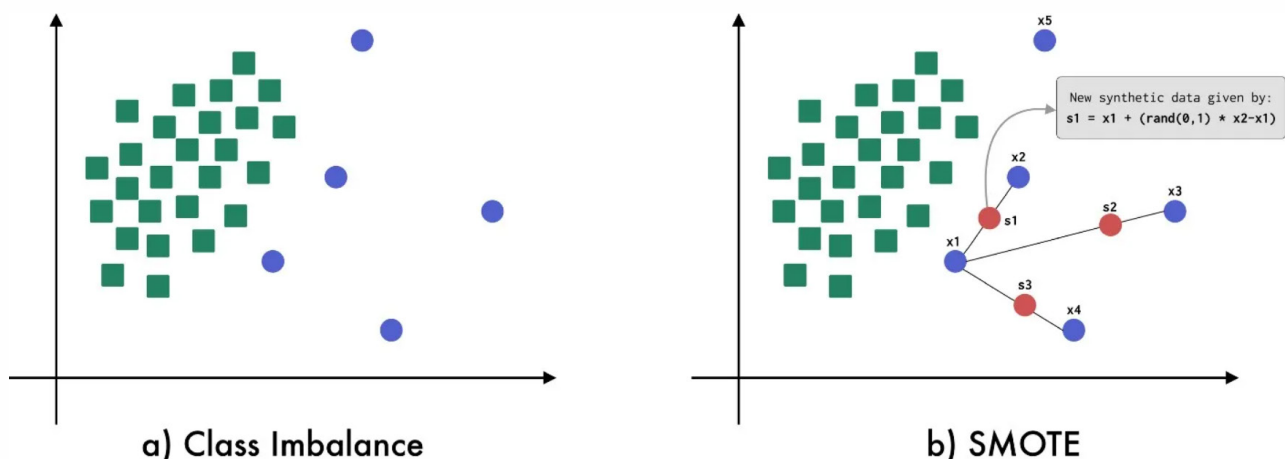
Oversampling com SMOTE

SMOTE cria dados artificiais com base nos **K** vizinhos mais próximos.

Algoritmo simplificado:

1. Seleciona a classe minoritária
2. Utiliza o KNN na classe minoritária com algum valor de **K**
3. Desenha-se linhas entre uma amostra da classe minoritária e seus **K** vizinhos
4. Escolhe um valor aleatório sobre essa linha

Figura 46 - Exemplo da utilização do SMOTE



Fonte: autoria própria.

A seguir é apresentado a implementação do oversampling com o SMOTE. Como entrada tem-se os dados do titanic anteriormente separados. Como saída tem-se ambas classes com a mesma quantidade (da classe com maior número de elementos).

Compare a quantidade de amostras **y_train_smote** apresentado abaixo com o **y_train** apresentado anteriormente. Note que a classe 0 é a que possui mais exemplos (298), enquanto que a classe 1 que possuía 201 antes, agora possui 298.

```
# cria o modelo smote através da classe SMOTE.
# novamente o random_state é utilizado para que os dados sejam replicáveis
smote = SMOTE(random_state=42)
# Cria novos dados com base no X_train e y_train utilizando o SMOTE.
# Esses novos dados são guardados nas variáveis X_train_smote e y_train_smote
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

# Visualizar distribuição da classe target após SMOTE
print(y_train_smote.value_counts())
```

```
survived
0      298
1      298
Name: count, dtype: int64
```

Compare as métricas de acurácia antes e depois do balanceamento.

Nota-se uma melhor acurácia dos dados quando há o balanceamento de classes.

```
# Comparação da acurácia com o balancelamento

print("Modelo treinado ignorando o desbalanceamento")
treina_modelo(X_train, y_train, X_test, y_test)

print("\n*****\n")
print("Modelo treinado após o SMOTE")
treina_modelo(X_train_smote, y_train_smote, X_test, y_test)
```

```
Modelo treinado ignorando o desbalanceamento
Acurácia no conjunto de validação: 0.71
Acurácia no conjunto de teste: 0.71
```

```
*****
```

```
Modelo treinado após o SMOTE
Acurácia no conjunto de validação: 0.73
Acurácia no conjunto de teste: 0.73
```

Undersampling

Diminui o dataset, retirando aleatoriamente da classe majoritária alguns exemplos. Nessa parte será apresentado o **RandomUnderSampler** que remove aleatoriamente dados da classe majoritária (classe 0).

Apesar de trazer o mesmo efeito de balanceamento entre classes, no undersampling reduz-se a quantidade de amostras. Nesse caso, serão eliminados aleatoriamente dados dos sobrevivente (rotulados com 1 na survived).

```
# cria o modelo undersampler através da classe RandomUnderSampler.
# novamente o random_state é utilizado para que os dados sejam replicáveis
undersampler = RandomUnderSampler(random_state=0)
# Eliminam-se dados com base no X_train e y_train utilizando o RandomUnderSampler.
# Os novos dados são guardados nas variáveis X_train_under e y_train_under
X_train_under, y_train_under = undersampler.fit_resample(X_train, y_train)

# Visualizar distribuição da classe target após undersampling
print(y_train_under.value_counts())
```

continua

```
survived
0      201
1      201
Name: count, dtype: int64
```

Exercício 1: Aplicar *Undersampling* e Treinar Modelo

Utilize a função **treina_modelo** (criada anteriormente) com os dados `X_train`, `y_train`, `X_test` e `y_test` também criados anteriormente.

Utilize a função **treina_modelo** (criada anteriormente) com os dados `X_train_under`, `y_train_under`, `X_test` e `y_test` também criados anteriormente.

Compare os resultados de acurácia do modelo treinado original (dados desbalanceados) e com dados balanceados pela técnica de *undersampling*.

```
# Exercício Resolvido
# Comparação da acurácia com o balancelamento

print("Modelo treinado ignorando o desbalancemanento")
treina_modelo(X_train, y_train, X_test, y_test)

print("\n*****\n")
print("Modelo treinado após o Undersampling")
treina_modelo(X_train_under, y_train_under, X_test, y_test)
```

```
Modelo treinado ignorando o desbalancemanento
Acurácia no conjunto de validação: 0.71
Acurácia no conjunto de teste: 0.71
```

```
*****
```

```
Modelo treinado após o Undersampling
Acurácia no conjunto de validação: 0.73
Acurácia no conjunto de teste: 0.73
```

Underfitting e *Overfitting*

Objetivos

- » Compreender os conceitos de *underfitting* e *overfitting* em modelos de *machine learning*.
- » Aprender a identificar *underfitting* e *overfitting* através de métricas de desempenho e visualizações.
- » Implementar técnicas para prevenir *underfitting* e *overfitting* em modelos utilizando Python.

1. Introdução

Em machine learning, underfitting e overfitting são problemas comuns que afetam a capacidade de generalização dos modelos. Underfitting ocorre quando um modelo é muito simples para capturar a complexidade dos dados, enquanto overfitting ocorre quando um modelo é muito complexo e captura o ruído dos dados de treino. Este notebook abordará esses conceitos e apresentará técnicas para mitigá-los.

Clique duas vezes (ou pressione "Enter") para editar

3. Exemplos Práticos

Exemplo 1

Nesse primeiro exemplo servirá para ilustrar modelos com underfitting, overfitting e um modelo ideal para o mesmo conjunto de dados.

O modelo real é definido pela função **true_function** e é uma função quadrática x^2 .

Utiliza-se essa função para gerar dados sintéticos e aleatórios ao redor desse modelo ideal, representado pelos pontos em vermelho e as variáveis **x** e **y**. **x** e **y** podem ser considerados o conjunto de dados de treino.

Criam-se os dados de teste **x_fit** e **y_true**

Carregando Bibliotecas e Dados

```
# Importação das bibliotecas que serão utilizadas
import matplotlib.pyplot as plt
import numpy as np

# Define o modelo ideal como sendo uma função quadrática
def true_function(x):
    return x**2

# Gerar dados sintéticos
# Define-se um gerador aleatório que seja reprodutível, é semelhante ao parâmetro
random_state visto anteriormente
np.random.seed(22)

# Geram-se 10 valores aleatórios entre -2 e 2
x = np.sort(np.random.rand(10) * 4 - 2) # random values between -2 and 2
# Para cada valor gerado anteriormente, aplica a função verdade `true_function(x)`
para obter o valor de y
# adiciona-se um ruído com `np.random.randn(10)` para que o valor não caia sempre
sobre o próprio modelo
y = true_function(x) + np.random.randn(10)
# Criam-se os dados de teste
# x_fit são 100 valores igualmente espaçados entre -2 e 2
x_fit = np.linspace(-2, 2, 100)
# y_true é o resultado do modelo ideal para os dados de x_fit
y_true = true_function(x_fit)
```

Underfitting

Para exemplificar o underfitting é utilizado o modelo de regressão linear **model_underfit**.

Os dados de treino **x** e **y** servem para treinar o modelo linear.

Com o modelo treinado testa-se seu desempenho passando os valores de teste **x_fit**, obtendo as predições e guardando em **y_underfit**.

Esses valores de **y_underfit** são utilizados em conjunto com **x_fit** para obter a curva do modelo ajustado da figura a esquerda.

```
from sklearn.linear_model import LinearRegression

# Cria o modelo de regressão linear
model_underfit = LinearRegression()
# Treina o modelo com dados de treino
model_underfit.fit(x.reshape(-1, 1), y)
# Obtem-se os resultados do modelo através do teste dos valores de x_fit
y_underfit = model_underfit.predict(x_fit.reshape(-1, 1))
```

Overfitting

Para exemplificar o overfitting é utilizado o modelo de regressão polinomial de grau 4 definido por **model_overfit**. O modelo de regressão polinomial é um modelo mais complexo que o de regressão linear utilizado no underfitting.

O pipeline consiste em dois passos:

- » Passo 1 (**PolynomialFeatures(degree)**): Primeiro, o pipeline aplica a transformação **PolynomialFeatures** com o grau definido anteriormente (4). Essa transformação gera novas features polinomiais a partir da feature original **x**.
- » Passo 2 (**LinearRegression()**): Em seguida, o pipeline aplica um modelo de regressão linear para aprender a relação entre as features polinomiais geradas na etapa anterior e a variável target **y**.

Os dados de treino **x** e **y** servem para treinar o modelo.

Com o modelo treinado testa-se seu desempenho passando os valores de teste **x_fit**, obtendo as predições e guardando em **y_overfit**.

Esses valores de **y_overfit** são utilizados em conjunto com **x_fit** para obter a curva do modelo ajustado da figura a direita.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

# Define o grau do modelo polinomial
degree = 4

# Cria o modelo de regressão polinomial
model_overfit = make_pipeline(PolynomialFeatures(degree), LinearRegression())
# Treina o modelo com dados de treino
```

continua

```
model_overfit.fit(x.reshape(-1, 1), y)
# Obtem-se os resultados do modelo através do teste dos valores de x_fit
y_overfit = model_overfit.predict(x_fit.reshape(-1, 1))
```

Comparação

No código abaixo são gerados 2 gráficos de comparação, um para o underfitting e outro para o overfitting.

Em cada gráfico são plotados:

- » Os dados de teste **x** e **y** representados por pontos vermelhos.
- » A curva fruto do modelo real **true_function** fruto do resultado da aplicação do conjunto de teste **x_fit** que obteve **y_true**. A curva é representada pela linha pontilhada.
- » A curva fruto do modelo (underfitting ou overfitting) fruto do resultado da aplicação do conjunto de teste **x_fit** no modelo **model_overfit** ou **model_underfit** que obtiveram **y_overfit** e **y_underfit** respectivamente. Essas curvas são representadas pelas linhas cheias.

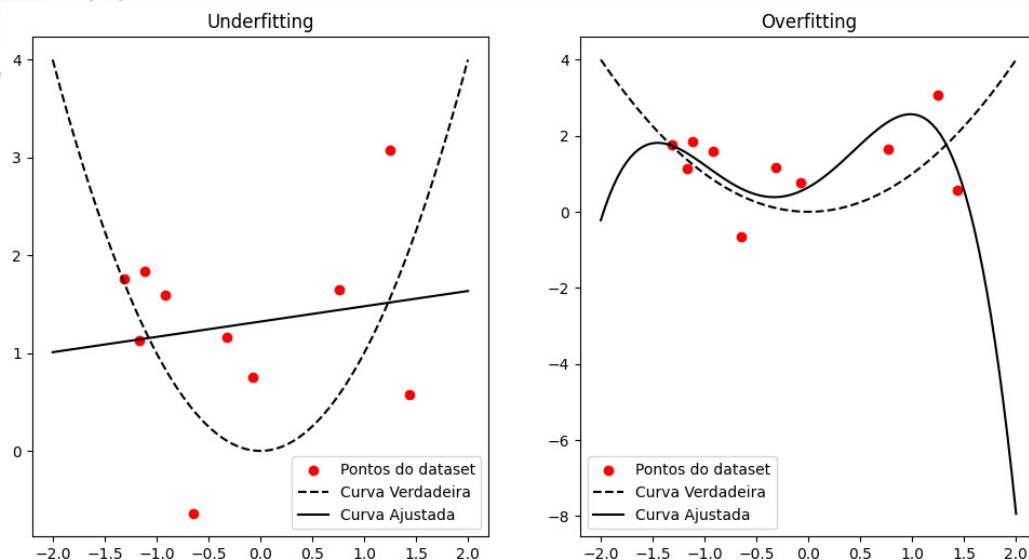
```
fig, axs = plt.subplots(1, 2, figsize=(12, 6))

# Underfitting plot
axs[0].scatter(x, y, color='red', label='Pontos do dataset')
axs[0].plot(x_fit, y_true, 'k--', label='Curva Verdadeira')
axs[0].plot(x_fit, y_underfit, 'k-', label='Curva Ajustada')
axs[0].set_title('Underfitting')
axs[0].legend()

# Overfitting plot
axs[1].scatter(x, y, color='red', label='Pontos do dataset')
axs[1].plot(x_fit, y_true, 'k--', label='Curva Verdadeira')
axs[1].plot(x_fit, y_overfit, 'k-', label='Curva Ajustada')
axs[1].set_title('Overfitting')
axs[1].legend()

plt.show()
```

Figura 47 - Underfitting vs Overfitting



Fonte: autoria própria.

Exemplo 2

Mais um exemplo agora para observar o impacto da escolha do parâmetro do modelo (degree) nas métricas de performance.

O código gera dados sintéticos com relação não linear, divide os dados em treino e teste, ordena os dados de teste para plotagem e define duas funções: uma para plotar os resultados do modelo e outra para calcular e imprimir métricas de desempenho (MSE e **R2**). Este código base pode ser utilizado para treinar diferentes modelos de regressão com features polinomiais e avaliar o seu desempenho.

O código pode ser dividido em algumas partes:

Geração de Dados Sintéticos:

A exemplo do que foi abordado no exemplo anterior, essa fase visa criar alguns dados fictícios.

- » **`np.random.seed(0)`**: Define uma semente para geração de números aleatórios, garantindo reprodutibilidade dos resultados.
- » **`X = np.sort(5 * np.random.rand(80, 1), axis=0)`**: Gera um array X com 80 linhas e 1 coluna, contendo valores aleatórios entre 0 e 5, ordenados em ordem crescente.
- » **`y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0])`**: Gera a variável target y baseada no seno dos valores de X com adição de um ruído gaussiano (normal) de média 0 e desvio padrão 0.1. A função `ravel()` transforma o array y em unidimensional.

Divisão de Dados:

Como visto anteriormente, a divisão dos dados em treino e teste é uma fase importante para a avaliação dos modelos.

- » **train_test_split**: Divide X e y em conjuntos de treinamento (X_train, y_train) e teste (X_test, y_test) com uma proporção de 80% para treinamento e 20% para teste. random_state=0 garante reprodutibilidade.

Ordenação para Plotagem:

Uma fase para auxiliar na visualização dos dados.

- » **z = sorted(zip(X_test,y_test))**: Cria uma lista z ordenando os pares (valor de X, valor de y) do conjunto de teste.
- » **X_test,y_test = zip(*z)**: Desempacota a lista ordenada z separando novamente as listas de X_test e y_test. Isso garante que os pontos de dados no teste sejam plotados na mesma ordem da previsão do modelo.

Função de Plotagem:

Para evitar a repetição de códigos, foi definida uma função específica para a plotagem dos gráficos.

plot_results: Função que recebe os dados de treinamento e teste, o modelo, features polinomiais (se houver) e um título, e gera um gráfico:

- » Plota os dados de treinamento (azul) e teste (vermelho) como pontos dispersos.
- » Plota a linha de previsão do modelo (verde) utilizando os dados de teste e as features polinomiais (se houver).
- » Adiciona título e legenda ao gráfico.
- » Exibe o gráfico utilizando plt.show().

Função de Métricas:

É definida uma função para obtenção das métricas cujo objetivo é análogo à função de plotagem.

print_metrics: Função que recebe os valores verdadeiros (y_true), os valores previstos (y_pred) e o nome do modelo, e calcula e imprime as métricas:

- » **mse**: Calcula o erro médio quadrático (Mean Squared Error) entre os valores verdadeiros e previstos.
- » **r2**: Calcula o R-quadrado (R^2), que indica a proporção da variância explicada pelo modelo.

- » Imprime o nome do modelo seguido das métricas MSE e R^2 formatadas com 4 casas decimais.

Carregando Bibliotecas e Dados

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import accuracy_score, mean_squared_error, r2_score

# Gerar dados sintéticos
np.random.seed(0)
X = np.sort(5 * np.random.rand(80, 1), axis=0)
y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0]) # Adiciona ruído

# Dividir os dados em conjunto de treinamento e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# Ordena os dados para facilitar a plotagem
z = sorted(zip(X_test, y_test))
X_test, y_test = zip(*z)

# Função para plotar os resultados
def plot_results(X_train, y_train, X_test, y_test, model, X_poly, title):
    plt.scatter(X_train, y_train, color='blue', label='Dados de Treinamento')
    plt.scatter(X_test, y_test, color='red', label='Dados de Teste')
    plt.plot(X_test, model.predict(X_poly), color='green', label='Modelo')
    plt.title(title)
    plt.legend()
    plt.show()

# Função para calcular e imprimir as métricas
def print_metrics(y_true, y_pred, model_name):
    mse = mean_squared_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    print(f"{model_name} - MSE: {mse:.4f}, R^2: {r2:.4f}")
```

Underfitting: Modelo Linear Simples

O código apresentado simula um caso de underfitting utilizando um modelo de regressão linear com features polinomiais de grau 1. Vamos analisar passo a passo:

Criando Features Polinomiais para Underfitting:

- » **poly_features_underfit = PolynomialFeatures(degree=1)**: Cria uma instância da classe PolynomialFeatures definindo o grau como 1.
- » **X_poly_train_underfit = poly_features_underfit.fit_transform(X_train)**: Aplica o transformador poly_features_underfit aos dados de treinamento X_train.

A função `fit_transform` encaixa o transformador nos dados de treinamento (calcula parâmetros internos, se necessário) e então aplica a transformação.

- » `X_poly_test_underfit = poly_features_underfit.transform(X_test)`: Aplica o transformador `poly_features_underfit` (já encaixado nos dados de treinamento) aos dados de teste `X_test`.

Treinando o Modelo de Underfitting:

- » `poly_reg_underfit = LinearRegression()`: Cria um modelo de regressão linear.
- » `poly_reg_underfit.fit(X_poly_train_underfit, y_train)`: Treina o modelo `poly_reg_underfit` utilizando as features polinomiais transformadas do treino (`X_poly_train_underfit`) e a variável target `y_train`.

Plotando os Resultados e Imprimindo Métricas:

`plot_results`:

Função definida anteriormente para plotar os resultados do modelo. Ela recebe os dados de treinamento e teste, o modelo treinado (`poly_reg_underfit`), as features polinomiais transformadas (`X_poly_test_underfit`) e um título descritivo ("Underfitting: Modelo Polinomial de Grau 1").

A função irá plotar os dados de treinamento e teste, e a linha de previsão do modelo utilizando as features polinomiais transformadas do teste (`X_poly_test_underfit`).

O título indica que este é um caso de underfitting devido ao uso de um polinômio de grau 1.

`print_metrics`:

Função definida anteriormente para calcular e imprimir métricas de desempenho (MSE e **R²**).

Ela é chamada duas vezes:

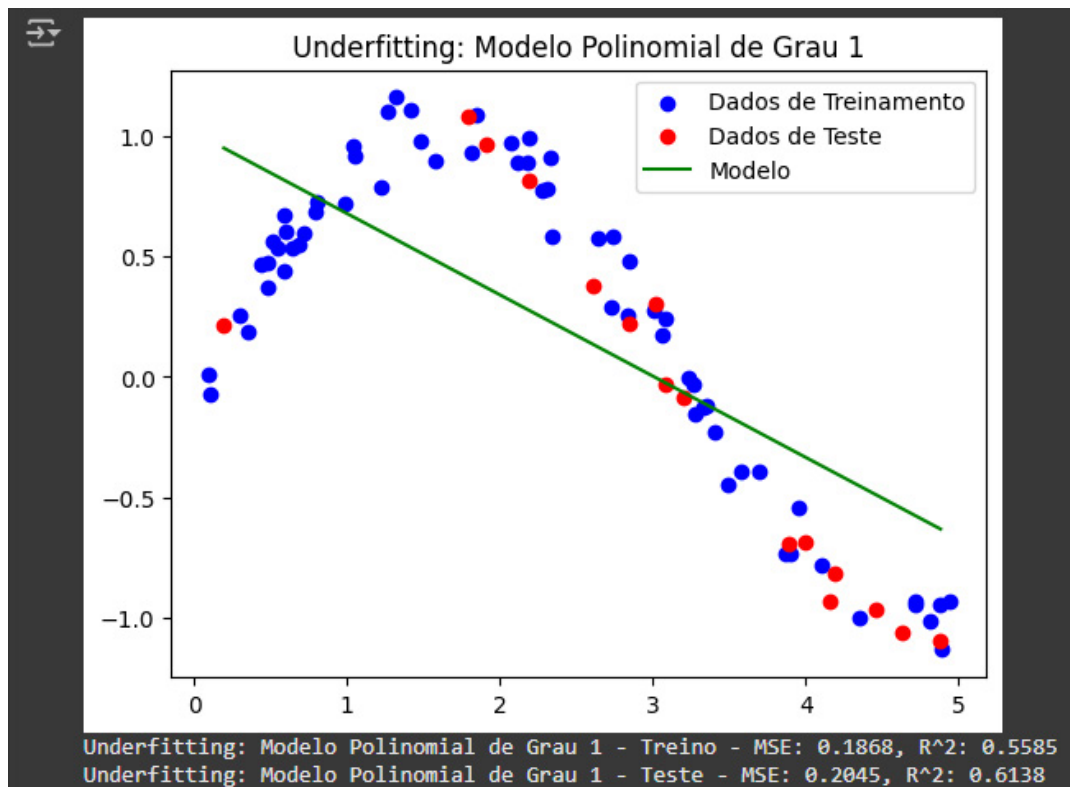
1. A primeira vez com os dados de treinamento (`y_train`) e a previsão do modelo no treinamento (`poly_reg_underfit.predict(X_poly_train_underfit)`) para avaliar o desempenho no conjunto de treino.
2. A segunda vez com os dados de teste (`y_test`) e a previsão do modelo no teste (`poly_reg_underfit.predict(X_poly_test_underfit)`) para avaliar o desempenho no conjunto de teste.

```
# Criando Features Polinomiais para Underfitting
poly_features_underfit = PolynomialFeatures(degree=1) # Grau 1 para underfitting
X_poly_train_underfit = poly_features_underfit.fit_transform(X_train)
X_poly_test_underfit = poly_features_underfit.transform(X_test)

# Treinando o Modelo de Underfitting
poly_reg_underfit = LinearRegression()
poly_reg_underfit.fit(X_poly_train_underfit, y_train)
plot_results(X_train, y_train, X_test, y_test, poly_reg_underfit, X_poly_test_underfit,
"Underfitting: Modelo Polinomial de Grau 1")

# Plotando os Resultados e Imprimindo Métricas
print_metrics(y_train, poly_reg_underfit.predict(X_poly_train_underfit), "Underfitting:
Modelo Polinomial de Grau 1 - Treino")
print_metrics(y_test, poly_reg_underfit.predict(X_poly_test_underfit), "Underfitting:
Modelo Polinomial de Grau 1 - Teste")
```

Figura 48 - Underfitting: Modelo Polinomial de Grau 1



Fonte: autoria própria.

Overfitting: Modelo Polinomial de Alto Grau

Semelhante ao apresentado anteriormente, com a única diferença que agora elevou-se o grau do polinômio para 15 (degree=15) para obter mais features e elevar a complexidade do modelo.

```
# Criando Features Polinomiais para overfitting
poly_features_overfit = PolynomialFeatures(degree=15) # Grau alto para overfitting
X_poly_train_overfit = poly_features_overfit.fit_transform(X_train)
```

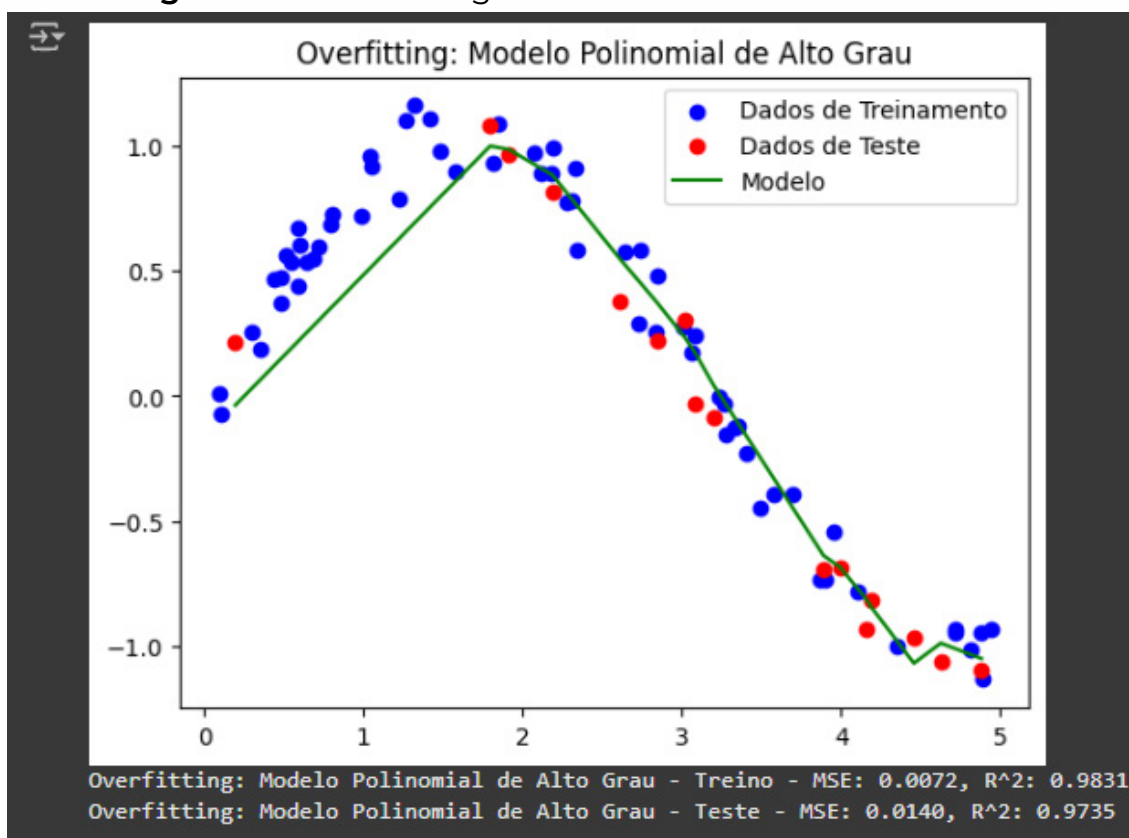
continua

```
X_poly_test_overfit = poly_features_overfit.transform(X_test)

# Treinando o Modelo de overfitting
poly_reg_overfit = LinearRegression()
poly_reg_overfit.fit(X_poly_train_overfit, y_train)
plot_results(X_train, y_train, X_test, y_test, poly_reg_overfit, X_poly_test_overfit,
"Overfitting: Modelo Polinomial de Alto Grau")

# Plotando os Resultados e Imprimindo Métricas
print_metrics(y_train, poly_reg_overfit.predict(X_poly_train_overfit), "Overfitting:
Modelo Polinomial de Alto Grau - Treino")
print_metrics(y_test, poly_reg_overfit.predict(X_poly_test_overfit), "Overfitting: Modelo
Polinomial de Alto Grau - Teste")
```

Figura 49 - Overfitting: Modelo Polinomial de Alto Grau



Fonte: autoria própria.

Fitting Adequado: Modelo Polinomial de Grau 3

Semelhante ao apresentado anteriormente, com a única diferença que agora utilizou o grau do polinômio 3 (degree=3) para obter mais features e adequar a complexidade do modelo.

```
poly_features_fit = PolynomialFeatures(degree=3) # Grau intermediário para fitting
adequado
X_poly_train_fit = poly_features_fit.fit_transform(X_train)
X_poly_test_fit = poly_features_fit.transform(X_test)

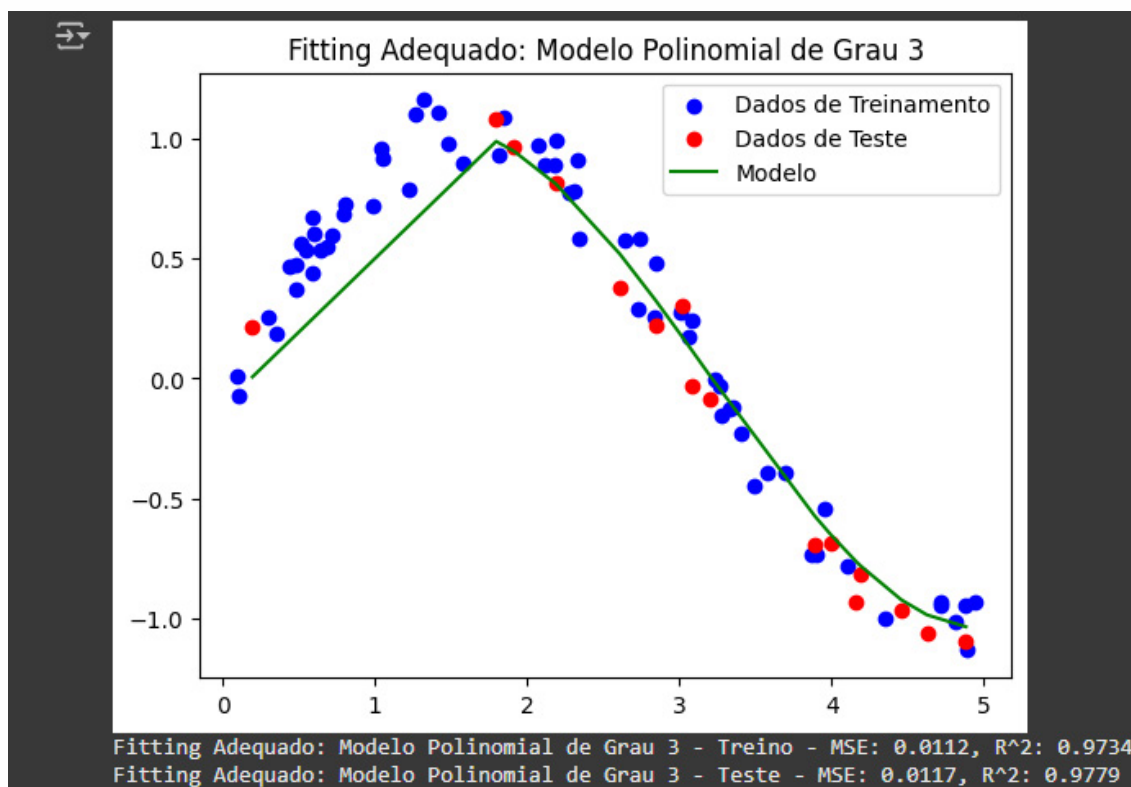
poly_reg_fit = LinearRegression()
poly_reg_fit.fit(X_poly_train_fit, y_train)
```

continua

```
plot_results(X_train, y_train, X_test, y_test, poly_reg_fit, X_poly_test_fit, "Fitting Adequado: Modelo Polinomial de Grau 3")

print_metrics(y_train, poly_reg_fit.predict(X_poly_train_fit), "Fitting Adequado: Modelo Polinomial de Grau 3 - Treino")
print_metrics(y_test, poly_reg_fit.predict(X_poly_test_fit), "Fitting Adequado: Modelo Polinomial de Grau 3 - Teste")
```

Figura 50 - Fitting Adequado: Modelo Polinomial de Grau 3



Fonte: autoria própria.

O que observar?

Observe as métricas MSE e R^2 entre os conjuntos de Treino e Teste em cada modelo.

- » **Underfitting:** valores elevado de MSE quando comparados com os modelos adequados e overfitting. Isso indica que os outros modelos representam melhor. Isso já era esperado uma vez que o modelo linear (grau 1) é o mais simples possível.
- » **Overfitting:** grande diferença no MSE quando comparado o dataset de Treino e Teste. Nesse caso, o modelo acaba se ajustando ao treino e é incapaz de generalizar para os novos dados (Teste).
- » **Adequado:** Não possui o melhor resultado de MSE no treino, mas obteve o melhor MSE no Teste. Também não há grandes diferenças de MSE entre o conjunto de Treino e Teste, indicando que o modelo conseguiu generalizar e manteve o mesmo acerto tanto para os dados que ele conhecia (Treino) como para os que ele nunca viu (Teste).

Referências Bibliográficas

McKinney, W. (2017). Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython. O'Reilly Media.

VanderPlas, J. (2016). Python Data Science Handbook: Essential Tools for Working with Data. O'Reilly Media.

Seaborn Documentation: <https://seaborn.pydata.org/>

Scikit-learn Documentation: <https://scikit-learn.org/stable/>

Pandas Documentation: <https://pandas.pydata.org/>



PARA RELEMBRAR...

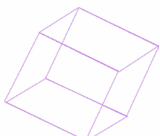
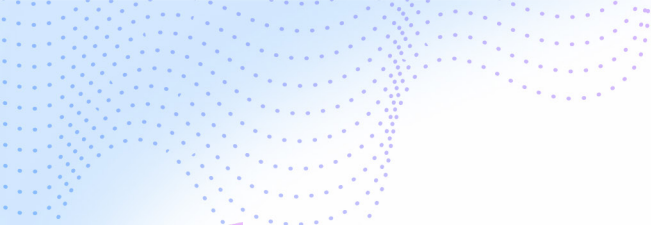
- » Lembre-se da importância dos dados no ML, um dado de entrada ruim resultará em modelos ruins, por isso é muito importante estar atento aos dados e os efeitos de treinar o modelo sem entender corretamente o que está ocorrendo. Tenha sempre em mente:
- » Qualidade dos dados como base para modelos eficazes
- » Diferenças entre dados estruturados e não estruturados
- » Tipos de conjuntos de dados: supervisionados, não supervisionados e semi-supervisionados
- » Processos de limpeza e pré-processamento de dados
- » Seleção e engenharia de características
- » Divisão dos dados em conjuntos de treino, validação e teste
- » Técnica de *cross-validation*
- » Tratamento de dados desbalanceados
- » Problemas de subajuste (*underfitting*) e sobreajuste (*overfitting*)

SAIBA MAIS...

- » [Conjunto de dados](#)
- » [Subajuste e sobreajuste](#)
- » [Dados desbalanceados](#)
- » [Validação cruzada](#)

Unidade VIII

Regressão



Unidade VIII - Regressão

A regressão, no contexto do ML, constitui uma técnica estatística fundamental para modelar a relação entre uma variável dependente contínua e uma ou mais variáveis independentes. Variável dependente contínua é aquela que estamos tentando prever ou explicar e que pode assumir qualquer valor numérico dentro de um intervalo. Ela é “dependente” porque seu valor é influenciado pelos valores de outras variáveis. Variáveis independentes são aquelas que utilizamos para prever o valor da variável dependente. Elas são chamadas de “independentes” porque, teoricamente, seus valores não são influenciados por outras variáveis no modelo. Um exemplo prático seria na previsão de consumo de energia em que a variável dependente contínua seria o consumo de energia em kWh (um valor numérico). As variáveis independentes poderiam ser: temperatura externa, número de pessoas na residência, hora do dia, etc. A variável dependente é o “efeito” que queremos explicar. As variáveis independentes são as “causas” que influenciam o efeito.

Essa metodologia, amplamente utilizada em diversas áreas do conhecimento, permite realizar previsões quantitativas com base em padrões identificados nos dados. Ao ajustar um modelo de regressão a um conjunto de dados, os cientistas de dados podem estimar o valor da variável dependente para novos conjuntos de dados, o que é de grande valor para tomada de decisões em diversos setores, como economia, finanças, indústria e marketing.

A escolha do modelo de regressão mais adequado depende das características dos dados e do problema a ser resolvido. Existem diversos tipos de regressão, cada um com suas particularidades e aplicações. A regressão linear, por exemplo, é a mais simples e amplamente utilizada, modelando a relação entre as variáveis através de uma equação linear. Já a regressão polinomial permite modelar relações não lineares, enquanto a regressão logística é utilizada para problemas de classificação, onde a variável dependente é binária.

A importância da regressão no ML reside na sua capacidade de extrair informações valiosas a partir de dados complexos. Ao identificar as relações entre as variáveis, os modelos de regressão podem ser utilizados para entender os fatores que influenciam um determinado fenômeno, otimizar processos, tomar decisões mais precisas e gerar novas hipóteses para pesquisa. Além disso, a regressão é uma técnica fundamental para outras técnicas de ML, como a árvore de decisão e a floresta aleatória.

Enquanto a regressão foca na predição de variáveis contínuas, como preços, temperaturas ou índices, a classificação é empregada para atribuir instâncias a categorias discretas. Por exemplo, determinar se um e-mail é spam ou não é um problema de classifi-

cação binária. Já prever a temperatura máxima para o próximo dia é um problema típico de regressão. A distinção entre esses dois tipos de problemas é crucial, pois os métodos e métricas de avaliação são distintos e a escolha do modelo adequado depende dessa compreensão.

8.1 Métricas de Avaliação

Para avaliar a performance de modelos de regressão, várias métricas são utilizadas:

8.1.1 Coeficiente de Determinação R^2

O coeficiente de determinação mede a proporção da variabilidade da variável dependente que é explicada pelas variáveis independentes no modelo. Os valores obtidos dessa métrica variam entre 0 e 1. Um valor de 1 indica que o modelo explica toda a variância dos dados, enquanto um valor de 0 indica que o modelo não explica nenhuma variância. O Coeficiente de Determinação, R^2 , é calculado como:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

onde:

y_i são os valores reais,

$\hat{y}_i$ são os valores preditos pelo modelo,

$\bar{y}$ é a média dos valores reais,

$\sum_{i=1}^n (y_i - \hat{y}_i)^2$ é a Soma dos Quadrados dos Resíduos (SSR),

$\sum_{i=1}^n (y_i - \bar{y})^2$ é a Soma Total dos Quadrados (SST).

O valor de R^2 varia entre 0 e 1:

$R^2 = 0$ indica que o modelo não explica nenhuma variação nos dados.

$R^2 = 1$ indica que o modelo explica toda a variação nos dados.

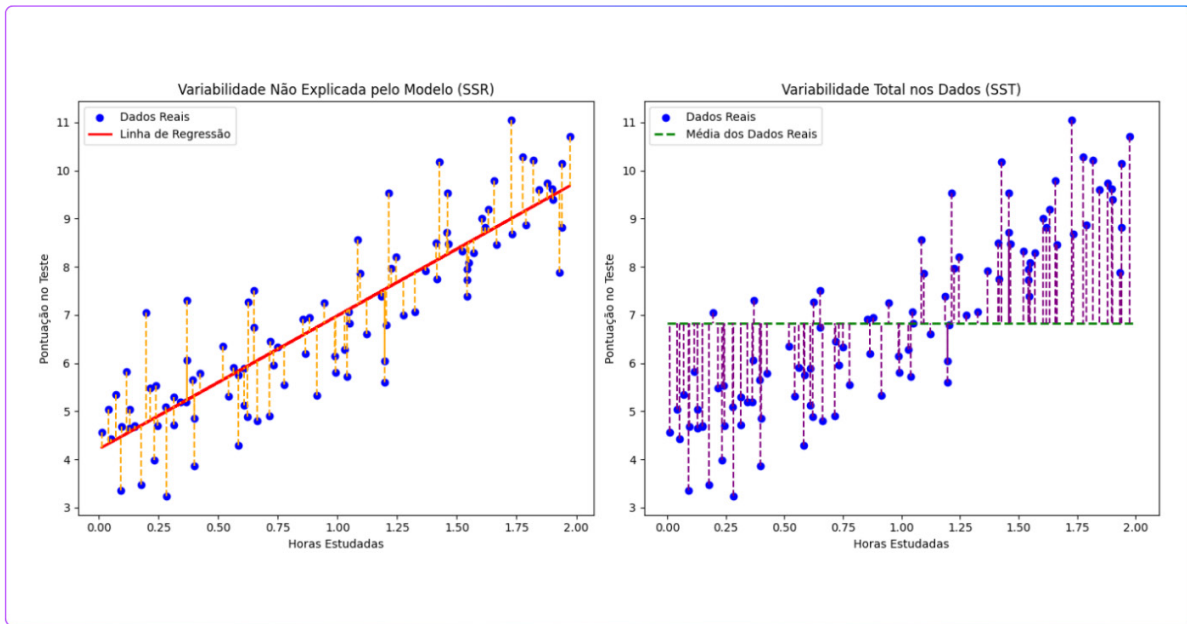
De forma facilitar a compreensão, a figura abaixo ilustra a aplicação das parcelas SSR e SST.

Variabilidade Não Explicada (SSR): As linhas pontilhadas laranja representam os erros do modelo, ou seja, a diferença entre o valor real (ponto em azul) e sua projeção ao modelo (linha vermelha). Quanto menores forem essas linhas, melhor o modelo se ajusta aos dados.

Variabilidade Total (SST): A linha horizontal verde representa a média dos dados. A distância entre os pontos e a linha representa a variabilidade total dos dados. O primeiro gráfico mostra a variabilidade que o modelo não consegue explicar.

Quanto menor a distância entre os pontos e a linha de regressão, melhor o modelo se ajusta aos dados. O segundo gráfico mostra a variabilidade total dos dados. Comparando os dois gráficos, pode-se ter uma ideia de quanto da variabilidade total está sendo explicada pelo modelo.

Figura 51 - Exemplo R^2



Fonte: autoria própria e disponível no Google Colab.

8.1.2 Erro Quadrático Médio (MSE)

O Erro Quadrático Médio, *Mean Squared Error* (MSE) é uma métrica utilizada para avaliar a performance de um modelo de regressão. Ele calcula a média dos quadrados das diferenças entre os valores observados (reais) e os valores preditos pelo modelo. O MSE é definido como:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

onde:

y_i são os valores reais,

$\hat{y}_i$ são os valores preditos pelo modelo,

n é o número de observações.

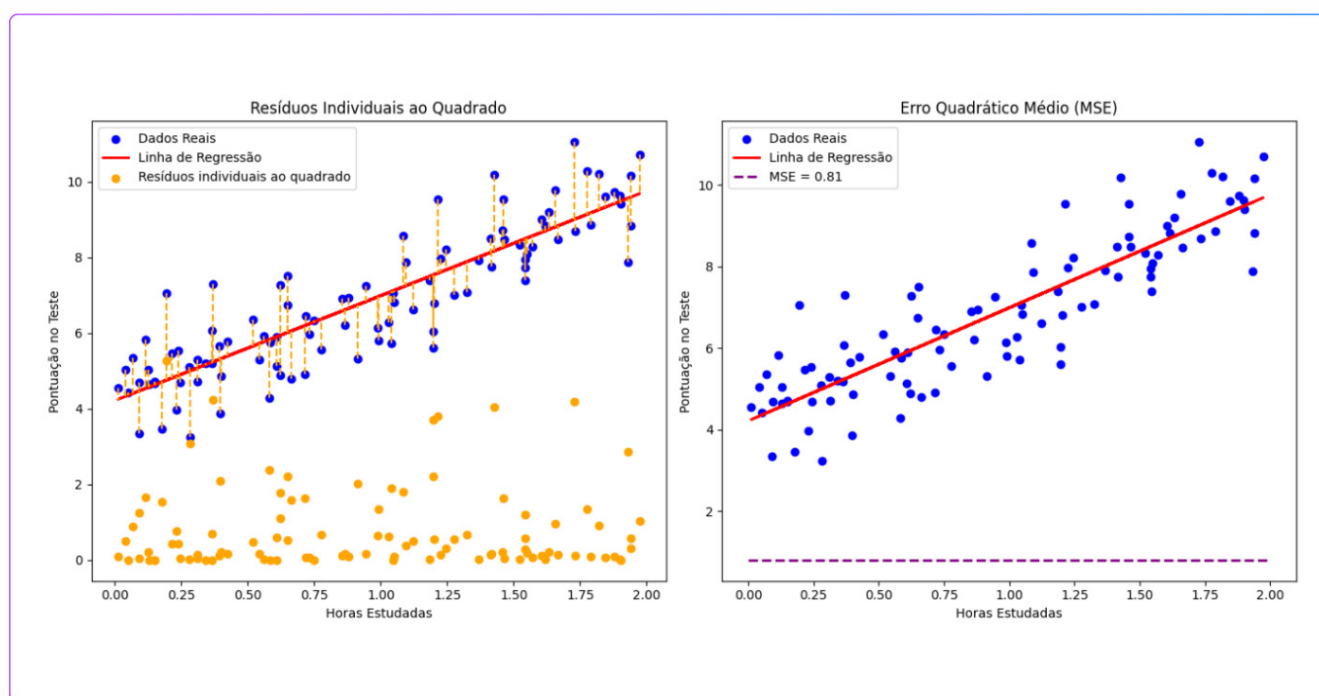
O MSE penaliza erros grandes mais severamente do que erros pequenos devido ao uso do quadrado das diferenças. Um MSE menor indica um modelo com melhor performance, enquanto um MSE maior indica um modelo com pior performance. De forma facilitar a compreensão, a figura abaixo ilustra o MSE. 1. No primeiro gráfico (Resíduos ao Quadrado), tem-se:

- » Os pontos azuis representam os dados reais.
- » A linha vermelha representa a linha de regressão ajustada pelo modelo.
- » As linhas tracejadas laranja e os pontos laranja representam os **erros individuais ao quadrado**, mostrando as diferenças entre os valores observados e preditos.

No segundo gráfico da figura abaixo (MSE), tem-se:

- » Os pontos azuis representam os dados reais.
- » A linha vermelha representa a linha de regressão ajustada pelo modelo.
- » A linha tracejada roxa horizontal representa o MSE, que é a média dos resíduos ao quadrado.

Figura 52 - Exemplo de erro quadrático médio (MSE)



Fonte: autoria própria e disponível no Google Colab.

8.1.3 Erro Absoluto Médio (MAE)

O MAE mede a magnitude média dos erros em um conjunto de previsões, sem considerar sua direção. É uma métrica linear, o que significa que todos os erros são ponderados igualmente. Quanto menor o MAE, melhor o modelo. Penaliza de forma linear todos os erros.

O Mean Absolute Error (MAE) é definido como:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

onde:

y_i são os valores reais,

$\hat{y}_i$ são os valores preditos pelo modelo,

n é o número de observações.

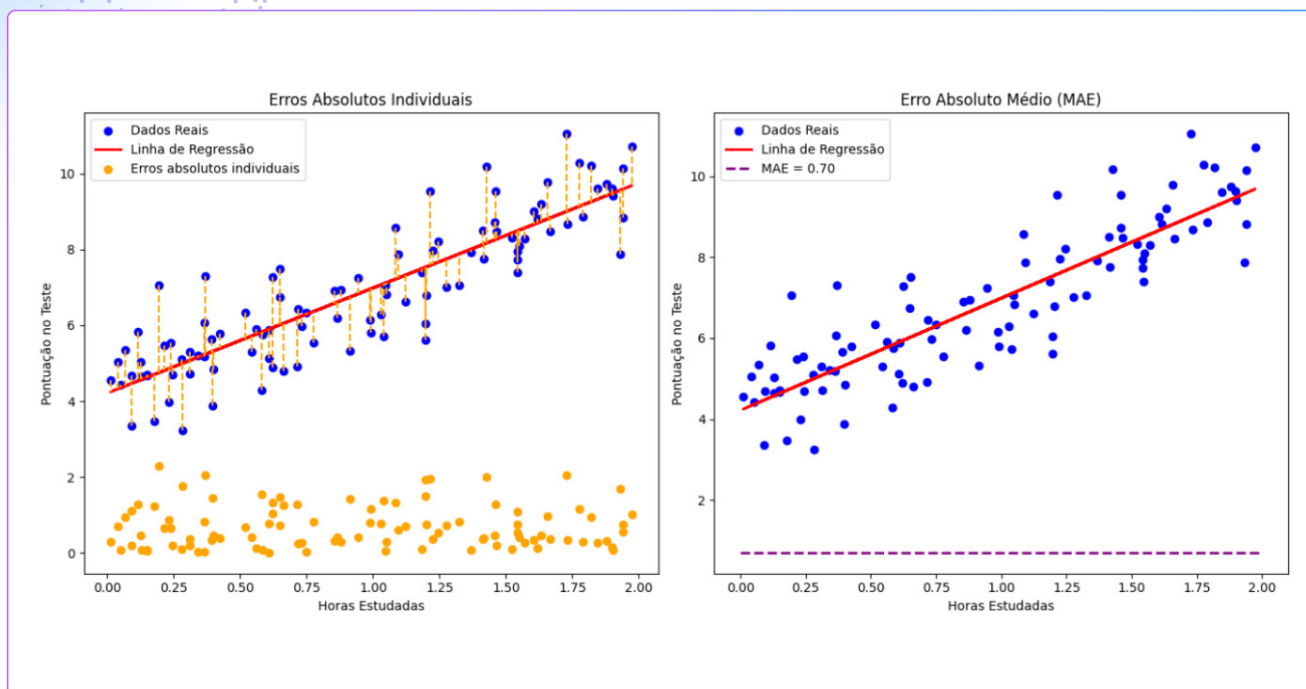
De forma facilitar a compreensão, a figura abaixo ilustra o MAE. No primeiro gráfico (Erros Absolutos Individuais) tem-se:

- » Os pontos azuis representam os dados reais.
- » A linha vermelha representa a linha de regressão ajustada pelo modelo.
- » As linhas tracejadas laranja e os pontos laranja representam os **erros absolutos individuais**, ou seja, as diferenças absolutas entre os valores observados e os valores preditos.

No segundo gráfico (MAE) tem-se:

- » Os pontos azuis representam os dados reais.
- » A linha vermelha representa a linha de regressão ajustada pelo modelo.
- » A linha tracejada roxa horizontal representa o valor do MAE, que é a média dos erros absolutos.

Figura 53 - Exemplo de erro absoluto médio (MAE)



Fonte: autoria própria e disponível no Google Colab.

8.1.4 Raiz do Erro Quadrático Médio (RMSE)

A raiz quadrada do MSE, traz a medida de erro para a mesma escala da variável dependente. Quanto menor o RMSE, melhor o modelo. Ela possui a mesma unidade dos dados originais e penaliza mais os erros grandes.

O *Root Mean Squared Error* (RMSE) é definido como:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

onde:

y_i são os valores reais,

$\hat{y}_i$ são os valores preditos pelo modelo,

n é o número de observações.

O RMSE penaliza mais severamente grandes erros devido ao uso do quadrado das diferenças, similar ao MSE, mas retorna os erros na mesma unidade dos valores observados.

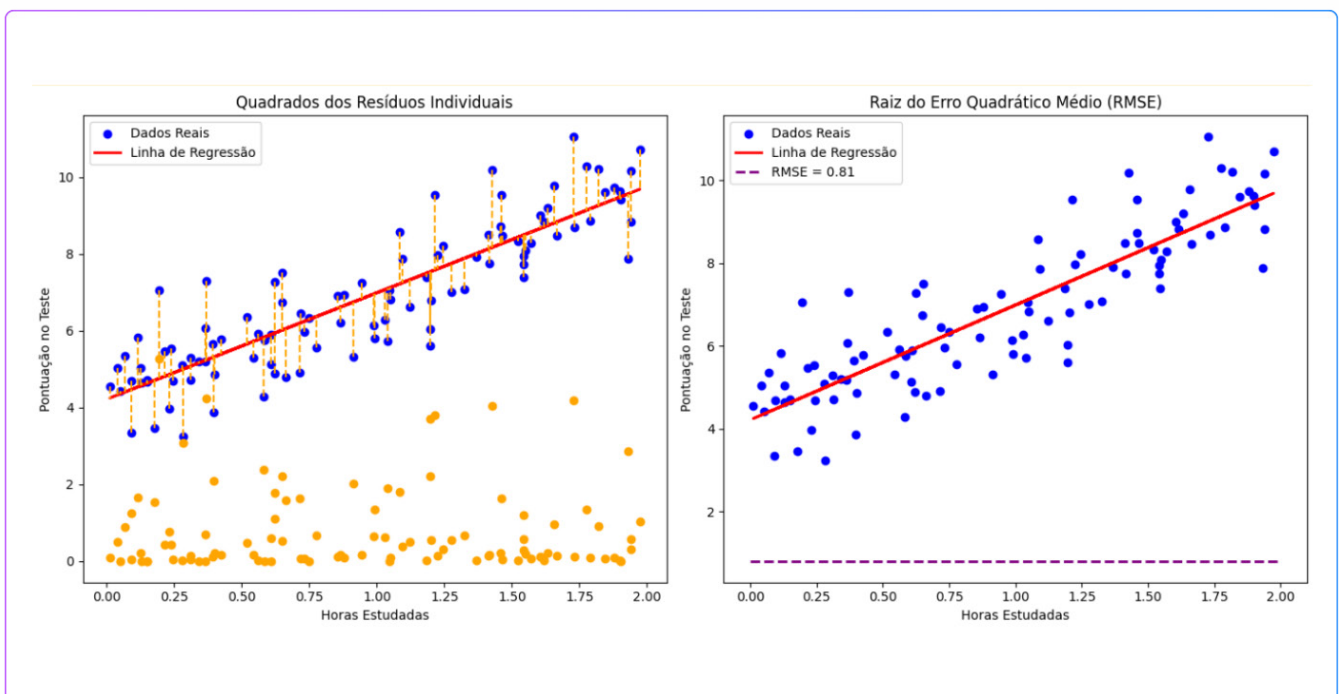
De forma facilitar a compreensão, a figura abaixo ilustra o MAE. No primeiro gráfico (Quadrados dos Resíduos Individuais) tem-se:

- » Os pontos azuis representam os dados reais.
- » A linha vermelha representa a linha de regressão ajustada pelo modelo.
- » As linhas tracejadas laranja e os pontos laranja representam os quadrados dos resíduos individuais, ou seja, as diferenças ao quadrado entre os valores observados e os valores preditos.

No segundo gráfico (RMSE), tem-se: A linha tracejada roxa horizontal representa o valor do RMSE, que é a raiz quadrada da média dos resíduos ao quadrado.

- » Os pontos azuis representam os dados reais.
- » A linha vermelha representa a linha de regressão ajustada pelo modelo.

Figura 54 - Exemplo de raiz do erro quadrático médio (RMSE)



Fonte: autoria própria e disponível no Google Colab.

No Quadro 2, é apresentado um resumo das vantagens e desvantagens das métricas acima explicadas. Há também a fórmula para cada uma na qual:

y_i são os valores reais

$\hat{y}_i$ são os valores preditos pelo modelo

n é o número de observações

Quadro 2 - Comparativo entre as métricas abordadas resume essa seção

MÉTRICA	VANTAGENS	DESVANTAGENS	FÓRMULA
R² (Coeficiente de Determinação)	Interpretação intuitiva como a proporção da variância explicada pelo modelo. Útil para comparar modelos diferentes.	Pode ser enganoso em modelos não-lineares ou com outliers. Não indica a qualidade do ajuste em termos absolutos.	$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)}{\sum_{i=1}^n (y_i - \bar{y})^2}$
MSE (Mean Squared Error)	Penaliza erros grandes mais severamente devido ao quadrado dos resíduos. Utilizado para otimização em muitos algoritmos de aprendizado de máquina.	Não é intuitivo, pois está na escala do quadrado da variável dependente. Sensível a outliers.	$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
MAE (Mean Absolute Error)	Interpretação intuitiva como o erro médio absoluto. Menos sensível a outliers comparado ao MSE.	Menos eficiente do ponto de vista matemático em certos algoritmos de otimização. Não penaliza erros grandes tão severamente quanto o MSE.	$MAE = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $
RMSE (Root Mean Squared Error)	Interpretação intuitiva, pois está na mesma escala da variável dependente. Penaliza erros grandes severamente como o MSE.	A penalização de erros grandes pode ser uma desvantagem em alguns contextos. Sensível a outliers.	$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$

Fonte: autoria própria.

8.2 Tipos de Regressão

A análise de regressão é uma técnica estatística fundamental utilizada para modelar a relação entre uma variável dependente e uma ou mais variáveis independentes. Existem vários tipos de modelos de regressão, cada um com suas características e aplicações específicas, algumas serão apresentadas nas próximas subseções.

8.2.1 Regressão Linear

Simple e múltipla, modela a relação linear entre variáveis. É o ponto de partida para muitos estudos de regressão. A regressão linear é um método estatístico para modelar

a relação entre uma variável dependente e uma ou mais variáveis independentes assumindo uma relação linear. A regressão linear simples assume a seguinte fórmula:

$$y = \beta_0 + \beta_1 X + \epsilon$$

onde:

y é a variável dependente.

X é a variável independente.

β_0 é o intercepto (valor de y quando $X = 0$).

β_1 é o coeficiente de inclinação (quanto y muda com uma unidade de mudança em X).

ϵ é o termo de erro (a diferença entre os valores observados e os valores preditos).

8.2.2 Regressão Ridge, Lasso e Elastic Net

A regressão linear é uma ferramenta poderosa para modelar relações entre variáveis. No entanto, quando lidamos com um grande número de preditores ou quando estes estão altamente correlacionados (multicolinearidade), o modelo pode se tornar instável e sobreajustar aos dados de treinamento, generalizando mal para novos dados. Para mitigar esses problemas, foram desenvolvidas técnicas de regularização, como *Ridge*, *Lasso* e *Elastic Net*.

Regressão Ridge (L2):

A Regressão *Ridge* utiliza a regularização L2. Ela adiciona um termo de penalidade baseado na norma L2 dos coeficientes à função de custo da regressão linear. Essa penalização faz com que o novo modelo não obtenha os melhores resultados no conjunto de treino, porém visa-se melhor desempenho no conjunto de teste. Sua fórmula:

$$\min(\|y - X\beta\|^2 + \lambda\|\beta\|^2)$$

Onde:

y é o vetor de variáveis dependentes

X é a matriz de variáveis independentes

β é o vetor de coeficientes

λ é o parâmetro de regularização

ou ainda:

$\|y - X\beta\|^2$: Esta parte é também conhecida como SSR que foi apresentada junto com a métrica R^2 , na qual:

y representa os valores observados;

$X\beta$ representa os valores previstos pelo modelo;

A diferença $(y - X\beta)$ representa os resíduos;

Ao elevar ao quadrado ($\|\dots\|^2$), obtemos a soma dos quadrados desses resíduos.

$\lambda\|\beta\|^2$: Esta é a parte da regularização L2 (*Ridge*), onde:

$\|\beta\|^2$ é a norma L2 ao quadrado dos coeficientes β ;

λ (lambda) é o parâmetro de regularização que controla a força da penalidade.

Algumas observações adicionais:

- » O termo RSS ($\|y - X\beta\|^2$) garante que o modelo se ajuste bem aos dados de treinamento.
- » O termo de regularização L2 ($\lambda\|\beta\|^2$) penaliza coeficientes grandes, incentivando o modelo a usar coeficientes menores e mais estáveis.
- » O parâmetro λ controla o equilíbrio entre estes dois objetivos. Um λ maior dá mais peso à regularização, enquanto um λ menor prioriza o ajuste aos dados.

Como principais características da regressão *Ridge* estão a redução do *overfitting* (sobreajuste) ao diminuir a magnitude dos coeficientes, a tendência de distribuir o peso entre todas as variáveis e sua eficácia quando há multicolinearidade entre as variáveis independentes.

Regressão Lasso (L1):

LASSO utiliza a regularização L1. Adiciona um termo de penalidade baseado na norma L1 dos coeficientes.

Fórmula:

$$\min(\|y - X\beta\|^2 + \lambda\|\beta\|_1)$$

Onde:

y é o vetor de variáveis dependentes

X é a matriz de variáveis independentes

β é o vetor de coeficientes

λ é o parâmetro de regularização

Como visto anteriormente, $\|y - X\beta\|^2$ refere-se ao SSR e $\lambda\|\beta\|_1$ é a norma L1. A principal diferença entre a L1 e L2 é que a segunda utiliza a norma ao quadrado. Pode-se entender β como uma inclinação que será adicionado à inclinação já existente do resultado do modelo linear.

No L1 o β está sob módulo, e no L2 ele é elevado o quadrado. Em ambos casos, o objetivo é evitar os valores negativos. A diferença é que para o L2, por menor que seja β , ele nunca poderá ser nulo, uma vez que sempre será elevado ao quadrado. Já para o L1, o módulo de 0 é 0, permitindo assim a seleção de variáveis.

Elastic Net:

O *Elastic Net* combina as regularizações L1 e L2, aproveitando as vantagens de ambas as técnicas.

Fórmula:

$$\min(\|y - X\beta\|^2 + \lambda_1\|\beta\|_1 + \lambda_2\|\beta\|^2)$$

Onde:

$\|y - X\beta\|^2$: Esta parte é também conhecida como SSR

$\lambda_1\|\beta\|_1$: componente da regularização *Lasso* (L1)

$\lambda_2\|\beta\|^2$: componente da regularização *Ridge* (L2)

Combina as propriedades do *Ridge* e do *Lasso*, pode selecionar variáveis (como o *Lasso*) e lidar com multicolinearidade (como o *Ridge*).

8.2.3 Árvores de Regressão

As árvores de regressão constituem uma técnica robusta e versátil no campo do aprendizado de máquina, especialmente adequada para problemas de regressão onde o objetivo é prever uma variável de saída contínua. Esta metodologia se distingue por sua capacidade de capturar relações não-lineares complexas entre variáveis preditoras e a variável alvo, oferecendo uma alternativa valiosa aos modelos lineares tradicionais.

A estrutura fundamental de uma árvore de regressão assemelha-se a um fluxograma invertido, onde cada nó interno representa uma decisão baseada em uma característica específica, e cada nó folha contém uma previsão numérica. O processo de construção da árvore envolve a divisão recursiva do conjunto de dados em subconjuntos cada vez menores, com base em regras de decisão que maximizam a homogeneidade dentro de cada subgrupo resultante. Um exemplo de árvore de regressão é ilustrado abaixo. Uma árvore de regressão é composta por:

- » **Nós de decisão:** Pontos onde o algoritmo faz uma escolha baseada em uma característica.
- » **Ramos:** Caminhos que conectam os nós, representando os resultados das decisões.
- » **Nós folha:** Pontos finais que contêm as previsões numéricas.

O algoritmo constroi a árvore de cima para baixo, seguindo estas etapas:

- » **Seleção da Melhor Característica:** O algoritmo examina todas as características disponíveis. Para cada característica, ele calcula uma métrica de “impureza” (geralmente a variância) para todas as possíveis divisões. A característica que resulta na maior redução da impureza é selecionada.
- » **Determinação do Ponto de Divisão:** Para a característica selecionada, o algoritmo determina o valor ótimo para dividir os dados. Este ponto de divisão minimiza a variância dentro dos subgrupos resultantes.
- » **Divisão do Nó:** O conjunto de dados no nó atual é dividido em dois subconjuntos com base na regra de decisão estabelecida.
- » **Recursão:** o processo é repetido para cada nó filho até que um critério de parada seja atingido.

A construção da árvore para quando:

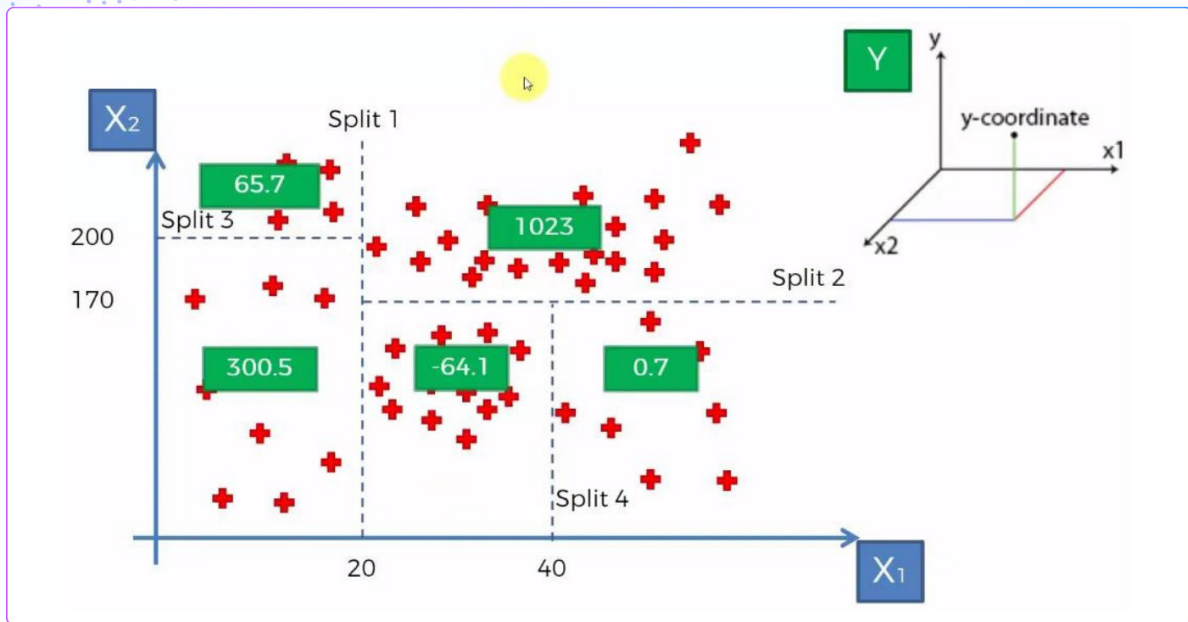
- » Um nó contém um número mínimo pré definido de amostras;
- » A profundidade máxima da árvore é alcançada;
- » A redução na impureza cai abaixo de um limiar especificado.

Para fazer uma previsão, um novo dado é passado pela árvore, seguindo as regras de decisão em cada nó. Quando atinge um nó folha, o valor associado a esse nó é a previsão.

As figuras abaixo apresentam um exemplo da aplicação de uma árvore de regressão. Na Figura 39 tem-se:

- » **Eixos X1 e X2:** Representam as duas variáveis preditoras utilizadas para construir a árvore.
- » **Pontos Vermelhos:** Cada ponto representa uma observação (exemplo) no conjunto de dados. A posição do ponto no gráfico indica os valores das variáveis preditoras para aquela observação.
- » **Linhas Verticais e Horizontais:** Representam os *splits* (divisões) realizados pela árvore de regressão. Cada *split* divide o espaço de características em duas regiões, criando nós filhos.
- » **Caixas Verdes:** Cada caixa representa uma região do espaço de características e contém um valor numérico. Esse valor é a previsão média da variável alvo (Y) para todas as observações que caem nessa região.

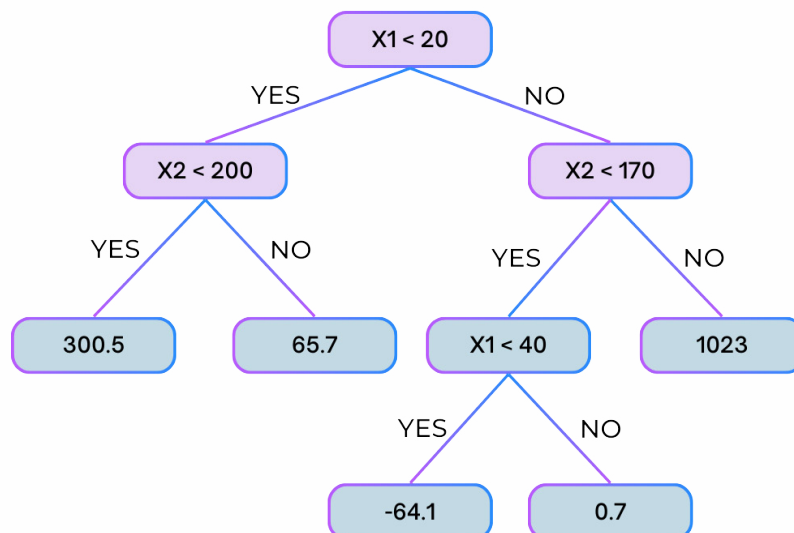
Figura 55 - Exemplo de árvore de regressão - dados



Fonte: [Medium, 2019](#).

Com base nisso, a Figura 40 pode ser interpretada da seguinte maneira. Se X_1 é menor que 20, verifica-se se o valor de X_2 é menor que 200. Se sim, então será retornado 300.5, que é o valor predito para o grupo de dados contidos à esquerda do *split* 1 e abaixo do *split* 3. Se o X_2 for maior que 200, então será retornado 65.7, que é o valor predito para o grupo de dados contidos à esquerda do *split* 1 e acima do *split* 3.

Figura 56 - Exemplo de árvore de regressão - árvore



Fonte: adaptada de [Medium \(2019\)](#).

As árvores de regressão oferecem diversas vantagens:

- » **Interpretabilidade:** A estrutura hierárquica permite uma fácil visualização e compreensão das decisões do modelo.
- » **Capacidade de lidar com não-linearidades:** Capturam eficientemente relações complexas e interações entre variáveis.
- » **Robustez a outliers:** São menos sensíveis a valores atípicos em comparação com métodos baseados em mínimos quadrados.
- » **Tratamento automático de variáveis categóricas:** Não requerem codificação prévia de variáveis categóricas.

Contudo, também apresentam algumas limitações:

- » **Tendência ao overfitting:** Árvores muito profundas podem se ajustar excessivamente aos dados de treinamento.
- » **Instabilidade:** Pequenas variações nos dados podem resultar em árvores significativamente diferentes.
- » **Dificuldade em capturar relações aditivas:** Podem ser menos eficientes que modelos lineares para relações puramente aditivas.

As árvores de regressão encontram aplicações em diversos domínios, incluindo: previsão de preços imobiliários, estimativa de demanda de produtos, avaliação de risco de crédito, previsão de consumo de energia.

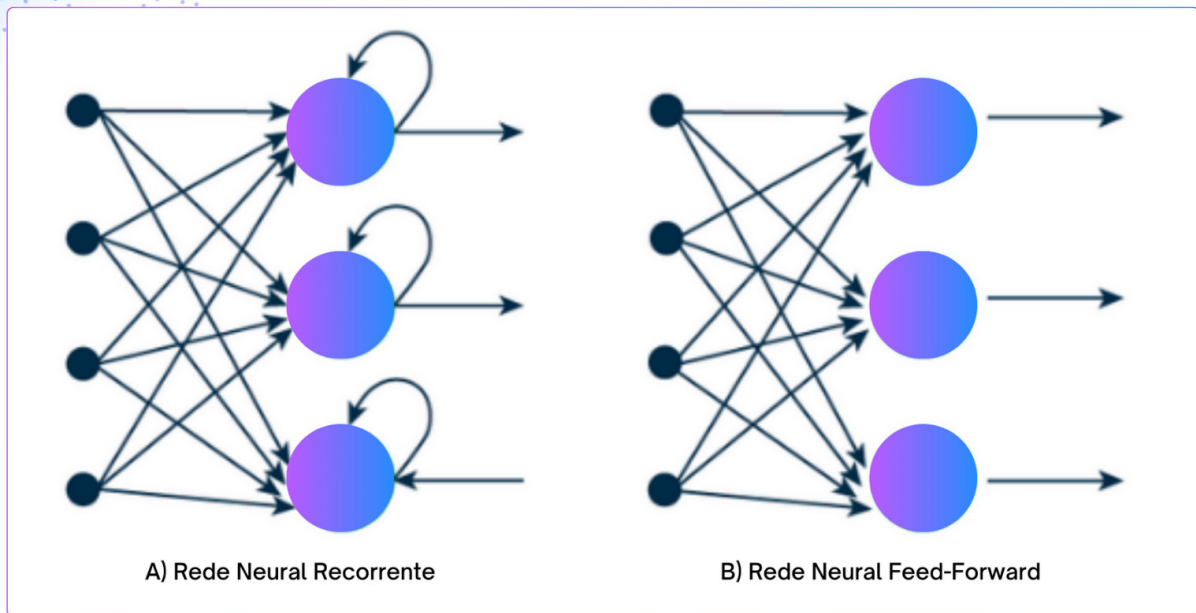
8.2.4 Recurrent Neural Network (RNN) ou Rede Neural Recorrente

As RNNs representam uma classe sofisticada de arquiteturas de RNAs, especificamente concebidas para lidar com dados sequenciais. Esta estrutura inovadora revolucionou diversas áreas da IA, particularmente aquelas que envolvem o processamento de informações temporais ou ordenadas.

A característica distintiva das RNNs reside em sua estrutura cíclica. Diferentemente das redes neurais *feedforward* tradicionais que serão detalhadas na unidade específica, as RNNs incorporam conexões de realimentação, permitindo que a informação persista ao longo do tempo. Esta propriedade confere à rede uma forma de “memória”, capacitando-a a considerar contextos anteriores na geração de saídas atuais.

Na figura abaixo é ilustrado à direita uma rede neural *feedforward*, sem recorrência. À esquerda é ilustrada a RNN, destacando-se pela seta adicional que retorna para a mesma camada de origem.

Figura 57 - Comparação entre redes neurais, RNN e Feed-Forward



Fonte: Adaptada de [GeekesForgeeks, 2024](#).

Na rede neural *feedforward* a informação flui em uma única direção, das camadas de entrada para as camadas de saída. Cada neurônio em uma camada só recebe informações da camada anterior e não possui conexões com neurônios em camadas anteriores. Isso limita a capacidade da rede de processar sequências de dados, onde o contexto anterior é importante.

Na RNN os neurônios em uma camada recebem informações não apenas da camada anterior, mas também de si mesmos em um tempo anterior. Isso cria um “loop” que permite à rede “lembrar” informações passadas. A presença de conexões recorrentes permite que a RNN capture dependências temporais nos dados, tornando-as ideais para processar sequências como texto, séries temporais e fala.

O treinamento de RNNs emprega uma variante do algoritmo de retropropagação, denominada *Backpropagation Through Time* (BPTT). Este método “desdobra” a RNN em uma rede *feedforward* profunda equivalente, permitindo a aplicação de técnicas de otimização baseadas em gradiente.

As RNNs encontram aplicações em uma vasta gama de domínios incluindo PLN, com tradução automática, geração de texto e análise de sentimentos. No reconhecimento de fala, na conversão de fala para texto e em sistemas de comando por voz. Na análise de Séries Temporais, com a previsão de demanda e análise de mercado financeiro. Na bioinformática com a análise de sequências de ácido desoxirribonucleico (DNA) ou previsão de estruturas proteicas.

Apesar de sua potência, as RNNs apresentam desafios significativos como: desvanecimento e explosão do gradiente. O gradiente é como um sinal que diz à rede neural como ela deve ajustar seus “pesos” (parâmetros internos) para melhorar suas previsões. Este sinal viaja de trás para frente na rede durante o treinamento. Quando trabalha-se com sequências longas em RNNs, esse gradiente precisa percorrer um caminho muito longo, causando o desvanecimento ou explosão do gradiente. Como consequência dis-

so, a rede tem dificuldade em aprender de eventos que ocorreram muito tempo atrás na sequência, ou os ajustes na rede se tornam muito grandes, fazendo com que o treinamento se torne instável. Além disso, o processamento sequencial pode ser computacionalmente intensivo, especialmente para sequências muito longas.

8.3 Notebook Colab

No [link do colab](#) serão abordados exemplos didáticos apresentando o cálculo manual e utilizando as bibliotecas disponíveis, bem serão criadas diversas ilustrações para fixar a aprendizagem. Dentre os conteúdos abordados nesse notebook incluem-se as métricas R^2 , MSE, MAE, RMSE, um exemplo detalhado de uso do modelo regressão linear e a aplicação direta dos modelos de regressão *ridge*, árvores de regressão e RNN.

Definição e Importância da Regressão em Análise de Dados e *Machine Learning*

A regressão é uma técnica estatística amplamente utilizada para modelar a relação entre uma variável dependente contínua e uma ou mais variáveis independentes. Em análise de dados e *machine learning*, a regressão é fundamental para previsões quantitativas, permitindo que analistas e cientistas de dados façam inferências e estimativas baseadas em padrões observados nos dados. A importância da regressão reside na sua aplicabilidade em diversos campos, desde economia e finanças, onde é usada para prever valores de mercado, até na indústria, onde pode otimizar processos e prever demandas.

Distinção entre Problemas de Regressão e Classificação

Enquanto a regressão foca na predição de variáveis contínuas, como preços, temperaturas ou índices, a classificação é empregada para atribuir instâncias a categorias discretas. Por exemplo, determinar se um e-mail é spam ou não é um problema de classificação binária. Já prever a temperatura máxima para o próximo dia é um problema típico de regressão. A distinção entre esses dois tipos de problemas é crucial, pois os métodos e métricas de avaliação são distintos e a escolha do modelo adequado depende dessa compreensão.

Métricas de Regressão: R^2 , MSE, MAE e RMSE com Python

Objetivos

- » Compreender as métricas de desempenho R^2 , MSE, MAE e RMSE em modelos de regressão.
- » Aprender a calcular e interpretar essas métricas utilizando Python.
- » Implementar exemplos práticos de uso dessas métricas em modelos de regressão.

Introdução

Em modelos de regressão, é fundamental avaliar a performance do modelo para en-

tender quão bem ele está se ajustando aos dados e para fazer previsões confiáveis. Existem diversas métricas para medir a qualidade de um modelo de regressão.

Para avaliar a performance de modelos de regressão, várias métricas são utilizadas:

- » **R² (Coeficiente de Determinação)**: Mede a proporção da variabilidade da variável dependente que é explicada pelas variáveis independentes no modelo.
- » **MSE (Erro Quadrático Médio)**: Média dos quadrados dos erros, fornece uma medida da variância dos erros.
- » **MAE (Erro Absoluto Médio)**: Média dos erros absolutos, sendo menos sensível a outliers que o MSE.
- » **RMSE (Raiz do Erro Quadrático Médio)**: Raiz quadrada do MSE, traz a medida de erro para a mesma escala da variável dependente.

Exemplos Práticos

Carregando Bibliotecas e Dados

Vamos criar um conjunto de dados fictício para ilustrar o cálculo do R². Suponhamos que estamos analisando a relação entre a quantidade de horas estudadas (variável independente) e a pontuação em um teste (variável dependente).

```
# Importação das bibliotecas que serão utilizadas
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
# Dependendo da versão do sklearn instalado pode ser que a linha abaixo apresente
problemas
# Essa linha é para uma versão mais recente, porém no colab ela será comentada para
evitar erros
# from sklearn.metrics import root_mean_squared_error

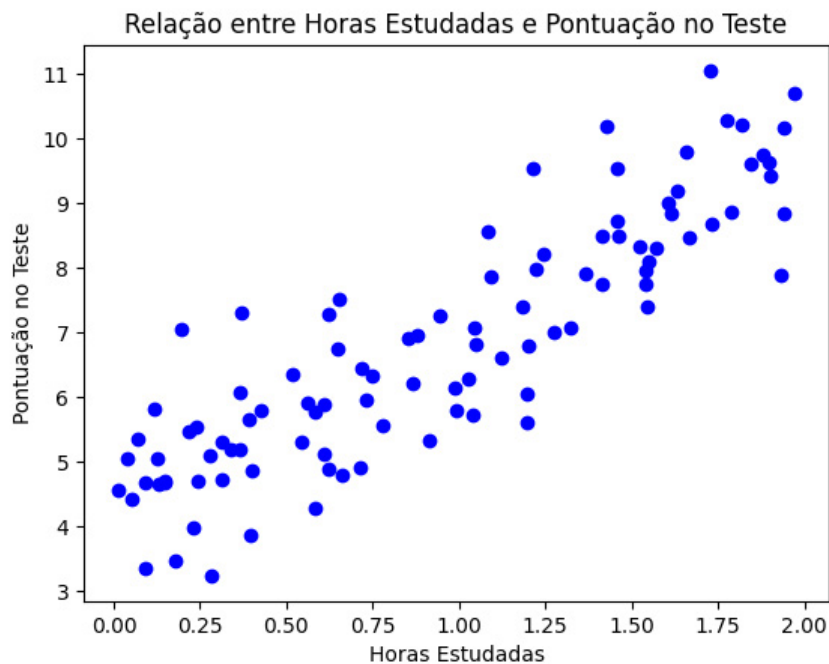
# Gerar dados fictícios
# o random.seed é necessário para manter a reprodutibilidade dos dados.
# ele gera sempre os mesmos números aleatórios
np.random.seed(42)
# Em X são gerados um vetor de 100 dados aleatórios com base no random.seed
X = 2 * np.random.rand(100, 1)
# Em y são gerados outro vetor com 100 dados aleatórios baseados em X e um ruído
y = 4 + 3 * X + np.random.randn(100, 1)

# Plotar os dados para facilitar a visualização
# gera um gráfico de pontos azuis com os valores reais dos pontos
plt.scatter(X, y, color='blue')
# Defini-se o título
plt.title('Relação entre Horas Estudadas e Pontuação no Teste')
# Defini-se o texto do eixo x
plt.xlabel('Horas Estudadas')
# Defini-se o texto do eixo y
```

continua

```
plt.ylabel('Pontuação no Teste')
# Mostra a figura
plt.show()
```

Figura 58 - Relação entre Horas Estudadas e Pontuação no Teste



Fonte: autoria própria.

Treinando o Modelo de Regressão Linear

Vamos ajustar um modelo de regressão linear simples aos dados.

Para utilizar as métricas é necessário ter dados reais e preditos, para isso será usado um simples modelo de regressão linear, que fará a predição de valores de **y** com base nos valores de **X**.

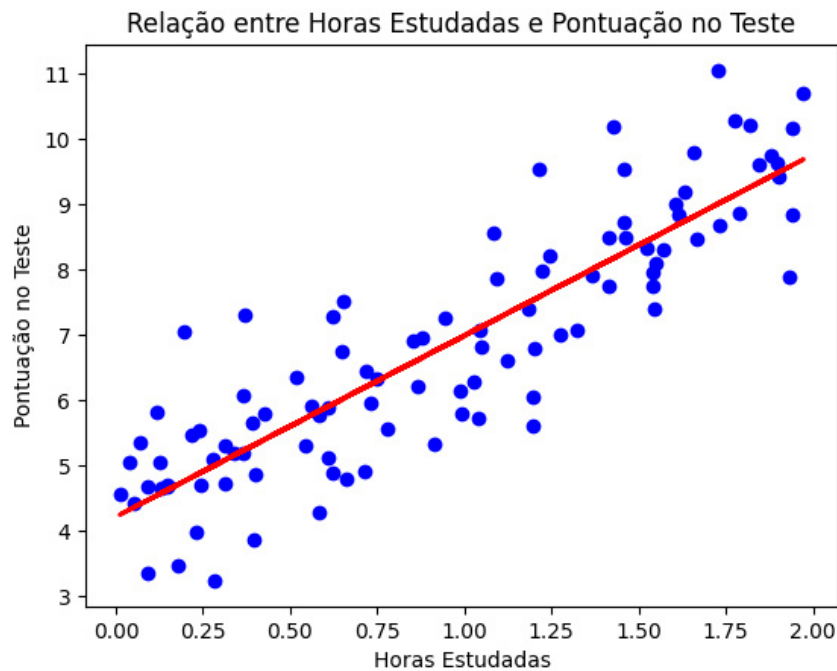
Esses valores preditos serão guardados no **y_pred**.

Ao fim, é plotado em vermelho o resultado desse modelo de predição. Em pontos na cor azul estão ilustrados os dados reais.

```
# Ajustar o modelo de regressão linear
# Cria um modelo de regressão linear
model = LinearRegression()
# Treina o modelo com base nos valores de X e y
model.fit(X, y)
# Utilizando o modelo já treinado, faz a predição dos valores de X e guarda em y_pred
y_pred = model.predict(X)

# Plotar os dados e a linha de regressão
plt.scatter(X, y, color='blue')
plt.plot(X, y_pred, color='red', linewidth=2)
plt.title('Relação entre Horas Estudadas e Pontuação no Teste')
plt.xlabel('Horas Estudadas')
plt.ylabel('Pontuação no Teste')
plt.show()
```

Figura 59 - Relação entre Horas Estudadas e Pontuação no Teste - Regressão Linear



Fonte: autoria própria.

Explicações Teóricas com Exemplos práticos

R^2 (Coeficiente de Determinação)

- » **Definição:** Mede a proporção da variância dos dados dependentes que é explicada pelo modelo de regressão.
- » **Interpretação:** Varia de 0 a 1. Um valor de 1 indica que o modelo explica toda a variância dos dados, enquanto um valor de 0 indica que o modelo não explica nenhuma variância.
- » **Função:** pode ser utilizado o pacote do sklearn `r2 = r2_score(y, y_pred)`

Explicação Detalhada do R^2

O coeficiente de Determinação R^2 é calculado como:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

onde:

y_i são os valores reais,

$\hat{y}_i$ são os valores preditos pelo modelo,

$\bar{y}$ é a média dos valores reais,

$\sum_{i=1}^n (y_i - \hat{y}_i)^2$ é a Soma dos Quadrados dos Resíduos (SSR),

$$\sum_{i=1}^n (y_i - \bar{y})^2 \text{ é a Soma Total dos Quadrados (SST).}$$

O valor de R^2 varia entre 0 e 1:

$R^2 = 0$ indica que o modelo não explica nenhuma variação nos dados.

$R^2 = 1$ indica que o modelo explica toda a variação nos dados.

Abaixo serão apresentados duas maneiras de cálculo do R^2 , uma manualmente utilizando a formulação matemática acima e outra mais direta, utilizando o pacote do sklearn.

Como é de se esperar, em ambos casos os valores encontrados são os mesmos.

```
# Calculando manualmente
# Média dos valores reais
y_mean = np.array([np.mean(y)])

# Calcular os componentes da fórmula do R²
ssr = np.sum((y - y_pred) ** 2)
sst = np.sum((y - y_mean) ** 2)

# Cálculo do R² utilizando a fórmula
r2_manual = 1 - (ssr/sst)

# Impressão do resultado
print(f"Coefficient of Determination (R²): {r2_manual:.2f}")

Coefficient of Determination (R²): 0.77

# Calcular o R²
# Passa-se os valores reais (y), os valores obtidos do modelo (y_pred)
# o valor do erro é guardado em r2
r2 = r2_score(y, y_pred)
print(f"Coefficient of Determination (R²): {r2:.2f}")

Coefficient of Determination (R²): 0.77
```

Visualização da Variabilidade Explicada

Este código permite visualizar graficamente como um modelo de regressão se ajusta aos dados e qual a proporção da variabilidade dos dados que o modelo consegue explicar. Apresentam-se a variabilidade não explicada pelo modelo (SSR) e a variabilidade Total (SST). Esses conceitos ajudam a entender melhor a fórmula do R^2 .

Variabilidade Não Explicada (SSR): As linhas pontilhadas laranja representam os erros do modelo, ou seja, a diferença entre o valor real (ponto em azul) e sua projeção ao

modelo (linha vermelha). Quanto menores forem essas linhas, melhor o modelo se ajusta aos dados.

Variabilidade Total (SST): A linha horizontal verde representa a média dos dados. A distância entre os pontos e a linha representa a variabilidade total dos dados.

O primeiro gráfico mostra a variabilidade que o modelo não consegue explicar. Quanto menor a distância entre os pontos e a linha de regressão, melhor o modelo se ajusta aos dados. O segundo gráfico mostra a variabilidade total dos dados. Comparando os dois gráficos, podemos ter uma ideia de quanto da variabilidade total está sendo explicada pelo modelo.

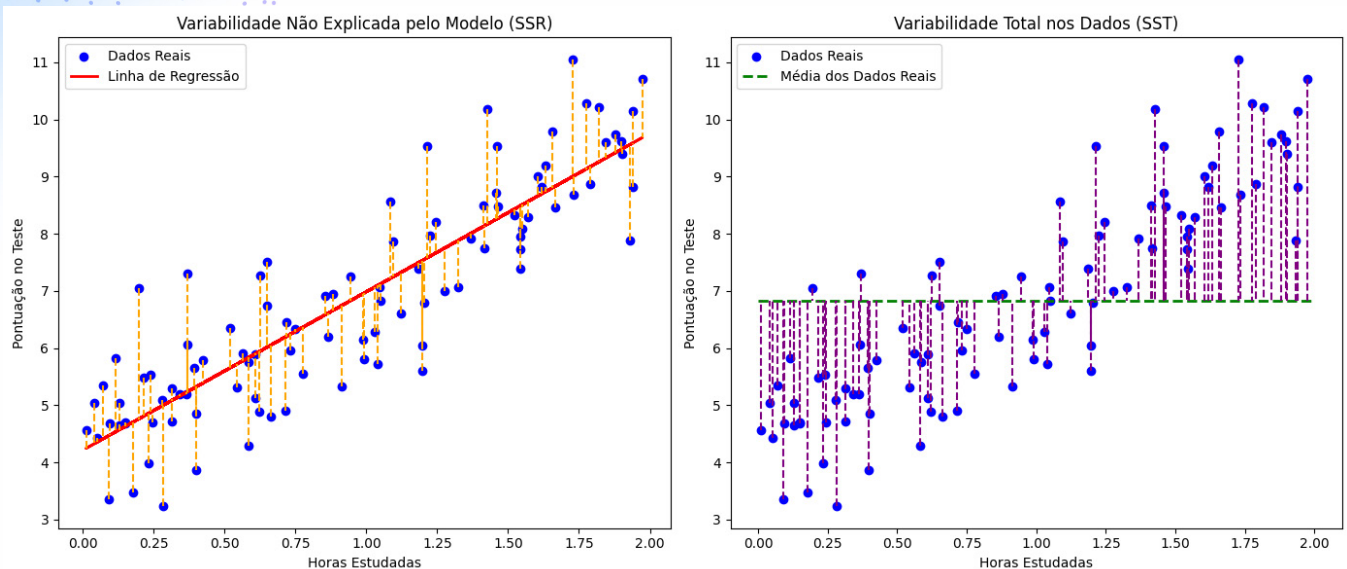
```
# Criar as subplots
fig, axs = plt.subplots(1, 2, figsize=(14, 6))

# Plotar a variabilidade não explicada (SSR)
axs[0].scatter(X, y, color='blue', label='Dados Reais')
axs[0].plot(X, y_pred, color='red', linewidth=2, label='Linha de Regressão')
for i in range(len(X)):
    axs[0].plot([X[i], X[i]], [y[i], y_pred[i]], color='orange', linestyle='dashed')
axs[0].set_title('Variabilidade Não Explicada pelo Modelo (SSR)')
axs[0].set_xlabel('Horas Estudadas')
axs[0].set_ylabel('Pontuação no Teste')
axs[0].legend()

# Plotar a variabilidade total (SST)
axs[1].scatter(X, y, color='blue', label='Dados Reais')
axs[1].hlines(y=y_mean, xmin=0, xmax=2, color='green', linestyle='dashed',
linewidth=2, label='Média dos Dados Reais')
for i in range(len(X)):
    axs[1].plot([X[i], X[i]], [y[i], y_mean], color='purple', linestyle='dashed')
axs[1].set_title('Variabilidade Total nos Dados (SST)')
axs[1].set_xlabel('Horas Estudadas')
axs[1].set_ylabel('Pontuação no Teste')
axs[1].legend()

# Mostrar os gráficos
plt.tight_layout()
plt.show()
```

Figura 60 - Gráfico SSR / SST



Fonte: autoria própria.

MSE (Mean Squared Error)

- » **Definição:** É a média dos quadrados dos erros entre os valores previstos e os valores reais.
- » **Interpretação:** Quanto menor o MSE, melhor o modelo. Penaliza mais os erros grandes do que os erros pequenos.
- » **Função:** pode ser utilizado o pacote do sklearn `mse = mean_squared_error(y, y_pred)`

Explicação Detalhada do MSE

O Mean Squared Error (MSE) é uma métrica utilizada para avaliar a performance de um modelo de regressão. Ele calcula a média dos quadrados das diferenças entre os valores observados (reais) e os valores preditos pelo modelo. A fórmula do MSE é:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

onde:

y_i são os valores reais,

$\hat{y}_i$ são os valores preditos pelo modelo,

n é o número de observações.

O MSE penaliza erros grandes mais severamente do que erros pequenos devido ao uso do quadrado das diferenças. Um MSE menor indica um modelo com melhor performance, enquanto um MSE maior indica um modelo com pior performance.

Abaixo serão apresentadas duas maneiras de cálculo do MSE, uma manualmente utilizando a formulação matemática acima e outra mais direta, utilizando o pacote do sklearn.

Como é de se esperar, em ambos casos os valores encontrados são os mesmos.

```
# Calculando manualmente

# Calcular a soma dos quadrados dos resíduos (SSR)
ssr = np.sum((y - y_pred) ** 2)
# Médias dos valores
mse_manual = ssr/len(y)

print(f"Mean Squared Error (MSE): {mse_manual:.2f}")
Mean Squared Error (MSE): 0.81

# Calcular o MSE
# Passa-se os valores reais (y), os valores obtidos do modelo (y_pred)
# o valor do erro é guardado em mse
mse = mean_squared_error(y, y_pred)
print(f'Mean Squared Error (MSE): {mse:.2f}')
Mean Squared Error (MSE): 0.81
```

Visualização dos Resíduos ao quadrado e MSE

Para ilustrar melhor, podemos plotar os resíduos ao quadrado e o MSE obtido pelo modelo.

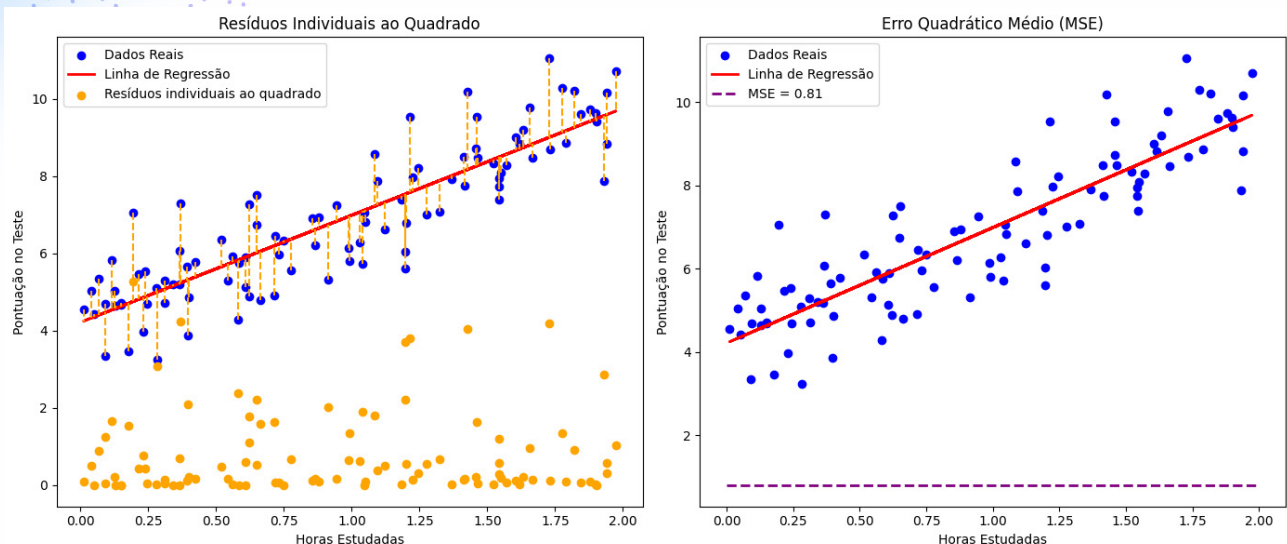
```
# Criar as subplots
fig, axs = plt.subplots(1, 2, figsize=(14, 6))

# Plotar os resíduos individuais ao quadrado
axs[0].scatter(X, y, color='blue', label='Dados Reais')
axs[0].plot(X, y_pred, color='red', linewidth=2, label='Linha de Regressão')
for i in range(len(X)):
    axs[0].plot([X[i], X[i]], [y[i], y_pred[i]], color='orange', linestyle='dashed')
axs[0].scatter(X, (y - y_pred)**2, color='orange', label='Resíduos individuais ao quadrado')
axs[0].set_title('Resíduos Individuais ao Quadrado')
axs[0].set_xlabel('Horas Estudadas')
axs[0].set_ylabel('Pontuação no Teste')
axs[0].legend()

# Plotar o MSE
axs[1].scatter(X, y, color='blue', label='Dados Reais')
axs[1].plot(X, y_pred, color='red', linewidth=2, label='Linha de Regressão')
axs[1].hlines(y=mse, xmin=0, xmax=2, color='purple', linestyle='dashed', linewidth=2,
label=f'MSE = {mse:.2f}')
axs[1].set_title('Erro Quadrático Médio (MSE)')
axs[1].set_xlabel('Horas Estudadas')
axs[1].set_ylabel('Pontuação no Teste')
axs[1].legend()

# Mostrar os gráficos
plt.tight_layout()
plt.show()
```

Figura 61 - Gráfico dos Resíduos ao quadrado e MSE



Fonte: autoria própria.

Explicação dos Gráficos

1. Primeiro Gráfico (Resíduos ao Quadrado):

- » Os pontos azuis representam os dados reais.
- » A linha vermelha representa a linha de regressão ajustada pelo modelo.
- » As linhas tracejadas laranja e os pontos laranja representam os erros individuais ao quadrado, mostrando as diferenças entre os valores observados e preditos.

2. Segundo Gráfico (MSE):

- » Os pontos azuis representam os dados reais.
- » A linha vermelha representa a linha de regressão ajustada pelo modelo.
- » A linha tracejada roxa horizontal representa plota o Erro Quadrático Médio (MSE), que é a média dos resíduos ao quadrado.

Mean Absolute Error (MAE)

- » **Definição:** É a média dos valores absolutos dos erros entre os valores previstos e os valores reais.
- » **Interpretação:** Quanto menor o MAE, melhor o modelo. Penaliza de forma linear todos os erros.
- » **Função:** pode ser utilizado o pacote do sklearn `mae = mean_absolute_error(y, y_pred)`

O MAE mede a magnitude média dos erros em um conjunto de previsões, sem considerar sua direção. É uma métrica linear, o que significa que todos os erros são ponderados igualmente.

Explicação Detalhada do MAE

O Mean Absolute Error (MAE) é definido como:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

onde:

y_i são os valores reais,

$\hat{y}_i$ são os valores preditos pelo modelo,

n é o número de observações.

Abaixo serão apresentados duas maneiras de cálculo do MAE, uma manualmente utilizando a formulação matemática acima e outra mais direta, utilizando o pacote do sklearn.

Como é de se esperar, em ambos casos os valores encontrados são os mesmos.

```
# Cálculo manual do MAE
err_abs = abs(y - y_pred)
# Somam-se os erros
sum_erros = np.sum(err_abs)
# Calcula o MAE
mae_manual = sum_erros/len(y)
# Impressão do resultado obtido
print(f'Mean Absolute Error (MAE): {mae_manual:.2f}')
```

Mean Absolute Error (MAE): 0.70

```
# Calcular o MAE com o uso do sklearn
# Passa-se os valores reais (y), os valores obtidos do modelo (y_pred)
# o valor do erro é guardado em mae
mae = mean_absolute_error(y, y_pred)
print(f'Mean Absolute Error (MAE): {mae:.2f}')
```

Mean Absolute Error (MAE): 0.70

Visualização dos Erros Absolutos e do MAE

Para ilustrar melhor, podemos plotar os resíduos absolutos e o MAE obtido pelo modelo.

```
# Criar as subplots
fig, axs = plt.subplots(1, 2, figsize=(14, 6))

# Plotar os erros absolutos individuais
axs[0].scatter(X, y, color='blue', label='Dados Reais')
axs[0].plot(X, y_pred, color='red', linewidth=2, label='Linha de Regressão')
for i in range(len(X)):
```

continua

```

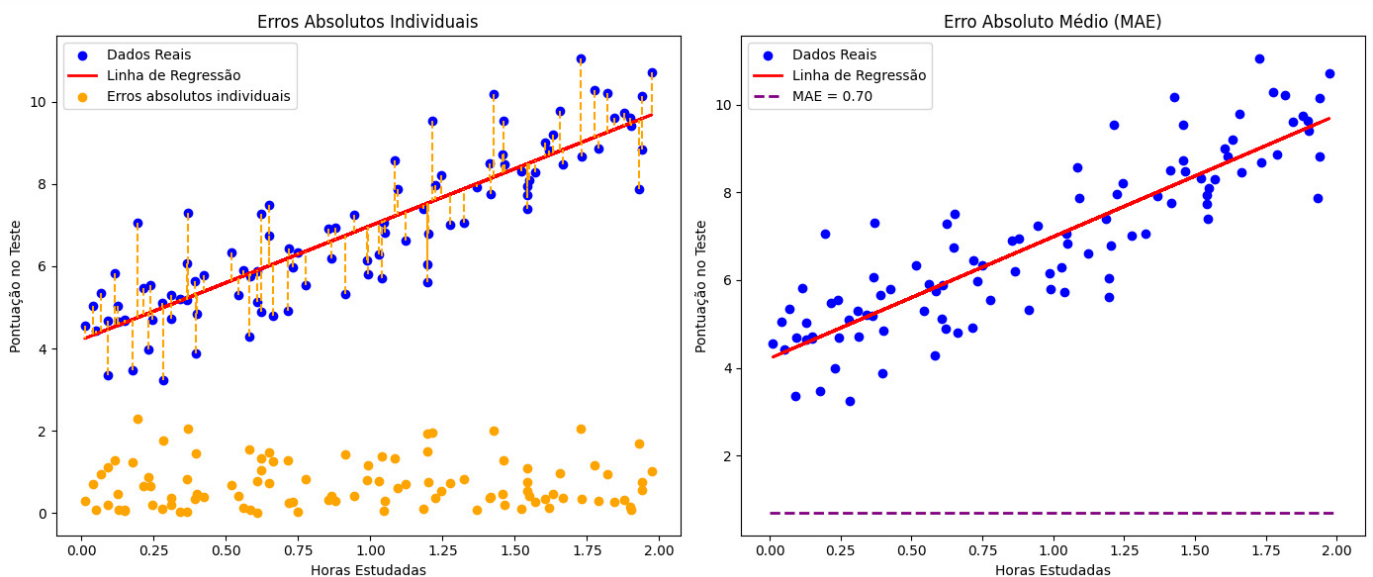
    axs[0].plot([X[i], X[i]], [y[i], y_pred[i]], color='orange', linestyle='dashed')
    axs[0].scatter(X, abs(y - y_pred), color='orange', label='Erros absolutos individuais')
    axs[0].set_title('Erros Absolutos Individuais')
    axs[0].set_xlabel('Horas Estudadas')
    axs[0].set_ylabel('Pontuação no Teste')
    axs[0].legend()

# Plotar o MAE
axs[1].scatter(X, y, color='blue', label='Dados Reais')
axs[1].plot(X, y_pred, color='red', linewidth=2, label='Linha de Regressão')
axs[1].hlines(y=mae, xmin=0, xmax=2, color='purple', linestyle='dashed', linewidth=2,
label=f'MAE = {mae:.2f}')
axs[1].set_title('Erro Absoluto Médio (MAE)')
axs[1].set_xlabel('Horas Estudadas')
axs[1].set_ylabel('Pontuação no Teste')
axs[1].legend()

# Mostrar os gráficos
plt.tight_layout()
plt.show()

```

Figura 62 - Gráfico dos Erros Absolutos e do MAE



Fonte: autoria própria.

Explicação dos Gráficos

1. Primeiro Gráfico (Erros Absolutos Individuais):

- » Os pontos azuis representam os dados reais.
- » A linha vermelha representa a linha de regressão ajustada pelo modelo.
- » As linhas tracejadas laranja e os pontos laranja representam os erros absolutos individuais, ou seja, as diferenças absolutas entre os valores observados e os valores preditos.

2. Segundo Gráfico (MAE):

- » Os pontos azuis representam os dados reais.
- » A linha vermelha representa a linha de regressão ajustada pelo modelo.
- » A linha tracejada roxa horizontal representa o valor do MAE, que é a média dos erros absolutos.

Para pensar:

Pergunta: Se o MAE utiliza os erros absolutos e o MSE os mesmos erros porém ao quadrado, por que o valor do MAE foi superior ao do MSE?

Resposta: Compare os pontos em laranja dos dois gráficos (MAE e MSE), note como ao elevar o quadrado (gráfico MSE), erros grandes ficam muito maiores ($10^2=100$), e erros pequenos ficam muito menores ($0.1^2=0.01$). Isso causa essa diferença nos valores.

RMSE (Root Mean Squared Error)

- » **Definição:** É a raiz quadrada da média dos quadrados dos erros.
- » **Interpretação:** Quanto menor o RMSE, melhor o modelo. Tem a mesma unidade dos dados originais e penaliza mais os erros grandes.

Explicação Detalhada do RMSE

O Root Mean Squared Error (RMSE) é definido como:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

onde:

y_i são os valores reais,

$\hat{y}_i$ são os valores preditos pelo modelo,

n é o número de observações.

O RMSE penaliza mais severamente grandes erros devido ao uso do quadrado das diferenças, similar ao MSE, mas retorna os erros na mesma unidade dos valores observados.

Abaixo serão apresentados duas maneiras de cálculo do MSE, uma manualmente utilizando a formulação matemática acima e outra mais direta, utilizando o pacote do sklearn.

Como é de se esperar, em ambos casos os valores encontrados são os mesmos.

```
# Calcular o RMSE manualmente

# Calcular a soma dos quadrados dos resíduos (SSR)
ssr = np.sum((y - y_pred) ** 2)
# Médias dos valores
mse_manual = ssr/len(y)
# A raiz quadrada da média
rmse_manual = np.sqrt(mse_manual)

print(f'Root Mean Squared Error (RMSE): {rmse_manual:.2f}')

Root Mean Squared Error (RMSE): 0.90

#Cálculo utilizando o pacote do sklearn
# Passa-se os valores reais (y), os valores obtidos do modelo (y_pred)
# nesse caso o parametro `squared` é necessário para que seja calculado a raiz
quadrada
do erro
# o valor do erro é guardado em rmse
rmse = mean_squared_error(y, y_pred, squared=False)

# Impressão dos resultados obtidos
print(f'Root Mean Squared Error (RMSE): {rmse:.2f}')

Root Mean Squared Error (RMSE): 0.90
```

Visualização dos Quadrado dos Resíduos e do RMSE

Para ilustrar melhor, podemos plotar o quadrado dos resíduos e o RMSE obtido pelo modelo.

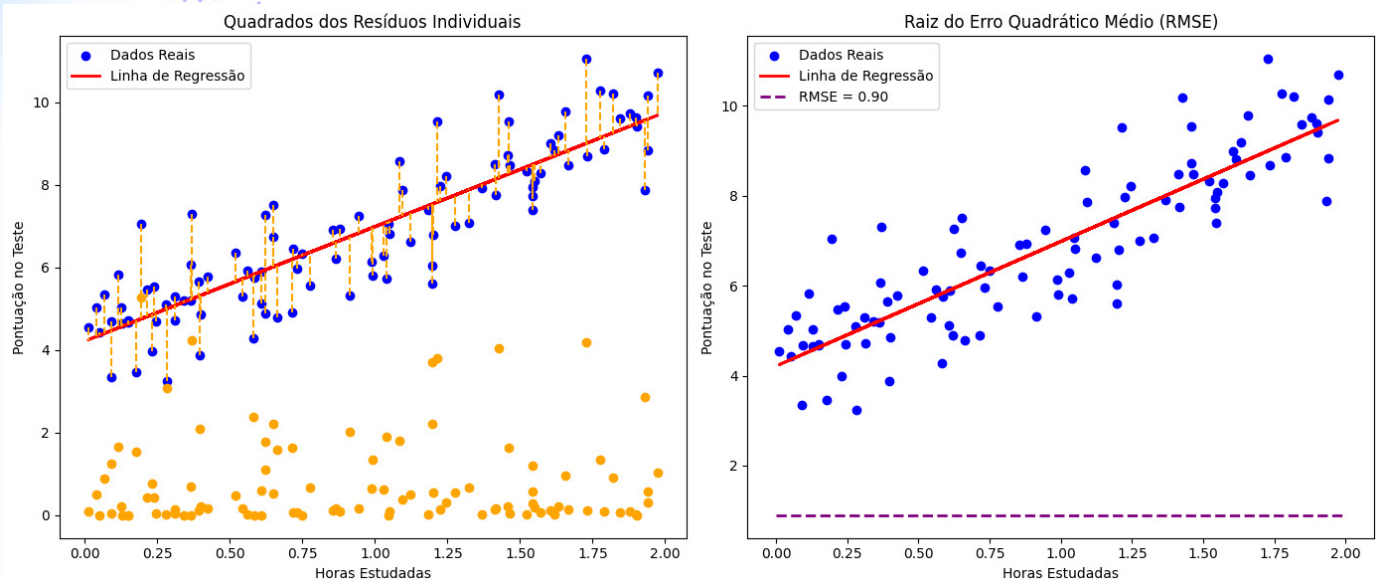
```
# Criar as subplots
fig, axs = plt.subplots(1, 2, figsize=(14, 6))

# Plotar os resíduos ao quadrado individuais
axs[0].scatter(X, y, color='blue', label='Dados Reais')
axs[0].plot(X, y_pred, color='red', linewidth=2, label='Linha de Regressão')
for i in range(len(X)):
    axs[0].plot([X[i], X[i]], [y[i], y_pred[i]], color='orange', linestyle='dashed')
    axs[0].scatter(X[i], (y[i] - y_pred[i])**2, color='orange')
axs[0].set_title('Quadrados dos Resíduos Individuais')
axs[0].set_xlabel('Horas Estudadas')
axs[0].set_ylabel('Pontuação no Teste')
axs[0].legend()

# Plotar o RMSE
axs[1].scatter(X, y, color='blue', label='Dados Reais')
axs[1].plot(X, y_pred, color='red', linewidth=2, label='Linha de Regressão')
axs[1].hlines(y=rmse, xmin=0, xmax=2, color='purple', linestyle='dashed', linewidth=2,
label=f'RMSE = {rmse:.2f}')
axs[1].set_title('Raiz do Erro Quadrático Médio (RMSE)')
axs[1].set_xlabel('Horas Estudadas')
axs[1].set_ylabel('Pontuação no Teste')
axs[1].legend()

# Mostrar os gráficos
plt.tight_layout()
plt.show()
```

Figura 63 - Gráfico do Quadrado dos Resíduos e do RMSE



Fonte: autoria própria.

Explicação dos Gráficos

1. Primeiro Gráfico (Quadrados dos Resíduos Individuais):

- » Os pontos azuis representam os dados reais.
- » A linha vermelha representa a linha de regressão ajustada pelo modelo.
- » As linhas tracejadas laranja e os pontos laranja representam os quadrados dos resíduos individuais, ou seja, as diferenças ao quadrado entre os valores observados e os valores preditos.

2. Segundo Gráfico (RMSE):

- » Os pontos azuis representam os dados reais.
- » A linha vermelha representa a linha de regressão ajustada pelo modelo.
- » A linha tracejada roxa horizontal representa o valor do RMSE, que é a raiz quadrada da média dos resíduos ao quadrado.

Para pensar:

Pergunta: Se o MSE calcula a média dos erros ao quadrado e o RMSE a raiz quadrada dessa média dos erros ao quadrado, por que o valor do RMSE foi superior ao do MSE?

Resposta: O valor de $\sqrt{0.81}$ (valor encontrado do MSE) é 0.90, a raiz quadrada de um número menor que 1 sempre vai ser um valor maior que o próprio valor.

Pergunta: O valor do RMSE sempre vai ser maior que o do MSE?

Resposta: Não, porque o valor do MSE não está limitado apenas a valores menores que 1. Se o MSE encontrado for de 4, seu RMSE será $\sqrt{4}=2$.

Para pensar:

Se possuo dois conjuntos de dados diferentes, sobre assuntos diferentes, por exemplo:

- » diferença de velocidade entre atletas de uma maratona
- » diferença de velocidade entre atletas de uma corrida de automóveis

Ao utilizar um modelo de regressão é possível estimar os valores preditos.

Pergunta: É possível comparar os erros e afirmar que o mesmo modelo é bom para predição em ambas atividades?

Resposta: Não, como os dados são de magnitudes diferentes, ou seja velocidade de atletas da ordem de dezenas de km/h e velocidade entre corridas de automóveis da ordem de centenas de km/h, os erros podem possuir escalas diferentes. Porém, se os erros estiverem na mesma escala, é possível fazer a comparação.

Pergunta: Uma diferença de 10% entre a velocidade real e predita em ambos datasets, resultariam sempre em erros equivalentes?

Resposta: Não necessariamente, o código abaixo ilustra melhor. Dependendo da métrica o fator escala pode fazer grande diferença.

```
# Conjunto de dados de uma maratona
y_foot = np.array([10.5, 12.1, 15.3, 17.2, 18.5, 23.2])
y_foot_pred = y_foot * 1.1

# Conjunto de dados de uma corrida de automóveis
y_car = np.array([150.4, 220.2, 157.8, 149.9, 153.0])
y_car_pred = y_car * 1.1

# Cálculo do R2 para ambos datasets
r2_foot = r2_score(y_foot, y_foot_pred)
r2_car = r2_score(y_car, y_car_pred)
print(f'R2 foot: {r2_foot:.2f}')
print(f'R2 car: {r2_car:.2f}')
print('*****')

# Cálculo do MAE para ambos datasets
mae_foot = mean_absolute_error(y_foot, y_foot_pred)
mae_car = mean_absolute_error(y_car, y_car_pred)
print(f'MAE foot: {mae_foot:.2f}')
print(f'MAE car: {mae_car:.2f}')
print('*****')
```

continua

```
# Cálculo do MSE para ambos datasets
mse_foot = mean_squared_error(y_foot, y_foot_pred)
mse_car = mean_squared_error(y_car, y_car_pred)
print(f'MSE foot: {mse_foot:.2f}')
print(f'MSE car: {mse_car:.2f}')
print('*****')

# Cálculo do RMSE para ambos datasets
rmse_foot = mean_squared_error(y_foot, y_foot_pred, squared=False)
rmse_car = mean_squared_error(y_car, y_car_pred, squared=False)
print(f'RMSE foot: {rmse_foot:.2f}')
print(f'RMSE car: {rmse_car:.2f}')
print('*****')
```

```
R2 foot: 0.84
R2 car: 0.61
*****
MAE foot: 1.61
MAE car: 16.63
*****
MSE foot: 2.78
MSE car: 283.78
*****
RMSE foot: 1.67
RMSE car: 16.85
*****
```

Exercícios

Com base nos dados gerados anteriormente:

1. Plote em um gráfico com os pontos de velocidade real e velocidade predita para cada dataset (maratona, corrida de veículos)
2. Plote em um gráfico de barras os valores comparativos das métricas entre os datasets

Exercício resolvido 1: Plote em um gráfico com os pontos de velocidade real e velocidade predita para cada dataset (maratona, corrida de veículos)

```
# Criação de um DataFrame com os valores das métricas para o caso da maratona
df_maratona = pd.DataFrame({'y': y_foot,
                             'y_pred': y_foot_pred
                             })
```

df_maratona

	y	y_pred
0	10.5	11.55
1	12.1	13.31
2	15.3	16.83
3	17.2	18.92
4	18.5	20.35
5	23.2	25.52

```
# Criação de um DataFrame com os valores das métricas para o caso da corrida de
automóveis
```

```
df_carros = pd.DataFrame({'y': y_car,
                          'y_pred': y_car_pred
                          })
```

```
df_carros
```

	y	y_pred
0	150.4	165.44
1	220.2	242.22
2	157.8	173.58
3	149.9	164.89
4	153.0	168.30

```
# Criar as subplots
```

```
fig, axs = plt.subplots(1, 2, figsize=(14, 6))
```

```
# Plotar os resíduos ao quadrado individuais
```

```
axs[0].scatter(df_maratona.index, df_maratona['y'], color='blue', label='Dados Reais')
```

```
axs[0].scatter(df_maratona.index, df_maratona['y_pred'], color='red', linewidth=2,
label='Dados Preditos')
```

```
axs[0].set_xlabel('Atleta')
```

```
axs[0].set_ylabel('Velocidade')
```

```
axs[0].set_title('Comparação Dados da Maratona')
```

```
axs[0].legend()
```

```
# Plotar os resíduos ao quadrado individuais
```

```
axs[1].scatter(df_carros.index, df_carros['y'], color='blue', label='Dados Reais')
```

```
axs[1].scatter(df_carros.index, df_carros['y_pred'], color='red', linewidth=2,
label='Dados Preditos')
```

```
axs[1].set_xlabel('Carro')
```

```
axs[1].set_ylabel('Velocidade')
```

```
axs[1].set_title('Comparação Dados da Corrida de Automóveis')
```

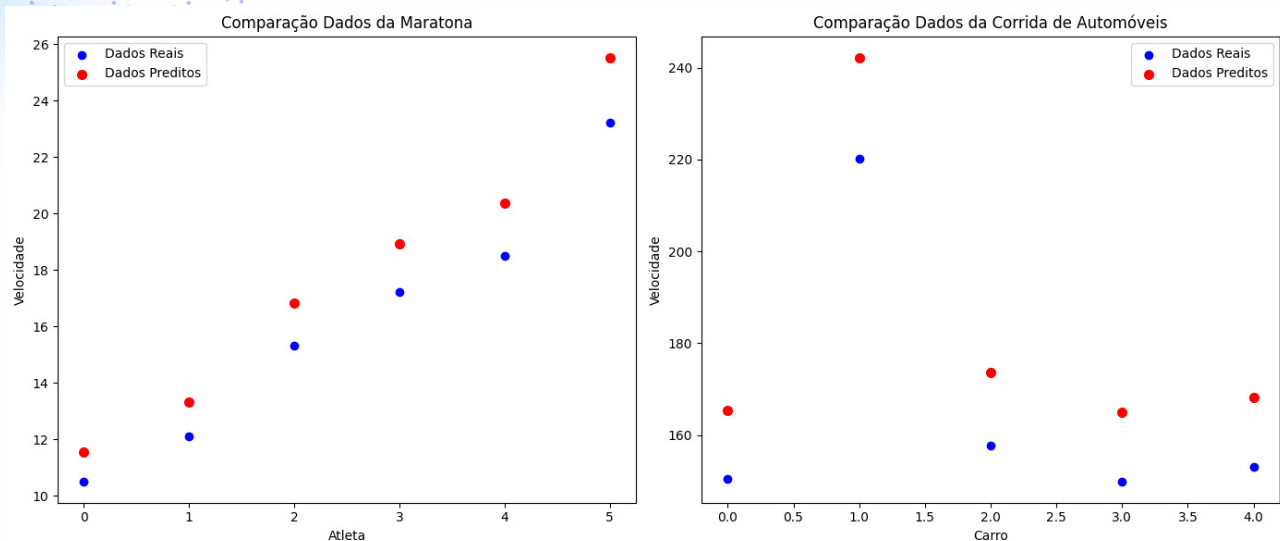
```
axs[1].legend()
```

```
# Mostrar os gráficos
```

```
plt.tight_layout()
```

```
plt.show()
```

Figura 64 - Comparação Dados da Corrida de Automóveis



Fonte: autoria própria.

Exercício resolvido 2: Plote em um gráfico de barras os valores comparativos das métricas entre os datasets

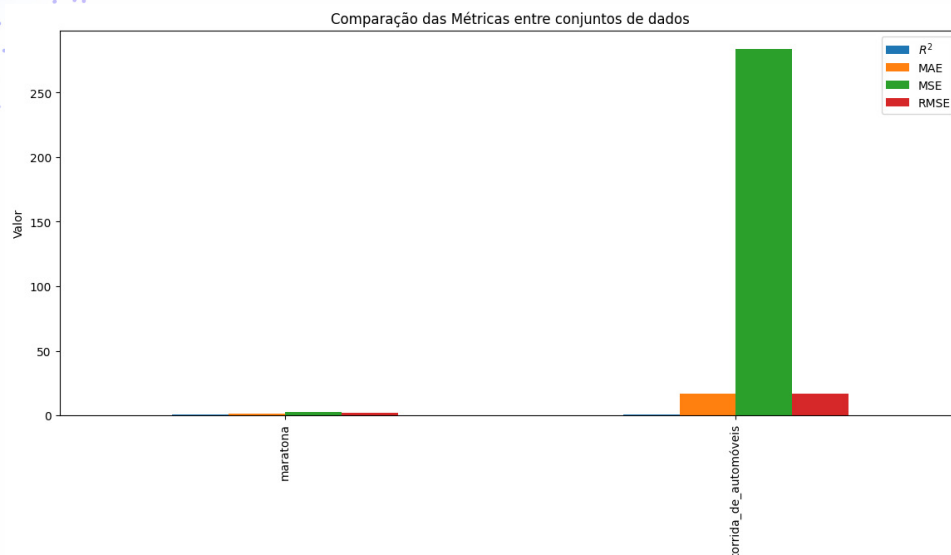
```
# Criação de um DataFrame com os valores das métricas
df_metrics = pd.DataFrame({'R^2': [r2_foot, r2_car],
                           'MAE': [mae_foot, mae_car],
                           'MSE': [mse_foot, mse_car],
                           'RMSE': [rmse_foot, rmse_car],
                           },
                           index=['maratona', 'corrida_de_automóveis'],
                           )
```

df_metrics

	R ²	MAE	MSE	RMSE
maratona	0.841793	1.613333	2.778467	1.666873
corrida_de_automóveis	0.614027	16.626000	283.776100	16.845655

```
df_metrics.plot(kind='bar', figsize=(14, 6))
plt.title('Comparação das Métricas entre conjuntos de dados')
plt.ylabel('Valor')
plt.show()
```

Figura 65 - Comparação das Métricas entre conjuntos de dados



Fonte: autoria própria.

Tipos de Regressão

Existem vários tipos de modelos de regressão, cada um com suas características e aplicações específicas:

- » **Regressão Linear:** Simples e múltipla, modela a relação linear entre variáveis. É o ponto de partida para muitos estudos de regressão.
- » **Regressão Ridge, Lasso e Elastic Net:** Incorporam penalizações para lidar com multicolinearidade e reduzir a complexidade do modelo, prevenindo sobreajuste.
- » **Árvores de Regressão:** As árvores de regressão são uma técnica de aprendizado de máquina que utiliza uma estrutura em forma de árvore para modelar a relação entre uma variável dependente contínua e uma ou mais variáveis independentes.
- » **RNN (Recurrent Neural Network) ou Rede Neural Recorrente:** é um tipo de arquitetura de rede neural projetada para processar sequências de dados. Suas principais características são: a existência de loops internos que permitem que informações persistam; sua capacidade de lidar com entradas e saídas de comprimento variável; o compartilhamento de parâmetros ao longo do tempo, reduzindo o número total de parâmetros a serem aprendidos. Especialmente útil para tarefas que envolvem dados sequenciais, como processamento de linguagem natural, reconhecimento de fala e previsão de séries temporais.

Objetivos

- » Compreender os fundamentos da regressão linear
- » Aprender a implementar e avaliar esses modelos utilizando Python.

- » Aplicar esses modelos em problemas de regressão com conjuntos de dados reais.

Introdução

A análise de regressão é uma técnica estatística fundamental utilizada para modelar a relação entre uma variável dependente e uma ou mais variáveis independentes. Este notebook abordará a Regressão Linear, porém cada método tem suas próprias características e aplicações específicas, que serão discutidas e implementadas em Python.

Regressão Linear

- » **Definição:** A regressão linear é um método estatístico para modelar a relação entre uma variável dependente e uma ou mais variáveis independentes assumindo uma relação linear.
- » **Fórmula:** $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$
- » **Interpretação:** Utilizada quando se espera que a variável dependente varie linearmente com as variáveis independentes.

Explicação Detalhada da Regressão Linear

A regressão linear simples assume a seguinte fórmula:

$$y = \beta_0 + \beta_1 X + \epsilon$$

onde:

y é a variável dependente.

X é a variável independente.

β_0 é o intercepto (valor de y quando $X = 0$).

β_1 é o coeficiente de inclinação (quanto y muda com uma unidade de mudança em X).

ϵ é o termo de erro (a diferença entre os valores observados e os valores preditos).

Passo 1: Carregando Biblioteca e dados

Vamos gerar alguns dados para exemplificar o processo de regressão linear.

Nesse exemplo é proposta a criação de um modelo linear para ser possível prever qual seria o valor esperado de produção de uma máquina conforme fosse o ajuste de sua velocidade de produção.

- » Em **X** temos a velocidade de produção de uma máquina
- » Em **y** temos a quantidade de produtos produzidos

A máquina real possui uma série de variáveis que a influenciam, por isso só observar só 2 pontos e concluir algo é muito perigoso.

Os dados gerados nesse exemplo são aleatórios, com o uso do `random.seed` e seguem a mesma técnica utilizada ao longo desse notebook.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Gerar dados aleatórios
np.random.seed(42)
X = 2 * np.random.rand(10, 1)
# Ordena os dados de X
X = sorted(X)
# Adequa o formato
X = np.array([arr.tolist() for arr in X])

# Cria os valores de y observados
y = 4 + 3 * X + np.random.randn(10, 1)
```

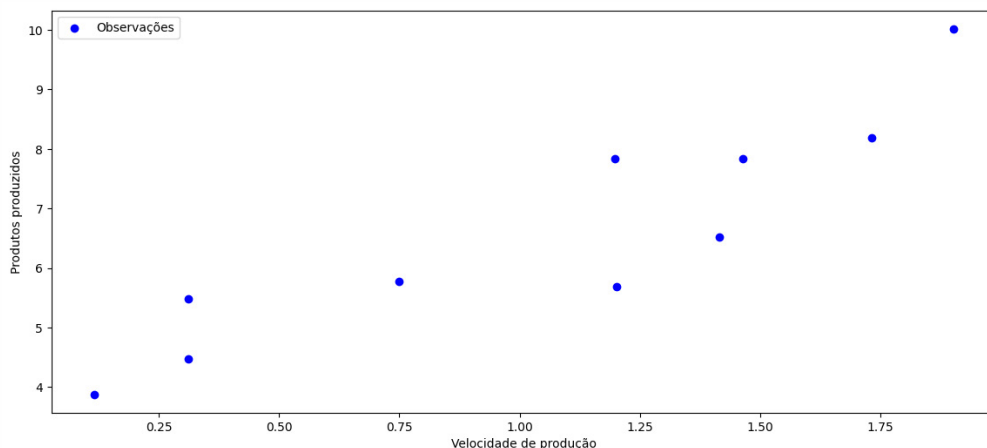
Passo 2: Plotagem dos dados

Plotar os dados em um gráfico ajuda a entender todo o processo. É possível ver no gráfico, que quanto maior a velocidade de produção, maior a produção. Mas, com base no que se tem até agora, só é possível saber a produção para o conjunto testado.

O modelo de regressão visa resolver a questão: Qual seria o valor esperado de produção para um tempo que ainda não testei?

```
fig, axs = plt.subplots(1, 1, figsize=(14, 6))
axs.scatter(X,y, color='blue', label='Observações')
# Adiciona a legenda
plt.legend()
# Adiciona rótulos dos eixos
plt.xlabel("Velocidade de produção")
plt.ylabel("Produtos produzidos")
plt.show()
```

Figura 66 - Gráfico para exemplificar o processo de Regressão Linear



Fonte: autoria própria.

Passo 3: Testa-se um modelo

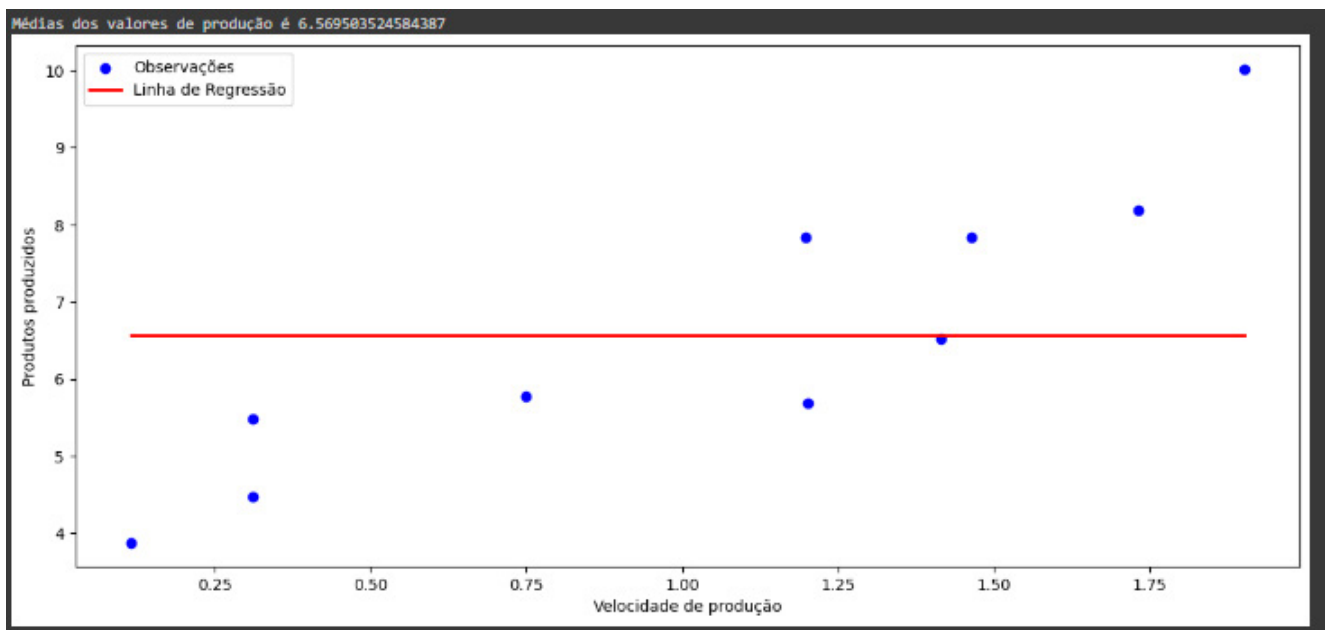
Um primeiro modelo mais intuitivo é encontrar a média dos valores e adotar esse como se fosse o modelo.

```
# Primeiro modelo: calcula-se a média dos valores de y
y_mean = np.mean(y)
print(f"Médias dos valores de produção é {y_mean}")

# Repetindo esse valor para cada valor de X, temos um vetor com a média
# y_pred é o valor predito para os valores X, utilizando um modelo que retornam a média dos valores
y_pred = np.array([y_mean for _ in range(len(X))])

# Plotando no gráfico
fig, axs = plt.subplots(1, 1, figsize=(14, 6))
# Plota os pontos
axs.scatter(X,y, color='blue', label='Observações')
# Plota o modelo
axs.plot(X, y_pred, color='red',linewidth=2, label='Linha de Regressão')
# Adiciona a legenda
plt.legend()
# Adiciona rótulos dos eixos
plt.xlabel("Velocidade de produção")
plt.ylabel("Produtos produzidos")
plt.show()
```

Figura 67 - Gráfico de Plotagem dos Dados



Fonte: autoria própria.

Passo 4: Encontrando a soma do quadrado dos resíduos

1. Para cada ponto (azul) é verificado então sua distância ao modelo (vermelho).
2. Eleva-se ao quadrado cada uma dessas distâncias (diferenças)
3. Soma-se esses quadrado das diferenças

```
# Distância ponto ao modelo
residuais = y_pred - y

# Residuais ao quadrado
residuais_sq = np.power(residuais, 2)

# Soma do quadrado dos resíduos
soma_residuos_sq = np.sum(residuais_sq)

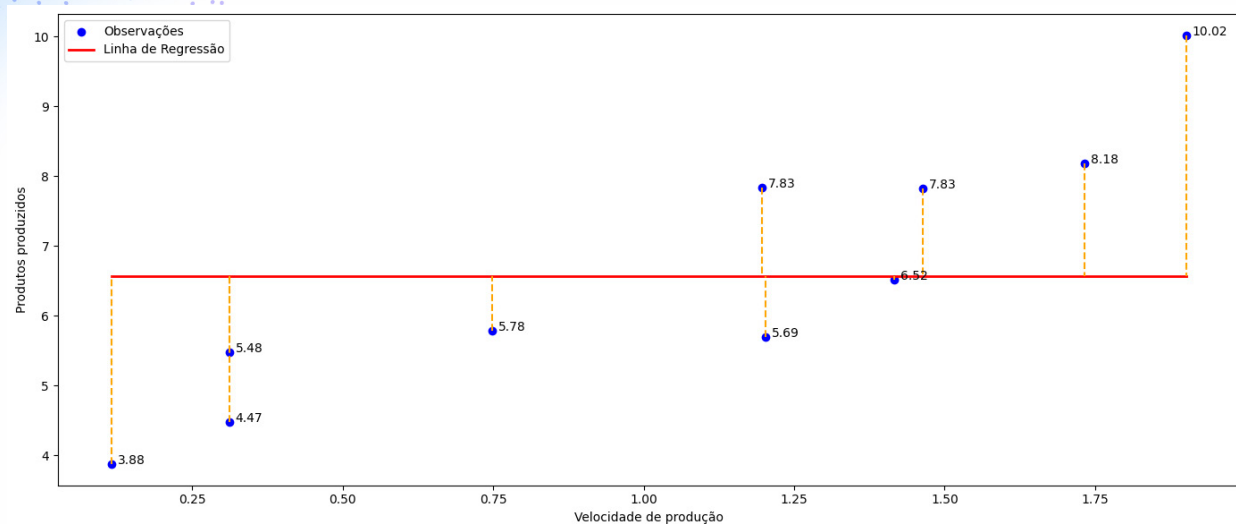
print(f"Soma do quadrado dos resíduos (SSR): {soma_residuos_sq.item():.2f}")
Soma do quadrado dos resíduos (SSR): 319.06
```

Para ilustrar o andamento, plotam-se as linhas de diferença (laranja) entre o valor real (pontos em azul) e o valor que seria no modelo (linha em vermelho).

```
# Plotando no gráfico
fig, axs = plt.subplots(1, 1, figsize=(14, 6))
# Plota os pontos
axs.scatter(X,y, color='blue', label='Observações')
# Plota o modelo
axs.plot(X, y_pred, color='red',linewidth=2, label='Linha de Regressão')

# Plota as distâncias entre pontos e modelo
for i in range(len(X)):
    # Plota as linhas das distâncias em laranja
    axs.plot([X[i], X[i]], [y[i], np.array([y_pred[i]])], color='orange',
linestyle='dashed')
    # Adiciona o rótulo do valor observado
    axs.text(x= X[i]+.01, y= y[i], s=str(np.round([*y[i]][0],2)))
# Adiciona a legenda
plt.legend()
# Adiciona rótulos dos eixos
plt.xlabel("Velocidade de produção")
plt.ylabel("Produtos produzidos")
plt.tight_layout()
plt.show()
```

Figura 68 - Gráfico da Soma do Quadrado dos Resíduos



Fonte: autoria própria.

Passo 5: Testa-se um novo modelo (gira a reta vermelha) e verifica-se o SSR (soma do quadrado dos resíduos).

Um novo modelo poderia ser:

$$y=4+1.5*X$$

Então aplica-se o Passo 4 novamente.

```
# Novo modelo
y_pred = 4 + 1.5 * X
```

Repetindo o passo 4

```
# Distância ponto ao modelo
residuais = y_pred - y

# Residuais ao quadrado
residuais_sq = np.power(residuais, 2)

# Soma do quadrado dos resíduos
soma_residuos_sq = sum(residuais_sq)

print(f"Soma do quadrado dos resíduos (SSR): {soma_residuos_sq.item():.2f}")
```

```
Soma do quadrado dos resíduos (SSR): 21.08
```

```
# Plotando no gráfico
fig, axs = plt.subplots(1, 1, figsize=(14, 6))
# Plota os pontos
axs.scatter(X,y, color='blue', label='Observações')
# Plota o modelo
axs.plot(X, y_pred, color='red',linewidth=2, label='Linha de Regressão')

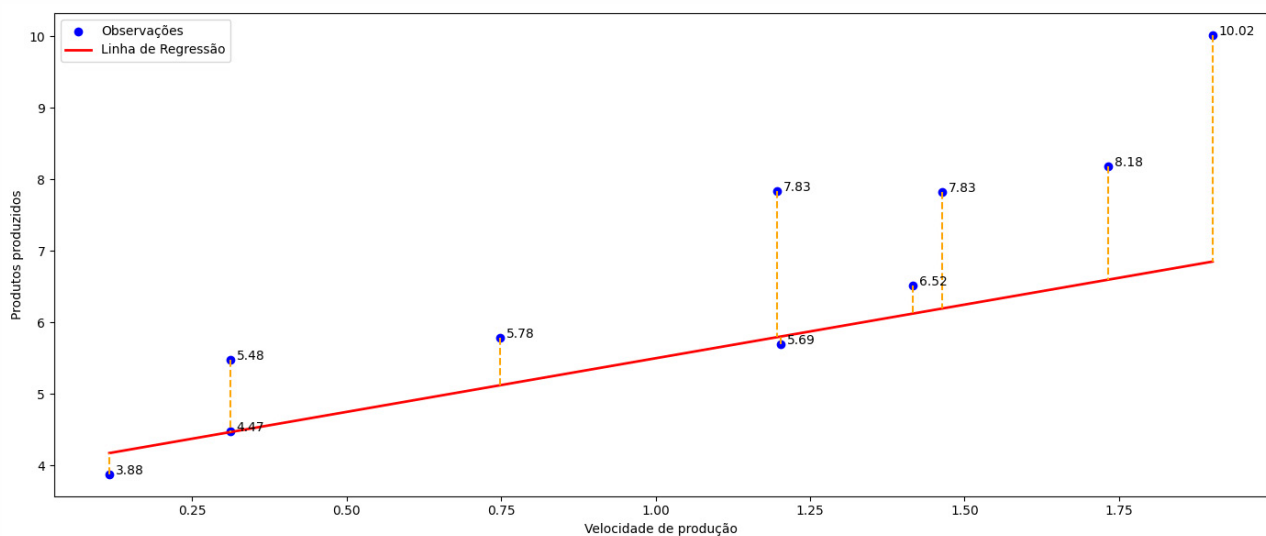
# Plota as distâncias entre pontos e modelo
for i in range(len(X)):
```

continua

```
# Plota as linhas das distâncias
axs.plot([X[i], X[i]], [y[i], y_pred[i]], color='orange', linestyle='dashed')
# Adiciona o rótulo do valor observado
axs.text(x= X[i]+.01, y= y[i], s=str(np.round([*y[i]][0],2)))

# Adiciona a legenda
plt.legend()
# Adiciona rótulos dos eixos
plt.xlabel("Velocidade de produção")
plt.ylabel("Produtos produzidos")
plt.tight_layout()
plt.show()
```

Figura 69 - Gráfico do SSR do Novo Modelo



Fonte: autoria própria.

Na primeira iteração foi obtido SSR de 31.91.

Na segunda iteração foi obtido valor de SSR de 21.08, portanto o segundo modelo é melhor que o primeiro.

Esse processo de teste de modelos se repete até que se encontre o valor mínimo do SSR. E esse processo de otimização que identifica o menor SSR chama-se **Método dos mínimos quadrados**.

Regressão com uso do sklearn

Aqui será abordado o emprego direto de modelos de regressão utilizando a biblioteca sklearn.

Esses modelos são de simples implementação, porém é necessário o entendimento de seu funcionamento, o ajuste de parâmetros, portanto a documentação deve ser sempre consultada.

Serão abordados os algoritmos: regressão linear, regressão ridge, árvores de decisão e redes neurais recorrentes.

Criação do dataset para treino dos modelos

O conjunto de dados Diabetes disponível através do `load_diabetes()` do Scikit-learn é um conjunto de dados padrão, frequentemente utilizado para fins de aprendizado e demonstração em tarefas de machine learning. Ele contém informações sobre pacientes com diabetes, incluindo diversas características médicas como idade, sexo, índice de massa corporal (IMC), pressão arterial, níveis de glicose, etc.

Suas principais características são:

- » **Dados reais:** Os dados são provenientes de um estudo real sobre diabetes, o que o torna um conjunto de dados realista para testar algoritmos de aprendizado de máquina.
- » **Variedade de features:** O dataset possui um número considerável de features, o que permite explorar diferentes técnicas de seleção e engenharia de features.
- » **Target:** O objetivo (target) é uma medida quantitativa da progressão da doença, o que o torna um problema de regressão.

```
# Importação da biblioteca
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

# Carregar um conjunto de dados de exemplo
from sklearn.datasets import load_diabetes
diabetes = load_diabetes()
# Em X ficam os dados de características que serão os preditores do progresso da doença
X = pd.DataFrame(diabetes.data, columns=diabetes.feature_names)
# Em y, ficam os valores de progresso da doença
y = diabetes.target

# Explicação do dataset
print(diabetes.DESCR)
```

```
.. _diabetes_dataset:
```

```
Diabetes dataset
-----
```

```
Ten baseline variables, age, sex, body mass index, average blood pressure, and six blood serum measurements were obtained for each of n = 442 diabetes patients, as well as the response of interest, a
```

continua

quantitative measure of disease progression one year after baseline.

****Data Set Characteristics:****

:Number of Instances: 442

:Number of Attributes: First 10 columns are numeric predictive values

:Target: Column 11 is a quantitative measure of disease progression one year after baseline

:Attribute Information:

- age age in years
- sex
- bmi body mass index
- bp average blood pressure
- s1 tc, total serum cholesterol
- s2 ldl, low-density lipoproteins
- s3 hdl, high-density lipoproteins
- s4 tch, total cholesterol / HDL
- s5 ltg, possibly log of serum triglycerides level
- s6 glu, blood sugar level

Note: Each of these 10 feature variables have been mean centered and scaled by the standard deviation times the square root of `n\_samples` (i.e. the sum of squares of each column totals 1).

Source URL:

<https://www4.stat.ncsu.edu/~boos/var.select/diabetes.html>

For more information see:

Bradley Efron, Trevor Hastie, Iain Johnstone and Robert Tibshirani (2004)
"Least Angle Regression," Annals of Statistics (with discussion), 407-499.
(https://web.stanford.edu/~hastie/Papers/LARS/LeastAngle_2002.pdf)

```
# Para ilustrar melhor cada modelo, será utilizado apenas a característica de IMC (bmi)
# Nesse caso, estará sendo utilizado os dados de IMC para treinar um modelo que,
# com base apenas no valor do IMC consiga prever o valor da progressão da doença
col = 'bmi'

# Dividir os dados em treino (80%) e teste (20%)
# utilização do random_state para manter a reprodutibilidade
X_train, X_test, y_train, y_test = train_test_split(X[col].to_numpy(), y, test_size=0.2, random_state=42)

# Ordena os dados para facilitar a plotagem
# Coloca X_test e y_test em uma tupla e ordena pela X_test
z = sorted(zip(X_test, y_test))
# "Descompacta" a lista ordenada em X_test e y_test
X_test, y_test = zip(*z)
# Transforma a tupla de valores X-test e y-test em um array.
X_test = np.array(X_test)
y_test = np.array(y_test)
```

Função para relatório

Para evitar repetição foi definida uma função para imprimir um relatório de performance do modelo.

O relatório consta de 2 partes: Gráfico e Métricas.

Gráfico:

- » No eixo x tem-se os valores da característica, no caso proposto bmi ou IMC.
- » No eixo y, tem-se os valores da progressão da doença.
- » Os pontos pretos correspondem os dados de Teste
- » A linha colorida representa o modelo ajustado aos dados de Teste

Métricas: Faz a comparação das métricas R^2 , MSE e MAE de performance do modelo entre os conjuntos de Treino e Teste.

```
def relatorio(X_test, y_test, y_train, y_pred_model, y_train_model, modelo, col,
color_model):
    # Plot outputs
    plt.scatter(X_test, y_test, color="black")
    plt.plot(X_test, y_pred_model, color=color_model, linewidth=3)
    plt.xticks(())
    plt.yticks(())
    plt.title(f"{modelo} treinado com dados apenas de {col}")
    plt.xlabel(col)
    plt.ylabel("Progressão da doença")
    plt.show()

    # Avaliação das Métricas
    print(f"{modelo} -  $R^2$  - Treino {r2_score(y_train, y_train_model):.4f} - Teste {r2_score(y_test, y_pred_model):.4f}")
    print(f"{modelo} - MSE - Treino {mean_squared_error(y_train, y_train_model):.4f} - Teste {mean_squared_error(y_test, y_pred_model):.4f}")
    print(f"{modelo} - MAE - Treino {mean_absolute_error(y_train, y_train_model):.4f} - Teste {mean_absolute_error(y_test, y_pred_model):.4f}")
```

Regressão Linear

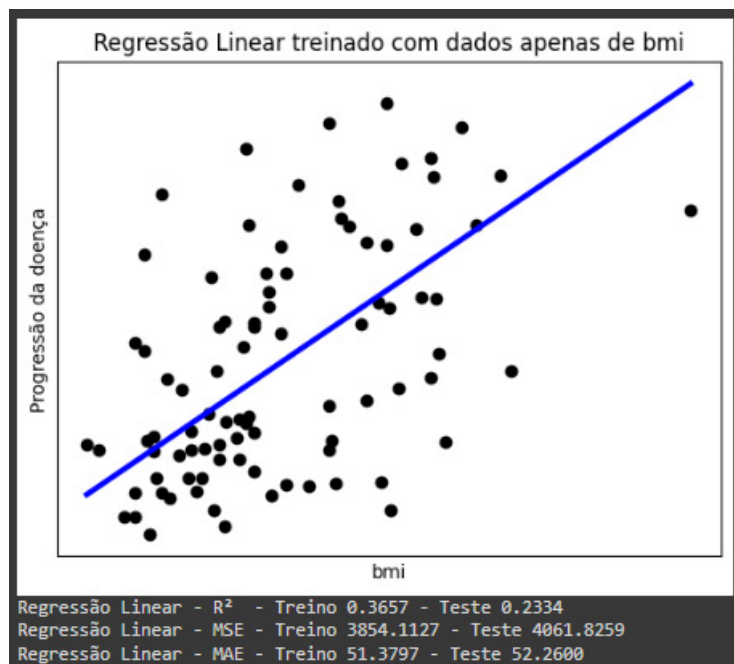
Utilizando a biblioteca do sklearn, abaixo é treinado um modelo de regressão linear com base no valores de treino do IMC **X_train** e de progressão da doença **y_train**.

- » São obtidos os valores preditos para o **X_test** e guardado em **y_pred_lin**.
- » São obtidos os valores preditos para o **X_train** e guardado em **y_train_lin**.
- » São verificadas as métricas de R^2 , MSE e MAE para os conjuntos de Treino e Teste.

```
# Modelo de Regressão Linear
lin_reg = LinearRegression()
# Treino do modelo
lin_reg.fit(X_train.reshape(-1, 1), y_train)
# Valores da predição com base nos dados de Teste
y_pred_lin = lin_reg.predict(X_test.reshape(-1, 1))
# Valores da predição com base nos dados de Treino
y_train_lin = lin_reg.predict(X_train.reshape(-1, 1))

# Imprime o relatório do modelo
modelo = 'Regressão Linear'
color_model = 'blue'
relatorio(X_test, y_test, y_train, y_pred_lin, y_train_lin, modelo, col, color_model)
```

Figura 70 - Gráfico de Regressão Linear treinado com dados apenas de bmi



Fonte: autoria própria.

Regressão Ridge

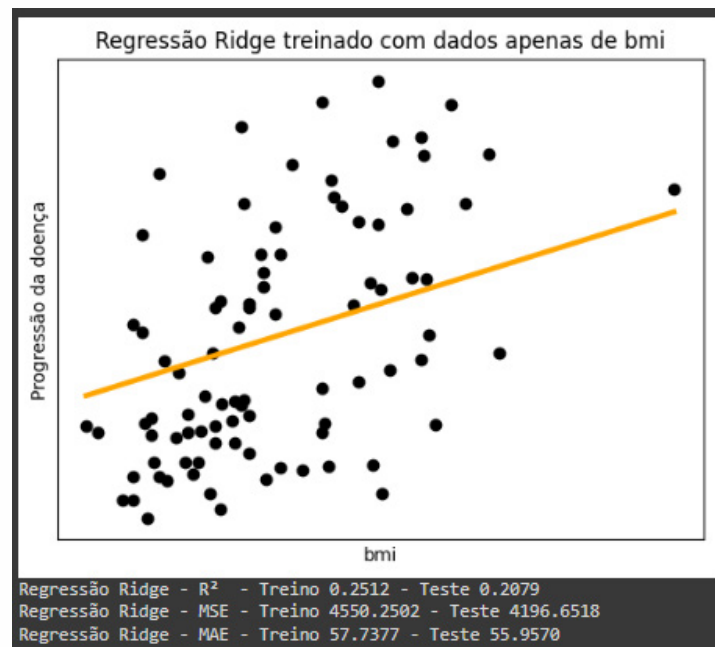
Utilizando a biblioteca do sklearn, abaixo é treinado um modelo de regressão ridge com base no valores de treino do IMC **X_train** e de progressão da doença **y_train**.

- » São obtidos os valores preditos para o **X_test** e guardado em **y_pred_ridge**.
- » São obtidos os valores preditos para o **X_train** e guardado em **y_train_ridge**.
- » São verificadas as métricas de R^2 , MSE e MAE para os conjuntos de Treino e Teste.

```
# Modelo de Regressão Ridge
ridge_reg = Ridge(alpha=1.0)
# Treino do modelo
ridge_reg.fit(X_train.reshape(-1, 1), y_train)
# Valores da predição com base nos dados de Teste
y_pred_ridge = ridge_reg.predict(X_test.reshape(-1, 1))
# Valores da predição com base nos dados de Treino
y_train_ridge = ridge_reg.predict(X_train.reshape(-1, 1))
```

```
# Imprime o relatório do modelo
modelo = 'Regressão Ridge'
color_model = 'orange'
relatório(X_test, y_test, y_train, y_pred_ridge, y_train_ridge, modelo, col, color_
modelo)
```

Figura 71 - Gráfico de Regressão Ridge treinado com dados apenas de bmi



Fonte: autoria própria.

Árvore de Regressão

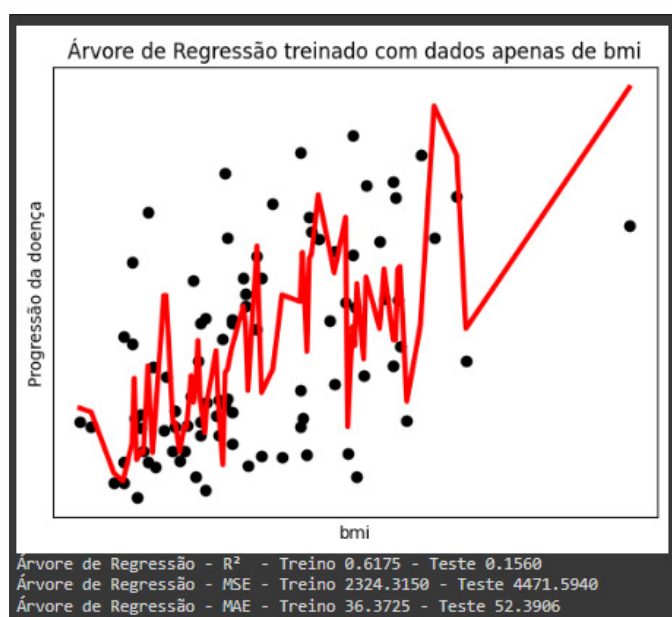
Utilizando a biblioteca do sklearn, abaixo é treinado um modelo de regressão ridge com base no valores de treino do IMC **X_train** e de progressão da doença **y_train**.

- » São obtidos os valores preditos para o **X_test** e guardado em **y_pred_tree**.
- » São obtidos os valores preditos para o **X_train** e guardado em **y_train_tree**.
- » São verificadas as métricas de R^2 , MSE e MAE para os conjuntos de Treino e Teste.

```
# Modelo de Árvore de Regressão
tree_reg = DecisionTreeRegressor(random_state=42)
# Treino do modelo
tree_reg.fit(X_train.reshape(-1, 1), y_train)
# Valores da predição com base nos dados de Teste
y_pred_tree = tree_reg.predict(X_test.reshape(-1, 1))
# Valores da predição com base nos dados de Treino
y_train_tree = tree_reg.predict(X_train.reshape(-1, 1))

# Imprime o relatório do modelo
modelo = 'Árvore de Regressão'
color_model = 'red'
relatorio(X_test, y_test, y_train, y_pred_tree, y_train_tree, modelo, col, color_model)
```

Figura 72 - Gráfico da Árvore de Regressão treinado com dados apenas de bmi



Fonte: autoria própria.

Redes Neurais Recorrentes (RNN)

Utilizando a biblioteca do sklearn, abaixo é treinado um modelo de regressão ridge com base no valores de treino do IMC **X_train** e de progressão da doença **y_train**.

A RNN necessita de uma formatação adicional nos vetores, portanto é realizado essa adaptação transformando **X_train** de dimensão (353,) em **X_train_rnn** de dimensão (353,1,1). De forma análoga é transformado **X_test** em **X_test_rnn**.

- » São obtidos os valores preditos para o **X_test_rnn** e guardado em **y_pred_lin**.
- » São obtidos os valores preditos para o **X_train_rnn** e guardado em **y_train_lin**.

```

# Preparação dos dados para RNN
X_train_rnn = X_train.reshape((X_train.shape[0], 1, 1))
X_test_rnn = X_test.reshape((X_test.shape[0], 1, 1))

# Cria o Modelo de RNN empilhando diversas camadas
# Esta arquitetura cria uma rede neural recorrente com duas camadas RNN,
# seguida por duas camadas densas para processamento não linear
# e uma camada de saída para a predição.
model = Sequential([
    SimpleRNN(64, activation='tanh', return_sequences=True, input_shape=(X_train_rnn.
shape[1], X_train_rnn.shape[2])),
    SimpleRNN(64, activation='tanh'),
    Dense(32, activation='relu'),
    Dense(16, activation='relu'),
    Dense(1)
])
model.compile(optimizer='adam', loss='mse')

# Treinamento
model.fit(X_train_rnn, y_train, epochs=500, verbose=0)

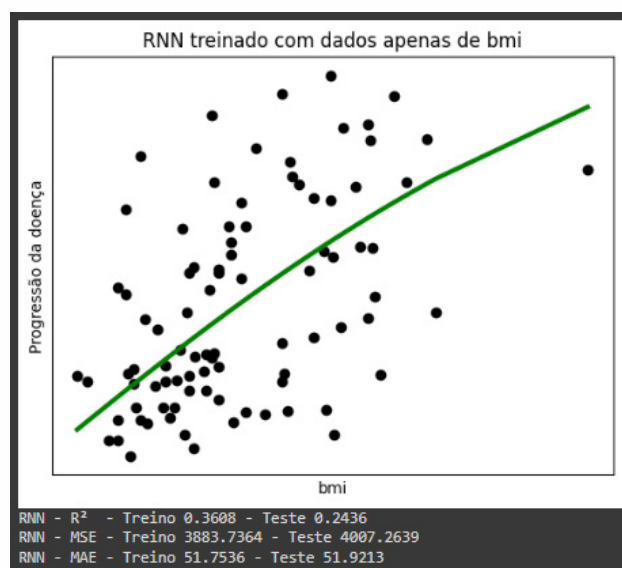
# Previsões
y_pred_rnn = model.predict(X_test_rnn)
y_train_rnn = model.predict(X_train_rnn)

/usr/local/lib/python3.10/dist-packages/keras/src/layers/rnn/rnn.py:204:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a
layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(**kwargs)
3/3 _____ 1s 141ms/step
12/12 _____ 0s 3ms/step

# Imprime o relatório do modelo
modelo = 'RNN'
color_model = 'green'
relatorio(X_test, y_test, y_train, y_pred_rnn, y_train_rnn, modelo, col, color_model)

```

Figura 73 - Gráfico RNN treinado com dados apenas de bmi



Fonte: autoria própria.

Conclusões

Foram testados 4 modelos distintos, sendo: Regressão Linear, Regressão Ridge, Árvore de Regressão e uma RNN.

Para cada modelo foram verificados as métricas para os conjuntos de Teste e Treino, bem como foi apresentado um gráfico do modelo ajustado ao conjunto de Teste.

Análise Crítica dos Resultados de Desempenho

A análise dos resultados de desempenho dos modelos de regressão linear, regressão Ridge, árvore de regressão e redes neurais recorrentes (RNN) revela importantes insights sobre o comportamento de cada modelo e sua adequação ao conjunto de dados. Abaixo, são discutidos aspectos relacionados ao desempenho, possíveis problemas de overfitting e a escolha do melhor modelo a ser aplicado.

- » A **regressão linear** apresenta um desempenho modesto, com uma redução significativa do R^2 no conjunto de teste, indicando que o modelo tem uma capacidade limitada de generalização. A proximidade entre os erros absolutos médios (MAE) de treino e teste sugere que o modelo não está superajustado (overfitted), mas também não é suficientemente robusto para capturar toda a variabilidade dos dados.
- » A **regressão Ridge**, uma variação regularizada da regressão linear, tem um desempenho ainda mais modesto, com valores de R^2 mais baixos tanto para treino quanto para teste. O maior valor de MSE no treino, comparado ao MSE do teste, sugere que a regularização pode estar penalizando excessivamente os coeficientes, levando a um underfitting.
- » A **árvore de regressão** apresenta um alto R^2 no conjunto de treino, mas um desempenho significativamente pior no teste, com uma queda abrupta no R^2 e um aumento no MSE. Isso é um forte indicador de overfitting, onde o modelo se ajusta muito bem aos dados de treino, mas não generaliza bem para novos dados.
- » A **RNN** apresenta valores de R^2 e MSE próximos tanto no treino quanto no teste. Isso é um indicador de que não há overfitting. Comparando graficamente a árvore de decisão e RNN, fica claro a questão do overfitting na árvore de decisão. A RNN mostra um desempenho muito semelhante ao da regressão linear no treino porém com um custo computacional muito mais elevado. Isso decorre principalmente porque no caso apresentado, o problema com apenas uma característica (bmi), não apresenta grandes dificuldades para a Regressão Linear, a RNN provavelmente obteria melhores resultados se todas as características do dataset fossem utilizadas.

Referências Bibliográficas

Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems. O'Reilly Media.

McKinney, W. (2017). Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython. O'Reilly Media.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning: Data Mining, Inference, and Prediction. Springer Series in Statistics.



SAIBA MAIS...

- » Regressão Linear - <https://youtu.be/7ArmBVF2dCs?si=XV-Ax7DZz0sN2Wx1>
- » Regressão Linear - <https://www.youtube.com/watch?v=PaFPbb66DxQ>
- » Regressão Ridge - <https://youtu.be/Q81RR3yKn30?si=K-TkT-W69SAM60uK>
- » Regressão Lasso - <https://youtu.be/NGf0voTmlcs?si=T8z8AlNTh8zl6iZF>
- » Comparação entre Ridge e Lasso
 - » https://youtu.be/Xm2C_gTAI8c?si=Ti2D0PTfnJAl1e4B
- » Árvores de regressão
 - » <https://youtu.be/g9c66TUylZ4?si=-fP85nMbFRgT2nw7>
- » R_2 - <https://youtu.be/bMccdk8EdGo?si=zRO2xrswQPq6xSZS>
- » R_2 - <https://youtu.be/aP8YP1aX2zc?si=WDFJh6kkSNPjdtgO>
- » Regressão Linear - <https://youtu.be/bMccdk8EdGo?si=zRO2xrswQPq6xSZS>
- » Regressão Linear - https://youtu.be/Wo9Vt7Sc_E0?si=hd7X7u5QX-2m57AI

PARA RELEMBRAR...

Regressão: Técnica estatística para modelar relações entre variáveis dependentes contínuas e variáveis independentes.

Importância: Fundamental para previsões quantitativas em diversos campos como economia, finanças e indústria.

Regressão vs. Classificação:

- » **Regressão:** Predição de variáveis contínuas (ex.: preços, temperaturas)
- » **Classificação:** Atribuição de instâncias a categorias discretas (ex.: spam ou não spam)

Métricas de Avaliação:

- » **R^2 :** Coeficiente de Determinação
- » **MSE:** Erro Quadrático Médio
- » **MAE:** Erro Absoluto Médio
- » **RMSE:** Raiz do Erro Quadrático Médio

Tipos de Regressão:

- » **Linear:** Simples e múltipla
- » *Ridge, Lasso e Elastic Net:* Incluem penalizações
- » **Árvores de Regressão:** Estrutura em forma de árvore
- » **RNN:** Para processar sequências de dados

Unidade IX

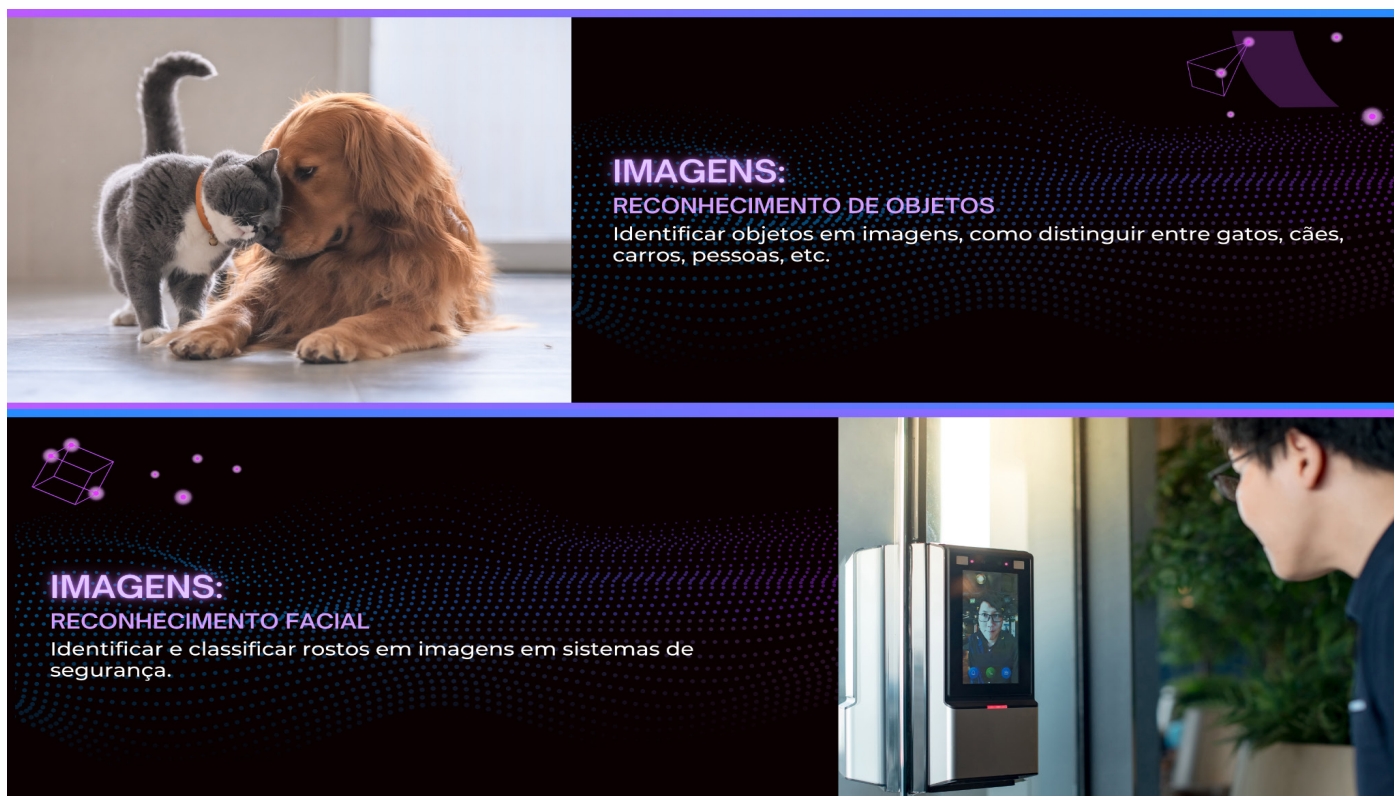
Classificação

Unidade IX - Classificação

Como já foi comentado nas unidades anteriores deste eBook, um dos problemas comuns no Aprendizado de Máquina é a tarefa de identificar as classes associadas a um determinado objeto. A partir de suas características, também chamadas de variáveis, utilizamos métodos computacionais inteligentes para tentar determinar a qual classe esse objeto pertence. Podemos usar como exemplo a simples tarefa de identificar se um determinado e-mail é spam ou não. Um e-mail pode conter texto, imagens e arquivos diversos, que são suas variáveis, e pode ser interpretado como spam ou não spam. O que determina se um e-mail é spam ou não é uma interpretação dessas variáveis de acordo com o interesse do dono da caixa de e-mail. Para uma pessoa, um e-mail de propaganda de um pet shop, por exemplo, pode ser irrelevante, logo, seria classificado como spam; mas para outra pessoa, pode ser de interesse, logo, não seria considerado spam. Dessa forma, classificar diferentes mensagens de e-mail entre as duas classes (spam e não spam) é uma tarefa que exige a interpretação das variáveis a cada nova mensagem, compondo assim uma tarefa de classificação de mensagens.

As tarefas de classificação são comuns por serem amplamente utilizadas na automação de processos que envolvem a interpretação de dados de diversos tipos. Se dividirmos por tipo de dado de entrada, podemos listar alguns exemplos de aplicações.

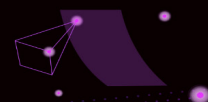
Figura 74 - Exemplos de aplicações de classificação





IMAGENS: DETECÇÃO DE DOENÇAS

Classificar imagens médicas (como radiografias, tomografias ou ressonâncias magnéticas) para identificar a presença ou ausência de doenças, como câncer, pneumonia ou outras condições.



LINGUAGEM NATURAL:

ANÁLISE DE SENTIMENTO

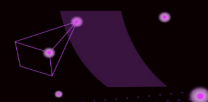
Classificar textos (como comentários em redes sociais ou avaliações de produtos) como positivos, negativos ou neutros.



LINGUAGEM NATURAL:

CLASSIFICAÇÃO DE TÓPICOS

Classificar textos em categorias específicas, como política, esportes, tecnologia, etc.



ÁUDIO:

RECONHECIMENTO DE COMANDOS

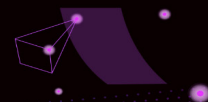
Identificar comandos verbalizados. Ok, Google!

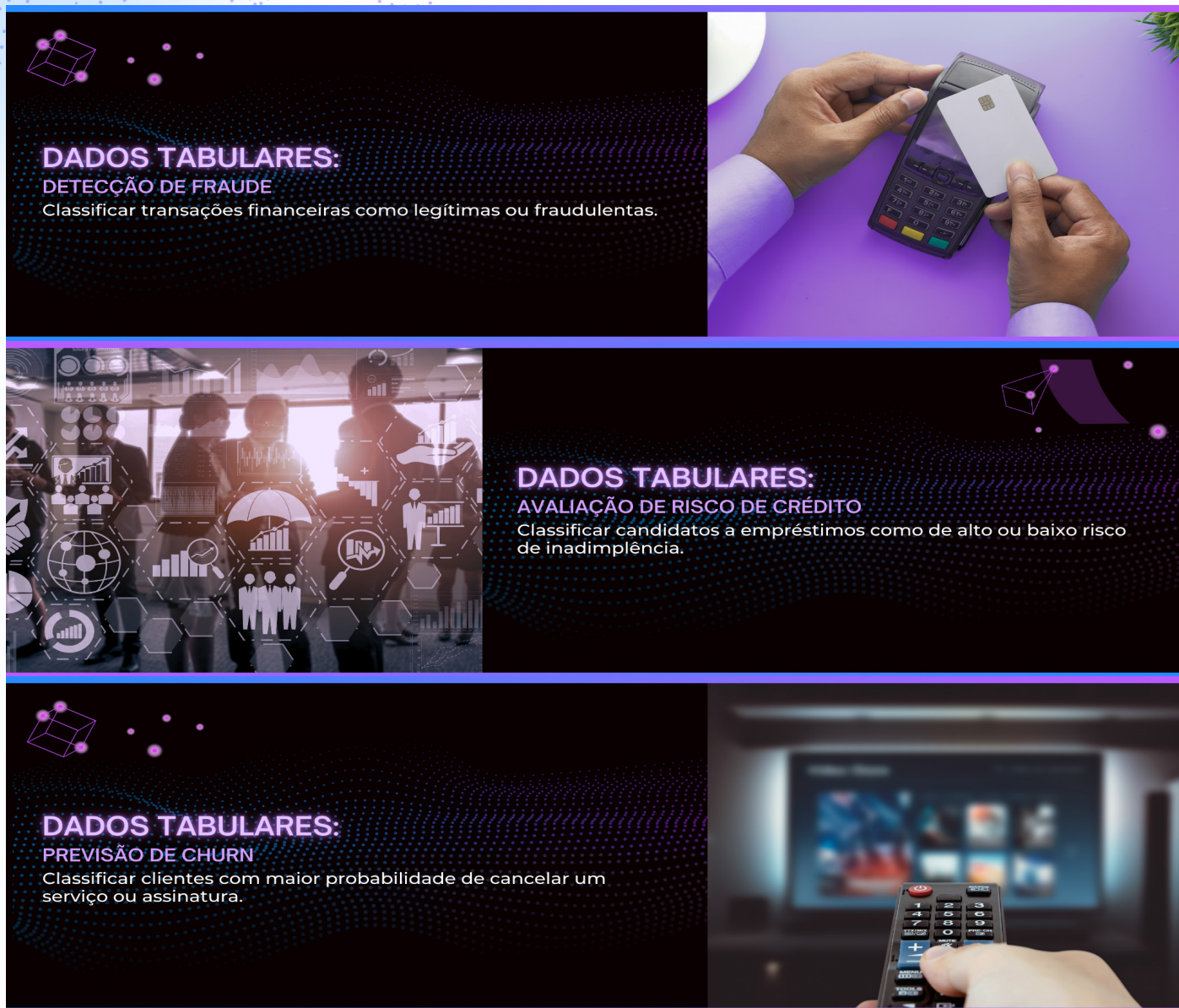


ÁUDIO:

DETECÇÃO DE ATIVIDADE

Classificar trechos de áudio entre voz humana e outros sons.





DADOS TABULARES:

DETECÇÃO DE FRAUDE

Classificar transações financeiras como legítimas ou fraudulentas.

DADOS TABULARES:

AValiação de Risco de Crédito

Classificar candidatos a empréstimos como de alto ou baixo risco de inadimplência.

DADOS TABULARES:

PREVISÃO DE CHURN

Classificar clientes com maior probabilidade de cancelar um serviço ou assinatura.

Fonte: autoria própria.



9.1 Classificadores

O elemento principal ao tratar um problema de classificação é o classificador. Classificadores são métodos de Aprendizado de Máquina que, a partir das variáveis extraídas de um objeto, tentam determinar a qual classe ele pertence. Esses métodos são desenvolvidos com o objetivo de compreender as possíveis variações dentro de uma classe. Por exemplo, em uma tarefa de reconhecimento de gatos a partir de uma imagem, podemos ter diversas variações, como posição, raça, cor, tamanho e formato, conforme ilustrado na Figura 45. Todas essas variações geram variáveis com algumas diferenças, mas que ainda assim representam exemplos da mesma classe. O objetivo dos classificadores é identificar os padrões comuns a cada classe (dois olhos, bigodes, orelhas, quatro patas, etc.) enquanto tentam generalizar esses padrões, considerando as possíveis variações. Assim, um bom classificador de gatos deve ser capaz de generalizar a definição de gato e reconhecer todos os diferentes bichanos.

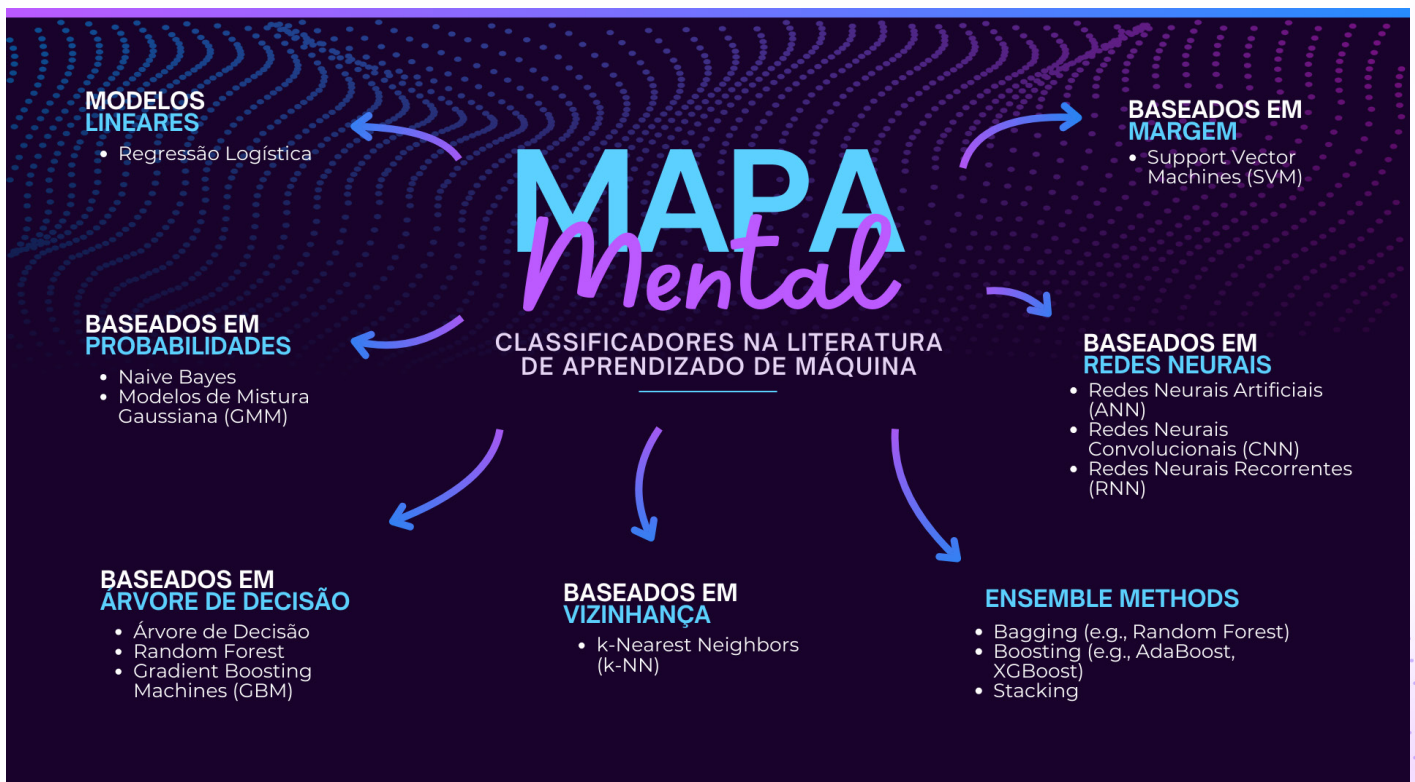
Figura 75 - Exemplos de diferentes amostras de uma classe “gato”



Fonte: autoria própria.

Na literatura de Aprendizado de Máquina temos diversos classificadores. Podemos categorizar estes métodos a partir do princípio de funcionamento de cada um como mostra o mapa mental a seguir.

Figura 76 - Mapa mental classificadores na literatura de aprendizado de máquina



Fonte: autoria própria.

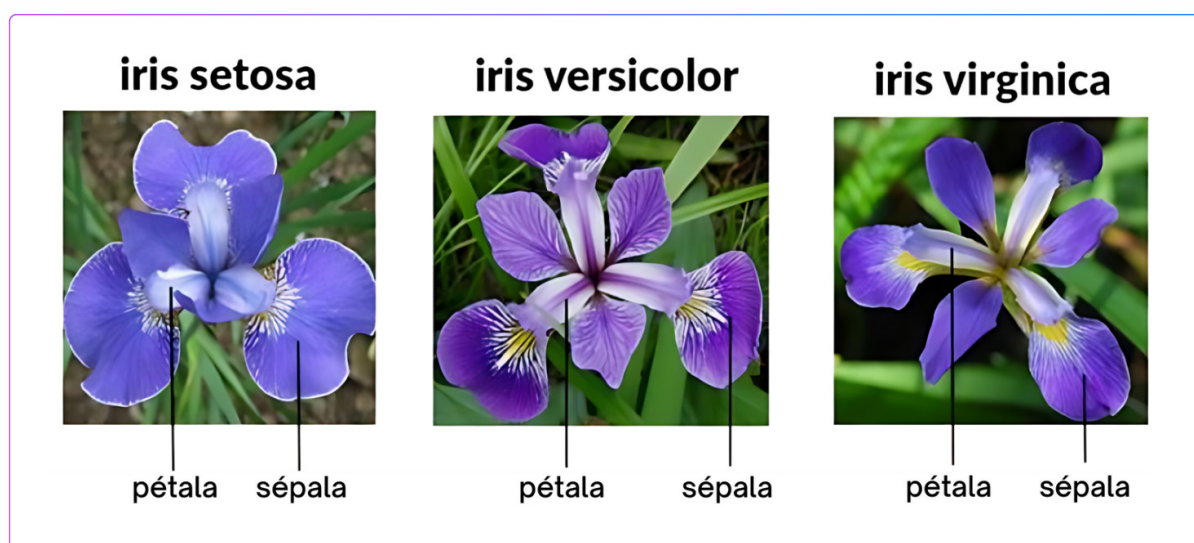
Para entendermos melhor vamos estudar alguns destes métodos de forma mais detalhada começando pelo KNN, ou k-Vizinhos Mais Próximos em português, que realiza a classificação com base na vizinhança de cada amostra de dados.

9.1.1 KNN: *k-Nearest Neighbors*

O KNN é um classificador clássico do Aprendizado de Máquina e apesar de possuir funcionamento simples e intuitivo é muito utilizado. O conceito principal do KNN é que os dados utilizados no treinamento do classificador delimitam a região do espaço de variáveis que compõem as classes com base na proximidade. Para melhor visualizar esta propriedade vamos usar como exemplo o *dataset* Íris.

O *dataset* Íris é composto por 50 amostras de três diferentes espécies de flores de plantas Íris, onde cada flor teve sua sépala e pétala medida. As partes das flores e diferentes espécies estão na imagem abaixo. O objetivo aqui é diferenciar as espécies de acordo com as características das suas flores.

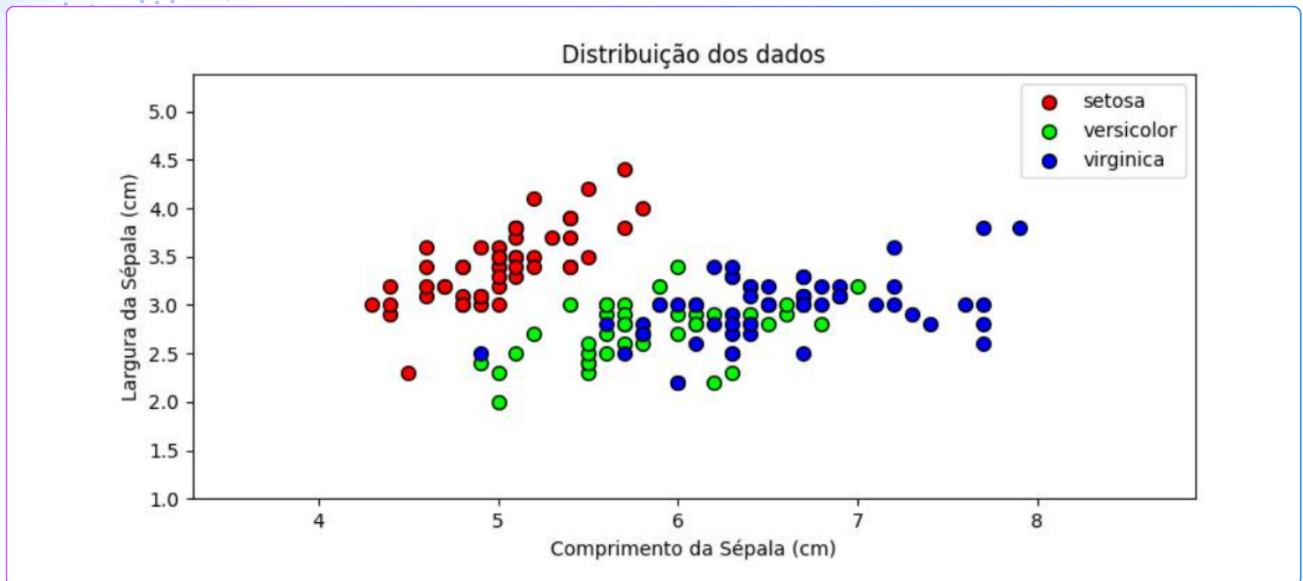
Figura 77 - Exemplos das três espécies de flores Iris



Fonte: adaptada de [Leitura de um dataset, 2020](#).

Se visualizarmos a distribuição dos dados em torno de duas das variáveis (comprimento e largura da pétala) podemos observar que as diferentes espécies formam regiões no espaço das variáveis. A espécie setosa apresenta uma sobreposição quando olharmos apenas o comprimento (eixo x) ou largura (eixo y), mas usando ambas as variáveis fica claro o surgimento de uma região própria da classe. O KNN atua justamente nesse conceito de vizinhança classificando um novo dado a partir dos k vizinhos mais próximos. Vizinhos são os dados já existentes na base de treinamento e que estão rotulados (já sabemos a sua resposta), a partir destes são encontrados os k mais próximos e feito uma votação para decidir qual classe atribuir a um novo dado. O valor de k é atribuído no processo de parametrização do algoritmo e deve ser otimizado para obter o melhor resultado.

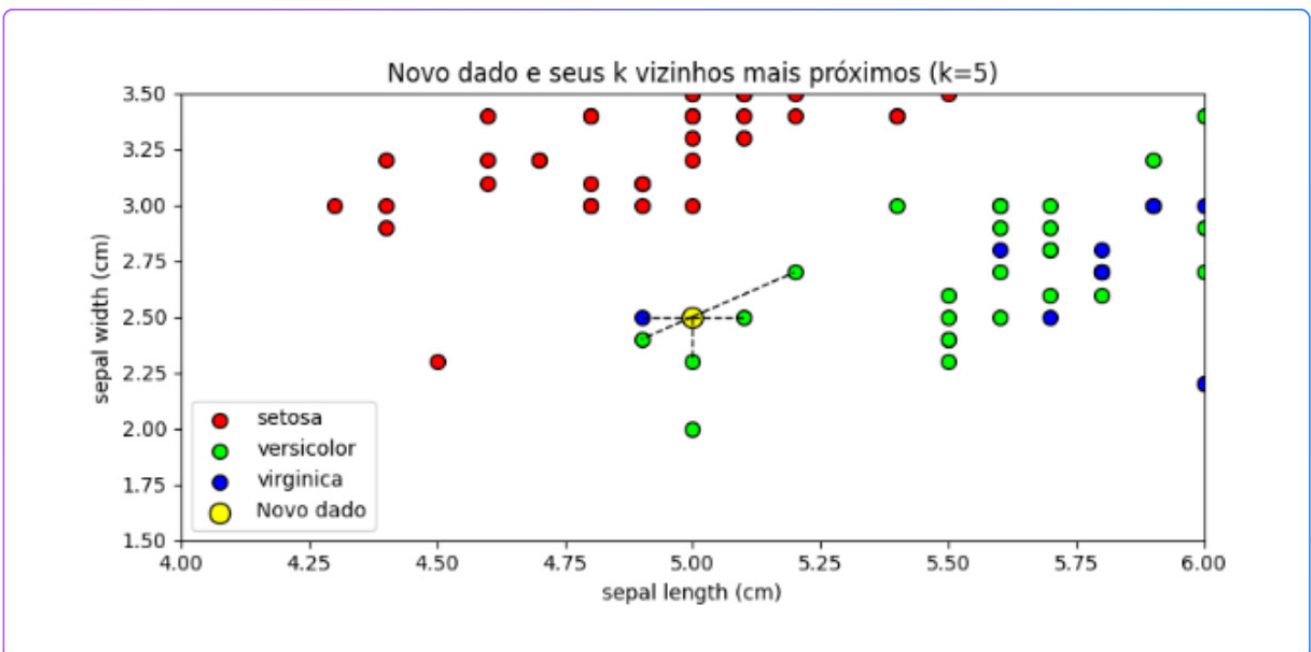
Figura 78 - Distribuição das amostras de flores com base no comprimento e largura da sépala



Fonte: autoria própria.

Dessa forma, se imaginarmos uma nova flor cuja pétala foi medida em termos de largura e comprimento, ao utilizarmos o KNN com valor de k igual a 5, iremos buscar os 5 exemplos de pétalas já conhecidos mais próximos para definir qual classe deve ser atribuída a esse novo exemplo. Na figura a seguir podemos ver um exemplo deste caso, onde o novo ponto será atribuído a versicolor por possuir 4 vizinhos mais próximos que faz parte desta classe.

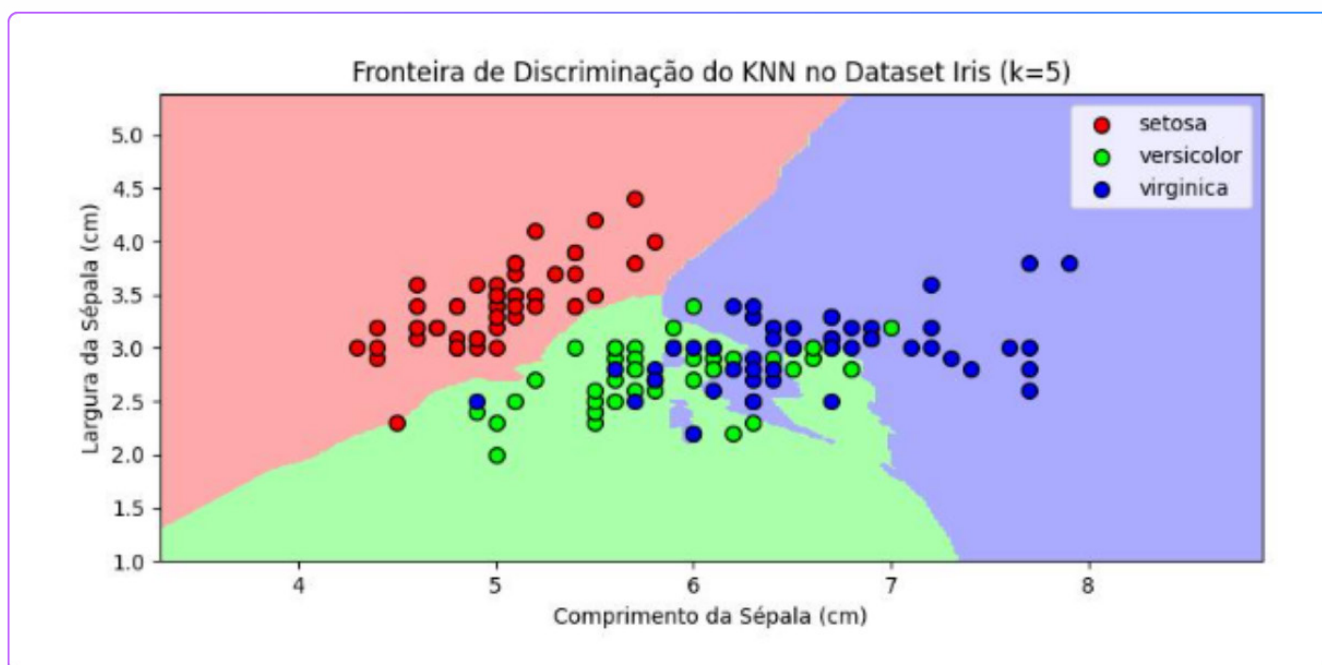
Figura 79 - Exemplo de aplicação do KNN em um novo dado



Fonte: autoria própria.

A partir deste processo podemos definir as regiões do espaço das variáveis conforme a proximidade dos 5 vizinhos mais próximos, criando assim uma fronteira de discriminação baseada na vizinhança dos dados. Na figura a seguir podemos visualizar a região de cada classe. Neste exemplo podemos ver que a classe setosa está quase perfeitamente discriminada das outras duas classes, porém as classes versicolor e virginica estão sobrepostas de uma forma que não é possível separá-las perfeitamente com o KNN. Dessa forma podemos afirmar que o KNN com apenas estas duas variáveis não é capaz de obter uma boa generalização. Uma alternativa para se obter um melhor resultado pode ser com uso das técnicas discutidas nas unidades anteriores, como: adicionar outras variáveis, normalizar ou transformar as variáveis utilizadas ou talvez remover *outliers* do conjunto de treino. De qualquer maneira, para mensurarmos a qualidade desta modelagem em classificar flores e sua capacidade de generalização é necessária uma metodologia de avaliação e métricas objetivas.

Figura 80 - Fronteira de discriminação de um KNN



Fonte: autoria própria.

9.2 Avaliação de Classificadores

A avaliação de modelos de Aprendizado de Máquina varia conforme o problema a ser tratado. No caso de um problema de classificação, há alguns critérios específicos que precisam ser observados.

O primeiro ponto a ser tratado é a divisão dos dados em conjuntos de treino e teste, ou em treino, validação e teste. A divisão dos dados deve ser coerente com o problema, de forma que cada amostra de treino seja independente das amostras de teste. Por exemplo, em um problema de imagens médicas, caso existam várias imagens de um mesmo paciente, elas devem ser alocadas ou no conjunto de treino ou no de teste, mas

sempre juntas. Dessa forma, garantimos que os conjuntos são Independentes e Identicamente Distribuídos (IID). Esse conceito vem da estatística e define que as amostras devem ser:

- » **Independentes:** Se os conjuntos de treino e teste não forem independentes, pode haver uma correlação entre os dados nos dois conjuntos. Isso pode levar o modelo a “lembrar” informações específicas do conjunto de treino e usar essas informações para obter um desempenho artificialmente alto no conjunto de teste, em vez de aprender padrões generalizáveis. Isso resulta em uma avaliação otimista e irrealista do modelo.
- » **Identicamente Distribuídos:** Se os conjuntos de treino e teste não forem identicamente distribuídos, o modelo treinado em um conjunto de dados pode não se comportar bem no conjunto de teste, pois os padrões que ele aprendeu podem não ser aplicáveis ao conjunto de teste. Isso levaria a uma avaliação incorreta do modelo, onde o desempenho no conjunto de teste não reflete o desempenho esperado em novos dados do mundo real.

Esse cuidado deve ser considerado para que a avaliação reflita de forma mais fidedigna como o modelo se comportará no futuro ao lidar com novos dados, conseguindo assim, mensurar a capacidade de generalização e identificando ao mesmo tempo um possível *overfitting* (sobreajuste), que no caso seria equivalente a “decorar” o conjunto de treino, que leva o classificador a um alto erro nos dados de teste.

Por fim, após treinar o modelo e executar o conjunto de teste, mensuramos a qualidade das predições a partir dos rótulos já conhecidos. Para isso temos métricas específicas para problemas de classificação.

9.2.1 Métricas para Classificação

Dentre as métricas a serem usadas, alguns pontos de atenção são importantes a serem considerados. Primeiro, o objetivo da aplicação. Cada métrica tem como função avaliar alguma qualidade ou defeito específico dos modelos, não existem métricas universais. Assim, é necessário avaliar com base no objetivo da aplicação que está sendo desenvolvida qual é o resultado esperado. Como por exemplo em um sistema médico voltado a classificação de risco de pacientes que chegam ao pronto atendimento em risco baixo e alto. Se o objetivo for fazer uma triagem automatizada espera-se que o modelo acerte as classificações em geral, e também é preciso verificar o desempenho por classe, para garantir que os pacientes de alto risco estão sendo classificados corretamente, pois são os casos mais críticos. Caso o objetivo seja algo parcialmente automatizado, de forma que um profissional de saúde irá receber uma probabilidade de risco entre 0 e 1 e interpretar tal informação, já é preciso avaliar não só se a classificação é correta, mas o quanto precisa ela é, pois se em dois casos de altíssimo risco um tiver probabilidade de 0,6 e o outro de 0,9, isso pode gerar interpretações diferentes.

Para interpretarmos as métricas, é necessário entender o contexto de classificação como a tentativa de classificar corretamente cada classe individualmente. Nesse contexto, usamos os termos “positivos” e “negativos” para se referir a pertencente ou não pertencente à classe. Ao realizar a classificação, temos os casos positivos classificados corretamente, chamados de Verdadeiros Positivos, e os classificados erroneamente como negativos, chamados de Falsos Negativos. Da mesma forma, temos os negativos classificados corretamente, chamados de Verdadeiros Negativos, e os erroneamente classificados como positivos, chamados de Falsos Positivos. Esses quatro valores formam a chamada matriz de confusão, a partir da qual extraímos as métricas.

Figura 81 - Matriz de confusão e seus componentes

		Valor Predito	
		Sim	Não
Real	Sim	Verdadeiro Positivo (TP)	Falso Negativo (FN)
	Não	Falso Positivo (FP)	Verdadeiro Negativo (TN)

Fonte: autoria própria.

A partir das combinações dos quatro valores temos as principais métricas:

9.2.1.1 Acurácia

Equação:

$$Acurácia = \frac{TP+TN}{TP + TN + FP + FN}$$

Definição: Acurácia é a proporção de acertos com relação ao número de respostas, sendo assim a métrica mais intuitiva pois nos entrega uma medida do quanto o modelo acerta/erra.

Pontos positivos:

- » Fácil compreensão

Pontos negativos:

- » Pode ser enganosa em conjuntos de dados desbalanceados, pois um modelo que prevê a classe majoritária para todos os exemplos ainda pode ter alta acurácia.

9.2.1.2 Precisão

Equação:

$$Precisão = \frac{TP}{TP + FP}$$

Definição: A precisão mede a proporção de instâncias corretamente identificadas como positivas em relação ao total de instâncias previstas como positivas. Ela indica o quanto as previsões positivas do modelo são confiáveis.

Pontos positivos:

- » Útil quando o custo de um falso positivo é alto.
- » Boa métrica quando se deseja minimizar alarmes falsos.

Pontos negativos:

- » Não considera os falsos negativos, o que pode ser problemático em casos onde a identificação de todos os positivos é crucial.

9.2.1.3 Recall

Equação:

$$Recall = \frac{TP}{TP + FN}$$

Definição: O *recall* (ou sensibilidade) mede a proporção de instâncias positivas corretamente identificadas pelo modelo em relação ao total de instâncias positivas reais. Ele indica a capacidade do modelo de detectar todos os positivos.

Pontos positivos:

- » Útil quando o custo de um falso negativo é alto.
- » Boa métrica quando se deseja capturar o máximo de positivos possível.

Pontos negativos:

- » Pode levar a muitos falsos positivos se não for balanceado com precisão.

9.2.1.4 F1-score

Equação:

$$F1 = 2 \times \frac{\text{Precisão} \times \text{Recall}}{\text{Precisão} + \text{Recall}}$$

Definição: O F1-Score é a média harmônica da precisão e do *recall*, oferecendo um equilíbrio entre as duas métricas. Ele é especialmente útil em situações onde é necessário um equilíbrio entre precisão e *recall*.

Pontos positivos:

- » Útil em cenários de classes desbalanceadas.
- » Balanceia os erros de falsos positivos e falsos negativos.

Pontos negativos:

- » Pode ser difícil de interpretar isoladamente.
- » Não diferencia a importância relativa de precisão e *recall*.

9.2.1.5 Matriz de confusão

Definição: A própria matriz de confusão pode ser usada como métrica, pois mostra o desempenho do modelo de classificação ao exibir o número de instâncias verdadeiras e falsas para cada classe. Ela ajuda a entender onde o modelo está cometendo erros.

Pontos positivos:

- » Proporciona uma visão detalhada do desempenho do modelo.

Pontos negativos:

- » Pode ser difícil de interpretar quando o número de classes é muito grande.
- » Não oferece uma métrica única de desempenho, exigindo cálculo de outras métricas para análise quantitativa.

Como podemos ver, nenhuma métrica é perfeita. As diferentes formas de interpretar acertos e erros nos permitem avaliar a performance de um classificador sob diferentes ângulos. Agora, vamos refletir sobre como essas métricas podem ser aplicadas em diferentes problemas.



9.2.1.6 Acurácia em Detecção de Fraude

Problema: Um banco quer identificar transações fraudulentas em tempo real.

Uso da Acurácia: A acurácia pode ser usada como uma métrica geral para avaliar o modelo se as classes de transações fraudulentas e não fraudulentas estiverem balanceadas.

Dificuldade: Se 99% das transações são legítimas, um modelo com 99% de acurácia pode parecer bom, mas pode estar ignorando completamente as fraudes (1% dos casos). Nesse cenário, a acurácia sozinha pode ser enganosa.

Alternativas:

- » **Recall:** Para garantir que o modelo detecte a maioria das fraudes.
- » **F1-Score:** Para equilibrar a precisão (evitar falsos positivos) e *recall* (detectar fraudes).



9.2.1.7 Precisão em Diagnóstico Médico (Detecção de Câncer)

Problema: Um sistema de apoio à decisão médica é usado para detectar câncer em imagens de exames.

Uso da Precisão: A precisão mede a proporção de diagnósticos corretos entre os casos identificados como positivos. Alta precisão significa menos falsos positivos.

Dificuldade: A precisão sozinha não garante que todos os casos de câncer sejam detectados.

Alternativas:

- » **Recall:** Para garantir que a maioria dos casos de câncer seja identificada.
- » **F1-Score:** Para equilibrar precisão e *recall*, maximizando a detecção de cânceres enquanto minimiza falsos positivos.



9.2.1.8 F1-Score em Detecção de Spam

Problema: Um sistema de e-mail precisa classificar mensagens como spam ou não-spam.

Uso do F1-Score: O F1-score equilibra precisão e *recall*, sendo útil para lidar com o trade-off entre capturar a maioria dos spams e evitar falsos positivos (e-mails legítimos marcados como spam).

Dificuldade: F1-Score pode ser difícil de interpretar isoladamente e não considera a distribuição das classes.

Alternativas:

- » **Matriz de Confusão:** Para entender a distribuição dos erros (falsos positivos e falsos negativos) e ajustar o modelo de acordo.

Alguns problemas exigem interpretações mais complexas para a avaliação de classificadores, indo além de apenas interpretar acertos e erros, mas também avaliando a qualidade da classificação. Uma das alternativas mais comuns para isso é o uso da Curva Receiver Operating Characteristic (ROC).

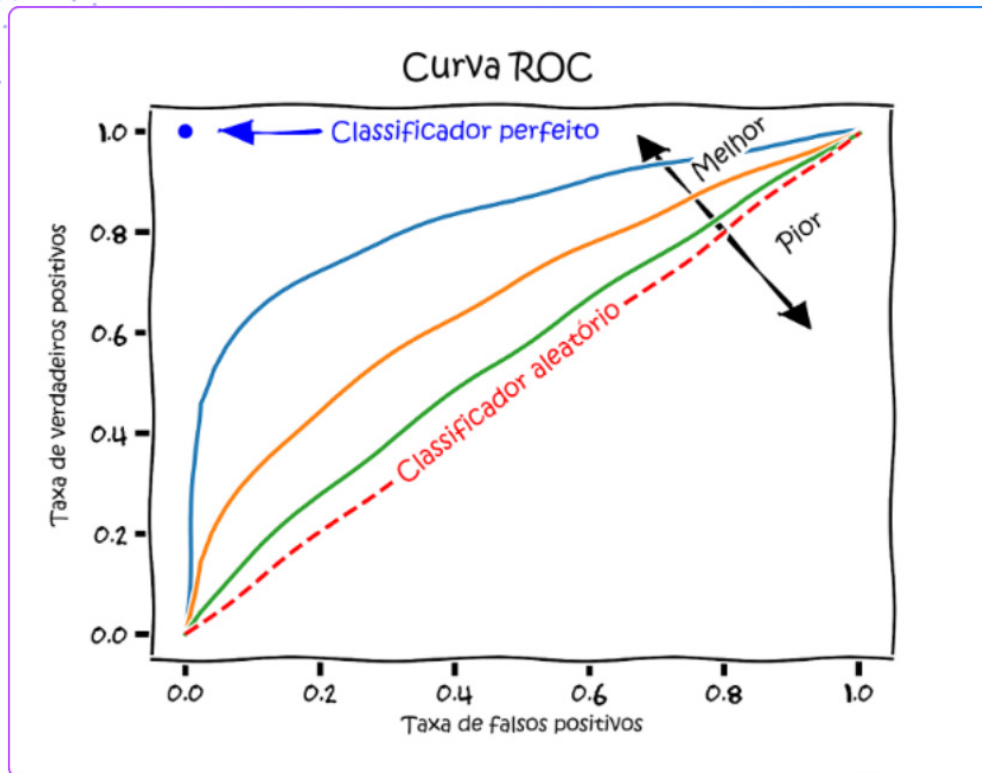


9.2.1.9 Curva ROC

A maioria dos classificadores realiza as classificações utilizando valores reais, de forma que um novo objeto pode ser classificado, por exemplo, como sendo 0,7 “gato”. Essas respostas são posteriormente convertidas para classes a fim de calcular métricas clássicas como Acurácia e Precisão, resultando em acertos e erros. No entanto, ao analisarmos mais a fundo a qualidade da classificação, podemos considerar que, se o objeto é um gato, ele deveria ser classificado como 0,9999... “gato” e não apenas 0,7 “gato”. Esse tipo de interpretação é comum em problemas chamados binários, onde há apenas duas classes, normalmente encaradas como positivo e negativo, ou presença e ausência. Alguns problemas já discutidos na Unidade são binários, como diagnóstico médico, identificação de spam e detecção de fraude. Por outro lado, o problema das flores Íris é um problema multiclasse, onde temos três classes diferentes, e uma métrica como a acurácia, por exemplo, precisa ser calculada por classe para, então, termos uma média geral entre todas as classes.

Voltando aos problemas binários, podemos interpretar a saída numérica dos classificadores e medir a qualidade a partir da Curva ROC. A Curva ROC mensura a taxa de Verdadeiros Positivos (*Recall*) junto da taxa de Falsos Positivos ($1 - \text{Especificidade}$, outra métrica calculada a partir da matriz de confusão) a partir de diferentes limiares de interpretação das classes. É como se considerássemos que, acima de 0,1, é “gato”, depois acima de 0,2, é “gato”, acima de 0,3, é “gato”, e assim por diante. Calculando as taxas em diferentes limiares, obtemos diferentes interpretações do classificador e, por fim, desenhamos uma curva com esses valores. Ao observar a curva, podemos visualizar a qualidade geral do classificador, tendo como referência um classificador aleatório (que acerta 50% das vezes) e um classificador perfeito, que formaria uma curva cobrindo toda a área do gráfico. Na figura a seguir, podemos visualizar três exemplos de classificadores e suas Curvas ROC. Quanto mais alta a curva, maior a área e, portanto, melhor o classificador.

Figura 82 - Exemplos de Curva ROC de diferentes classificadores



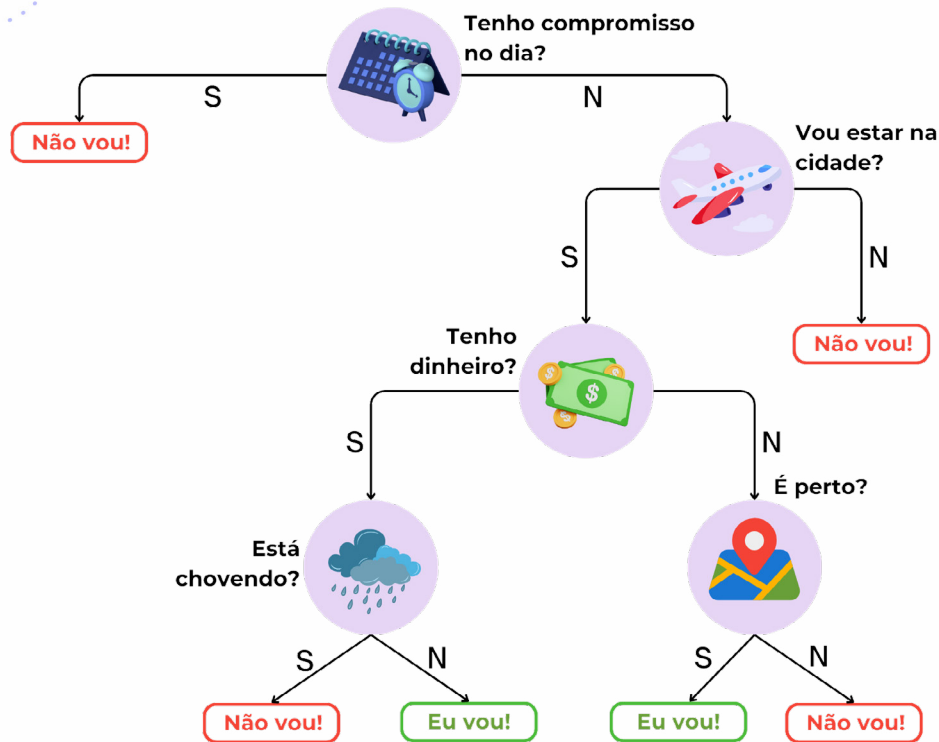
Fonte: Wikipedia, 2023.

Como a Curva ROC pode ser complexa de interpretar, uma alternativa é calcular a Área Sob a Curva (AUC - Area Under the Curve). Dessa forma, podemos resumir toda a curva em um único valor entre 0 e 1 que sintetiza a qualidade do classificador.

9.3 Classificadores Modernos

Com o tempo muitos classificadores, como os listados no início da unidade, foram propostos a fim de maximizar as métricas que discutimos anteriormente. Dentre estes os que mais obtiveram sucesso em problemas complexos foram os classificadores baseados em *Ensembles* que usam de Árvores de Decisão. O conceito de *Ensemble* refere-se à combinação de vários classificadores individuais para formar um modelo mais robusto e preciso. A ideia principal por trás dos *ensembles* é que a combinação de vários modelos "fracos" pode resultar em um modelo "forte". Os métodos de *ensemble*, como *Bagging* e *Boosting*, têm sido amplamente utilizados para melhorar o desempenho em tarefas de classificação. Enquanto isso, uma Árvore de Decisão realiza a classificação por meio de uma estrutura hierárquica, onde cada "nó" da árvore representa uma condição sobre as *features* (características) dos dados, e cada "ramo" representa o resultado de um teste sobre essa condição, conduzindo a decisões finais ou previsões nos "nós folha". Como na figura a seguir onde podemos ver um exemplo de Árvore de Decisão para decidir entre ir em um compromisso ou não.

Figura 83 - Exemplo de uma árvore de decisão sobre o problema de decidir sair de casa ou não



Fonte: autoria própria.

Dentre os classificadores que combinam *Ensemble* e Árvore de Decisão podemos destacar o *Extreme Gradient Boosting* (XGBoost) e *Light Gradient Boosting Machine* (LightGBM).



9.3.1 XGBoost

- » **Arquitetura:** XGBoost é um algoritmo baseado em *Gradient Boosting* que implementa técnicas como regularização para evitar *overfitting* (sobreajuste), além de outras otimizações como paralelização e uso de histogramas para acelerar sua execução.
- » **Desempenho:** XGBoost é conhecido por seu alto desempenho em termos de precisão e velocidade, sendo capaz de lidar com grandes conjuntos de dados e modelos complexos.
- » **Aplicações:** É amplamente utilizado em diversas áreas, desde previsão de risco de crédito até diagnóstico médico, onde a maximização de métricas como F1-score e AUC-ROC é crucial.

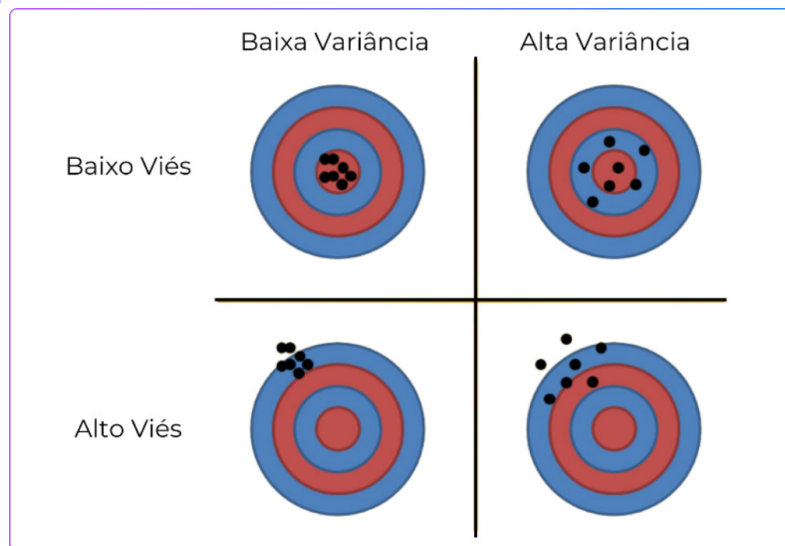
9.3.2 LightGBM

- » **Arquitetura:** *LightGBM* é outro algoritmo de *Gradient Boosting*, mas com foco em maior eficiência. Ele utiliza técnicas como o algoritmo de aprendizado baseado em folhas (leaf-wise), que cresce as árvores de decisão de maneira mais eficiente em termos de memória e tempo de computação.
- » **Desempenho:** *LightGBM* é particularmente rápido em treinar modelos em grandes volumes de dados e pode lidar melhor com características de alta dimensão e dados esparsos.
- » **Aplicações:** *LightGBM* é usado em muitos problemas industriais, incluindo classificação de texto, recomendação de produtos e detecção de fraudes, onde a precisão e a velocidade são críticas.

XGBoost e *LightGBM* são algoritmos altamente eficazes em evitar o *overfitting* (sobreajuste), graças ao uso de técnicas de regularização e otimização que melhoram a capacidade de generalização dos modelos. Eles são otimizados para alta performance, utilizando paralelização e métodos de processamento eficientes, o que permite o treinamento em grandes conjuntos de dados de maneira relativamente rápida. Além disso, esses algoritmos são extremamente flexíveis e escaláveis, suportando diferentes tipos de dados e permitindo ajustes finos nos parâmetros do modelo para alcançar o melhor desempenho possível em métricas como acurácia, F1-score e AUC-ROC. Outra vantagem significativa é a capacidade de lidar com dados desbalanceados, graças a opções internas para ajustar pesos de classe e métricas específicas, permitindo que XGBoost e *LightGBM* encontrem o equilíbrio ideal entre precisão e *recall*.

Outro ponto relevante de ambos os classificadores é a capacidade de equilibrar viés e variância, abordando de forma eficiente esse dilema clássico do Aprendizado de Máquina. O desafio de encontrar o equilíbrio ideal entre viés e variância é fundamental para o desenvolvimento de modelos eficazes e robustos. Esses dois conceitos estão diretamente ligados ao desempenho de um modelo em termos de sua capacidade de generalização para dados novos e não vistos. Na figura a seguir temos uma ilustração do efeito de ambos os conceitos.

Figura 84 - Ilustração do efeito do viés e variância



Fonte: autoria própria.

Viés: Refere-se à diferença entre as previsões médias de um modelo e os valores reais que ele está tentando prever. Um modelo com alto viés tende a fazer suposições fortes e simplistas sobre os dados, o que pode levar a subajuste (*underfitting*), onde o modelo é incapaz de capturar a complexidade dos dados de treinamento, resultando em baixa performance tanto nos dados de treinamento quanto nos dados de teste.

Variância: Refere-se à sensibilidade do modelo às variações nos dados de treinamento. Um modelo com alta variância tende a se ajustar muito de perto aos dados de treinamento, capturando até mesmo o ruído presente, o que leva a um superajuste (*overfitting*). Embora o modelo apresente alta performance nos dados de treinamento, ele falha ao generalizar para novos dados, resultando em desempenho inconsistente nos dados de teste.



9.4 Notebook Colab

No [notebook](#) a seguir iremos praticar os conceitos estudados na unidade ao estudar e solucionar um problema de classificação de vinhos através das suas características químicas.

Objetivos de Aprendizagem

- » Compreender os fundamentos do algoritmo k-Nearest Neighbors (k-NN) e sua aplicação em problemas de classificação;
- » Desenvolver habilidades práticas na manipulação e análise de dados utilizando bibliotecas Python, como Scikit-learn e Pandas;
- » Avaliar o desempenho de modelos de Machine Learning através de técnicas de validação e visualização.

Classificação usando o k-NN

Neste notebook, vamos explorar o dataset de vinhos disponível na biblioteca **sklearn.datasets**. Utilizaremos técnicas de classificação e validação para construir e avaliar um modelo de Machine Learning utilizando o algoritmo k-Nearest Neighbors (k-NN).

O que é a biblioteca Scikit-learn?

Scikit-learn é uma biblioteca de Machine Learning em Python que fornece ferramentas simples e eficientes para análise e modelagem de dados. Ela suporta tanto aprendizado supervisionado quanto não supervisionado e é amplamente utilizada devido à sua facilidade de uso e vasta gama de funcionalidades, incluindo algoritmos de classificação, regressão, clustering, e redução de dimensionalidade.

Datasets de exemplos disponíveis no Scikit-learn

A biblioteca **sklearn.datasets** inclui vários datasets de exemplo que são amplamente usados para aprendizado e benchmarking em Machine Learning. Alguns dos mais populares incluem:

- » Iris: Um conjunto de dados clássico de classificação com informações sobre 3 tipos de flores de íris.
- » Digits: Um dataset de classificação que contém imagens de dígitos escritos à mão (0 a 9).
- » Wine: O dataset que vamos utilizar neste notebook, que contém informações sobre a composição química de diferentes tipos de vinhos.
- » Breast Cancer: Dados sobre características de tumores para diagnóstico de câncer de mama.

O dataset Wine

O dataset Wine consiste em 178 amostras de vinho com 13 características químicas cada, como o teor alcoólico, a quantidade de ácido málico, magnésio, fenóis totais, e outros. Estas amostras são divididas em três classes diferentes, representando três cultivos de uva diferentes. O objetivo em muitos estudos usando este dataset é classificar corretamente o tipo de vinho com base nas suas características químicas.

```
from sklearn import datasets # Biblioteca de datasets
vinho = datasets.load_wine() # Escolha do dataset de vinhos
print(vinho.DESCR)
```

```
.. _wine_dataset:

Wine recognition dataset
-----

**Data Set Characteristics:**

: Number of Instances: 178
: Number of Attributes: 13 numeric, predictive attributes and the class
```

continua

:Attribute Information:

- Alcohol
- Malic acid
- Ash
- Alcalinity of ash
- Magnesium
- Total phenols
- Flavanoids
- Nonflavanoid phenols
- Proanthocyanins
- Color intensity
- Hue
- OD280/OD315 of diluted wines
- Proline

- class:

- class_0
- class_1
- class_2

:Summary Statistics:

	Min	Max	Mean	SD
Alcohol:	11.0	14.8	13.0	0.8
Malic Acid:	0.74	5.80	2.34	1.12
Ash:	1.36	3.23	2.36	0.27
Alcalinity of Ash:	10.6	30.0	19.5	3.3
Magnesium:	70.0	162.0	99.7	14.3
Total Phenols:	0.98	3.88	2.29	0.63
Flavanoids:	0.34	5.08	2.03	1.00
Nonflavanoid Phenols:	0.13	0.66	0.36	0.12
Proanthocyanins:	0.41	3.58	1.59	0.57
Colour Intensity:	1.3	13.0	5.1	2.3
Hue:	0.48	1.71	0.96	0.23
OD280/OD315 of diluted wines:	1.27	4.00	2.61	0.71
Proline:	278	1680	746	315

:Missing Attribute Values: None

:Class Distribution: class_0 (59), class_1 (71), class_2 (48)

:Creator: R.A. Fisher

:Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)

:Date: July, 1988

This is a copy of UCI ML Wine recognition datasets.

<https://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data>

The data is the results of a chemical analysis of wines grown in the same region in Italy by three different cultivators. There are thirteen different measurements taken for different constituents found in the three types of wine.

Original Owners:

Forina, M. et al, PARVUS -

An Extendible Package for Data Exploration, Classification and Correlation.

continua

Institute of Pharmaceutical and Food Analysis and Technologies,
Via Brigata Salerno, 16147 Genoa, Italy.

Citation:

Lichman, M. (2013). UCI Machine Learning Repository
[<https://archive.ics.uci.edu/ml>]. Irvine, CA: University of California,
School of Information and Computer Science.

```
|details-start|  
**References**  
|details-split|
```

(1) S. Aeberhard, D. Coomans and O. de Vel,
Comparison of Classifiers in High Dimensional Settings,
Tech. Rep. no. 92-02, (1992), Dept. of Computer Science and Dept. of
Mathematics and Statistics, James Cook University of North Queensland.
(Also submitted to Technometrics).

The data was used with many others for comparing various
classifiers. The classes are separable, though only RDA
has achieved 100% correct classification.
(RDA : 100%, QDA 99.4%, LDA 98.9%, 1NN 96.1% (z-transformed data))
(All results using the leave-one-out technique)

(2) S. Aeberhard, D. Coomans and O. de Vel,
"THE CLASSIFICATION PERFORMANCE OF RDA"
Tech. Rep. no. 92-01, (1992), Dept. of Computer Science and Dept. of
Mathematics and Statistics, James Cook University of North Queensland.
(Also submitted to Journal of Chemometrics).

```
|details-end|
```

O código acima importa a biblioteca de datasets do **sklearn** e carrega o dataset de vinhos usando **datasets.load_wine()**. O comando **print(vinho.DESCR)** exibe uma descrição detalhada do dataset, incluindo informações sobre as características e classes, que é útil para entender melhor a estrutura dos dados.

```
print (vinho.feature_names) # Mostra os rótulos das características  
print (vinho.target_names) # Mostra os rótulos das classes  
print (vinho.data[0:5]) # Mostra as 5 primeiras amostras do dataset  
print(vinho.target) # Mostra os identificadores das classes  
print(vinho.data.shape) # Mostra o tamanho total do dataset
```

```
['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium', 'total_phenols', 'flavanoids', 'nonflavanoid_phenols', 'proanthocyanins', 'color_intensity', 'hue', 'od280/od315_of_diluted_wines', 'proline']  
['class_0' 'class_1' 'class_2']  
[[1.423e+01 1.710e+00 2.430e+00 1.560e+01 1.270e+02 2.800e+00 3.060e+00  
 2.800e-01 2.290e+00 5.640e+00 1.040e+00 3.920e+00 1.065e+03]  
 [1.320e+01 1.780e+00 2.140e+00 1.120e+01 1.000e+02 2.650e+00 2.760e+00  
 2.600e-01 1.280e+00 4.380e+00 1.050e+00 3.400e+00 1.050e+03]  
 [1.316e+01 2.360e+00 2.670e+00 1.860e+01 1.010e+02 2.800e+00 3.240e+00  
 3.000e-01 2.810e+00 5.680e+00 1.030e+00 3.170e+00 1.185e+03]  
 [1.437e+01 1.950e+00 2.500e+00 1.680e+01 1.130e+02 3.850e+00 3.490e+00  
 2.400e-01 2.180e+00 7.800e+00 8.600e-01 3.450e+00 1.480e+03]
```

continua

```
[1.324e+01 2.590e+00 2.870e+00 2.100e+01 1.180e+02 2.800e+00 2.690e+00
3.900e-01 1.820e+00 4.320e+00 1.040e+00 2.930e+00 7.350e+02]]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2]
(178, 13)
```

Aqui, estamos explorando o dataset mais a fundo. Usamos

- » **vinho.feature_names** para listar os nomes das características (atributos) dos dados.
- » **vinho.target_names** para listar os nomes das classes (tipos de vinho).
- » **vinho.data[0:5]** para visualizar as cinco primeiras amostras do dataset.
- » **vinho.target** para ver os identificadores das classes correspondentes às amostras.
- » **vinho.data.shape** para obter a dimensão do dataset, mostrando o número total de amostras e características.

Agrupamento dos identificadores das classes com List Comprehension

Para entender a distribuição das classes no dataset, utilizamos a função **groupby** da biblioteca **itertools**. Esta função nos permite agrupar as amostras de acordo com suas classes e contar quantas amostras pertencem a cada classe. A técnica de list comprehension utilizada aqui é um método conciso e eficiente para criar listas em Python.

```
from itertools import groupby
[len(list(group)) for _, group in groupby(vinho.target)]
```

[59, 71, 48]

Este código executa os seguintes passos:

1. **groupby(vinho.target)**: Agrupa os identificadores das classes (**vinho.target**) sequencialmente.
2. **for _, group in groupby(vinho.target)**: Itera sobre cada grupo criado pelo **groupby**. O primeiro valor (chave de agrupamento) é ignorado (**_**), e o segundo valor (**group**) é a lista de itens pertencentes a esse grupo.
3. **len(list(group))**: Para cada grupo, converte-o em uma lista e calcula o tamanho dessa lista, que representa o número de amostras naquela classe.
4. **[len(list(group)) for _, group in groupby(vinho.target)]**: Gera uma lista com o número de amostras para cada classe.

Divisão do dataset em amostras treinamento e de teste

Uma prática comum em Machine Learning é dividir o dataset em conjuntos de treinamento e teste. O conjunto de treinamento é usado para treinar o modelo, enquanto o conjunto de teste é usado para avaliar seu desempenho em dados que não foram vistos durante o treinamento. Aqui, utilizamos a função `train_test_split` do `sklearn` para realizar essa divisão.

Escolhendo a proporção dos dados

A proporção entre os conjuntos de treinamento e teste é geralmente escolhida com base no tamanho do dataset e na necessidade de obter um bom balanceamento entre o treinamento do modelo e a validação de sua performance. Uma divisão comum é 70% dos dados para treinamento e 30% para teste. Isso proporciona um conjunto de treinamento suficientemente grande para modelar bem os dados, enquanto mantém um conjunto de teste significativo para avaliação.

```
from sklearn.model_selection import train_test_split
X_treino, X_teste, y_treino, y_teste = train_test_split(vinho.data, vinho.target,
test_size=0.3)
print(y_treino) # Amostras selecionadas para treinamento
print(y_teste) # Amostras selecionadas para teste
```

```
[0 1 0 2 2 1 1 1 0 1 0 0 2 1 0 1 0 2 1 0 2 0 2 2 1 2 1 1 1 2 0 2 1 1 0
 2 1 1 0 2 1 0 2 2 1 0 0 0 1 1 1 1 1 1 0 0 2 2 2 2 1 1 0 1 2 2 0 0 1 2 2 0
 2 0 2 2 0 0 1 1 1 1 2 1 0 0 1 2 1 2 1 2 0 1 1 1 0 1 1 1 1 0 2 1 1 1 0 1 2
 2 0 0 1 2 1 2 0 0 1 0 1 2]
[0 1 2 2 1 1 1 1 1 2 0 0 1 0 0 2 1 1 0 0 1 0 2 0 1 2 1 2 2 1 0 0 0 0 0 1 2
 2 1 2 0 1 0 0 1 0 0 2 1 2 0 1 0 0]
```

O código divide o dataset em dois subconjuntos: `X_treino` e `y_treino` (dados e classes de treinamento), e `X_teste` e `y_teste` (dados e classes de teste). O parâmetro `test_size=0.3` indica que 30% dos dados serão usados para teste. Após a divisão, o código exibe as classes selecionadas para os conjuntos de treinamento e teste. Em machine learning é usual utilizarmos `X` (maiúsculo) para denotar uma matriz que representa os objetos nas linhas e as características nas colunas. Já o `y` é utilizado para representar a resposta dos objetos, ou seja, suas respectivas classes.

Treinamento e avaliação do modelo k-NN

Agora que os dados estão preparados, podemos treinar um modelo de classificação. Vamos usar o algoritmo k-Nearest Neighbors (k-NN), que é um dos mais simples e intuitivos em Machine Learning.

Mas como funciona o k-NN?

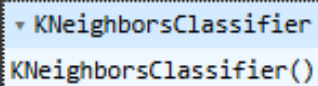
O k-NN é um algoritmo de classificação baseado em instâncias. Ele opera sob o princípio de que uma amostra deve ser classificada da mesma forma que as amostras mais próximas a ela em um espaço de características. Para determinar a classe de uma nova amostra, o k-NN analisa os `k` exemplos mais próximos no conjunto de treinamento e atribui ao objeto desconhecido a classe que é mais comum entre esses vizinhos. A esco-

lha de **k** é crítica e pode afetar significativamente o desempenho do modelo. Um valor muito pequeno de **k** pode tornar o modelo sensível ao ruído, enquanto um valor muito grande pode diluir a influência dos vizinhos mais próximos além de tender para classes majoritárias em problemas com conjunto de dados desbalanceados.

Treinamento do modelo com o método **fit**

O método **fit** do k-NN é usado para treinar o modelo. Ele simplesmente armazena as amostras de treinamento e suas classes correspondentes para posterior consulta durante a fase de predição. As entradas para o método **fit** são os dados de treinamento (**X_treino**) e as respectivas classes (**y_treino**). O método **fit** não possui saídas; ele apenas prepara o modelo para realizar predições.

```
from sklearn.neighbors import KNeighborsClassifier # Importa o classificador
knn = KNeighborsClassifier(n_neighbors=5) # Cria o modelo com k = 5
knn.fit(X_treino, y_treino) # Treinamento
```



```
KNeighborsClassifier
KNeighborsClassifier()
```

Predição com o método **predict**

Após o treinamento, podemos usar o método **predict** para prever as classes das amostras no conjunto de teste. O método **predict** recebe como entrada os dados de teste (**X_teste**) e retorna as classes previstas para essas amostras. Este é o processo onde o modelo compara as novas amostras com as armazenadas durante o treinamento e decide a classe de cada nova amostra com base nos 'k' vizinhos mais próximos.

```
y_predito = knn.predict(X_teste) # Predição
print(y_predito) # Mostra o resultado da predição
```

```
[0 1 1 1 1 2 1 1 2 2 0 0 1 0 1 2 2 1 0 1 1 0 1 0 1 1 1 2 2 1 0 0 0 0 0 1 1
 2 1 1 0 2 0 0 1 0 2 2 1 1 0 1 0 1]
```

Avaliação do modelo com o método **score**

O método **score** é utilizado para avaliar a performance do modelo após a predição. Ele calcula a acurácia, que é a proporção de amostras corretamente classificadas em relação ao total de amostras testadas. A entrada para o método **score** são os dados de teste (**X_teste**) e as classes reais (**y_teste**). A saída é a acurácia, um valor entre 0 e 1 que indica a precisão do modelo.

```
acuracia = knn.score(X_teste, y_teste) # Calcula a acurácia da predição
print(acuracia) # Mostra a acurácia da predição
```

```
0.7222222222222222
```

Validação cruzada

Para obter uma estimativa mais confiável do desempenho do modelo, utilizamos a validação cruzada. Na validação cruzada, o dataset é dividido em várias partes, ou 'folds',

e o modelo é treinado e testado múltiplas vezes, cada vez utilizando uma parte diferente como conjunto de teste e as demais como conjunto de treinamento. Existem vários tipos de validação cruzada, cada um com suas vantagens e desvantagens.

Tipos de validação cruzada

- » **k-Fold Cross Validation:** O dataset é dividido em 'k' partes, e o modelo é treinado 'k' vezes, cada vez utilizando uma parte diferente como conjunto de teste. A vantagem é que todos os dados são eventualmente utilizados tanto para treinamento quanto para teste. A desvantagem é que pode ser computacionalmente caro.
- » **Leave-One-Out Cross Validation (LOOCV):** Cada amostra do dataset é utilizada uma vez como conjunto de teste, enquanto as outras amostras formam o conjunto de treinamento. Embora seja mais preciso, é ainda mais caro computacionalmente do que o k-Fold.
- » **Stratified k-Fold:** Semelhante ao k-Fold, mas garante que a proporção de classes em cada *fold* seja representativa do dataset original, o que é especialmente útil em datasets desbalanceados.

Validação cruzada com `cross_val_score`

A função `cross_val_score` facilita a realização da validação cruzada. Ela recebe como entradas o modelo (neste caso, um classificador k-NN), os dados (`vinho.data`), as classes (`vinho.target`) e o número de folds (`cv=5`). A função executa a validação cruzada e retorna uma lista com as acurácias obtidas em cada iteração. A média dessas acurácias pode ser utilizada como uma estimativa da performance geral do modelo.

```
from sklearn.model_selection import cross_val_score
import numpy as np
knn = KNeighborsClassifier(n_neighbors=3)
acuracias_vc = cross_val_score(knn, vinho.data, vinho.target, cv=5)
print(acuracias_vc)
print('Média das acurácias: {}'.format(np.mean(acuracias_vc)))
```

```
[0.63888889 0.69444444 0.66666667 0.65714286 0.85714286]
Média das acurácias: 0.7028571428571428
```

No código acima, realizamos uma validação cruzada 5-folds. O modelo k-NN com **k=3** é avaliado 5 vezes, cada vez com uma parte diferente dos dados sendo usada para teste. O resultado mostra as acurácias de cada uma das 5 iterações, e a média dessas acurácias é calculada e exibida.

Avaliando diversos valores de **k**

No código abaixo, geramos um gráfico de linha representando os valores das acurácias do modelo k-NN com k variando entre 10 e 35, de 5 em 5, aplicando a validação cruzada.

```

import seaborn as sns
import matplotlib.pyplot as plt

# Lista dos valores de 'k' para o k-NN
ks = [10, 15, 20, 25, 30, 35]

# Lista vazia para armazenar as acurácias médias
acuracias = []

# Loop para calcular a acurácia média para cada valor de 'k'
for k in ks:
    # Cria o classificador k-NN com o valor de 'k' atual
    knn = KNeighborsClassifier(n_neighbors=k)

    # Executa a validação cruzada 5-fold e armazena as acurácias obtidas
    acuracias_vc = cross_val_score(knn, vinho.data, vinho.target, cv=5)

    # Calcula a acurácia média das 5 dobras e adiciona à lista de acurácias
    acuracias.append(np.mean(acuracias_vc))

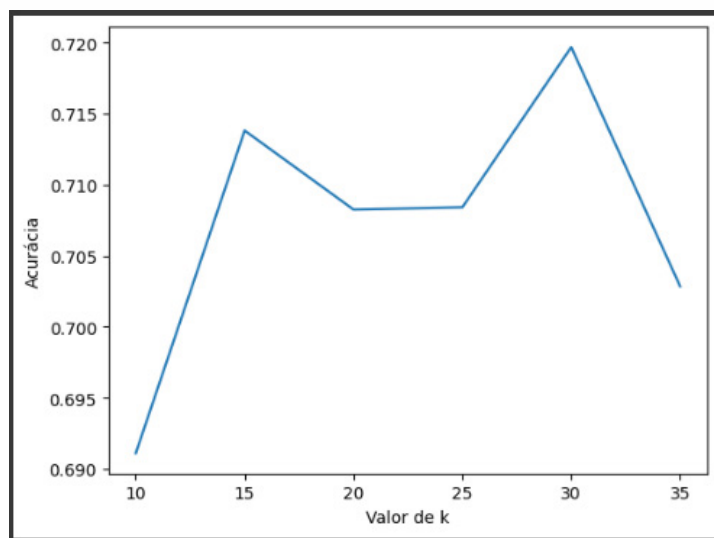
# Criar o gráfico de linha com os valores de 'k' no eixo x e as acurácias médias no eixo y
sns.lineplot(x=ks, y=acuracias)

# Adicionar as legendas dos eixos
plt.ylabel('Acurácia')
plt.xlabel('Valor de k')

# Mostrar o gráfico na tela
plt.show()

```

Figura 85 - Gráfico de linha representando os valores das acurácias do modelo k-NN



Fonte: autoria própria.

O código começa importando as bibliotecas necessárias: **Seaborn** para a criação de gráficos e **Matplotlib** para manipulação de gráficos. Em seguida, uma lista de valores de **k** é definida: **[10, 15, 20, 25, 30, 35]**. Esses valores serão usados para configurar o número de vizinhos que o algoritmo k-NN considera ao fazer previsões. Para cada valor de **k**, o código realiza o seguinte:

- » Cria um modelo k-NN com o valor atual de **k**.
- » Executa uma validação cruzada 5-fold usando o modelo e o dataset de vinhos.
- » Calcula a acurácia média dos 5 folds da validação cruzada.
- » Armazena a acurácia média em uma lista para posterior plotagem no gráfico.

Após calcular as acurácias médias para todos os valores de **k**, o código utiliza **Seaborn** para criar um gráfico de linha. O eixo **x** do gráfico representa os valores de **k**, enquanto o eixo **y** mostra as acurácias médias correspondentes. Por fim, o gráfico gerado é exibido, permitindo uma visualização clara de como o desempenho do modelo k-NN varia com diferentes valores de **k**. Isso ajuda a identificar qual valor de **k** proporciona a melhor acurácia para o dataset utilizado.



SAIBA MAIS...

- » k-nn - <https://youtu.be/HVXime0nQeI?si=zRbX-4XwNUouNy9h>
- » Árvores de decisão - https://www.youtube.com/watch?v=_L39rN6gz7Y
- » Florestas aleatórias - https://www.youtube.com/watch?v=J4Wdy0Wc_xQ
- » Regressão Logística - <https://www.youtube.com/watch?v=yIYKR4sgzI8>
- » SVM - <https://www.youtube.com/watch?v=efR1C6CvhmE&feature=youtu.be>
- » ROC e AUC - <https://youtu.be/4jRBRDbJemM?si=HFvY9Ish7trA8Ww1>
- » Matriz de Confusão - <https://www.youtube.com/watch?v=Kdsp6soqA7o>
- » Métricas - <https://towardsdatascience.com/understanding-confusion-matrix-a-9ad42dcfd62>

PARA RELEMBRAR...

Classificação:

- » Tarefa fundamental em aprendizagem de máquina
- » Atribui categorias ou rótulos a dados de entrada
- » Ampla aplicabilidade em medicina, finanças, marketing e segurança

Aplicações:

- » Diagnóstico de doenças
- » Detecção de fraudes bancárias
- » Análise de sentimentos
- » Reconhecimento de imagens

Métricas de avaliação:

- » Precisão: proporção de previsões positivas corretas
- » *Recall* (sensibilidade): proporção de instâncias positivas identificadas corretamente
- » *F1-score*: média harmônica entre precisão e *recall*
- » Matriz de confusão: visão detalhada do desempenho do modelo

Algoritmos básicos:

- » k-NN (Vizinhos mais próximos)
- » Árvores de Decisão
- » Florestas Aleatórias
- » Regressão Logística
- » SVM

Curva ROC e AUC:

- » ROC: representa o desempenho do modelo em diferentes limiares
- » AUC: área sob a curva ROC, medida agregada de desempenho
- » AUC de 1,0 indica classificador perfeito, 0,5 indica classificador aleatório
- » Úteis para comparar modelos e selecionar limiares ótimos, especialmente com classes desbalanceadas

Unidade X

Agrupamento



Unidade X - Agrupamento

Dentre as técnicas de aprendizado não supervisionado, a mais comum é o agrupamento, também conhecido como clusterização (do inglês *clustering*). O objetivo do agrupamento é encontrar padrões em dados não rotulados. Diferente da classificação, que identifica o padrão nas variáveis que representam uma classe, no agrupamento buscamos padrões apenas observando as variáveis em busca de grupos, ou clusters, que são subconjuntos dos dados com comportamentos em comum, mas que apresentam diferenças entre si.

Um exemplo de agrupamento é a interpretação de padrões de consumo de clientes. Imaginando uma loja *online* que vende calçados, podemos ter diversos tipos de clientes. Alguns podem comprar mais calçados esportivos, outros preferem calçados casuais, e alguns apenas opções mais formais. Os clientes não possuem rótulos explícitos para determinarmos classes, mas, ao analisarmos o padrão de consumo, podemos identificar perfis comuns. Ao observar o histórico de compras combinado com dados como idade, gênero e profissão, é possível agrupar os clientes em agrupamentos que reúnem perfis de compra semelhantes, ao mesmo tempo que separam grupos com perfis diferentes.

Os diferentes algoritmos de agrupamento buscam identificar similaridades entre os dados enquanto tentam compreender a dissimilaridade entre os grupos. Baseado nas diferentes abordagens para tratar o problema, podemos listar alguns desses métodos no mapa mental a seguir.

Figura 86 - Mapa mental com os algoritmos de agrupamento

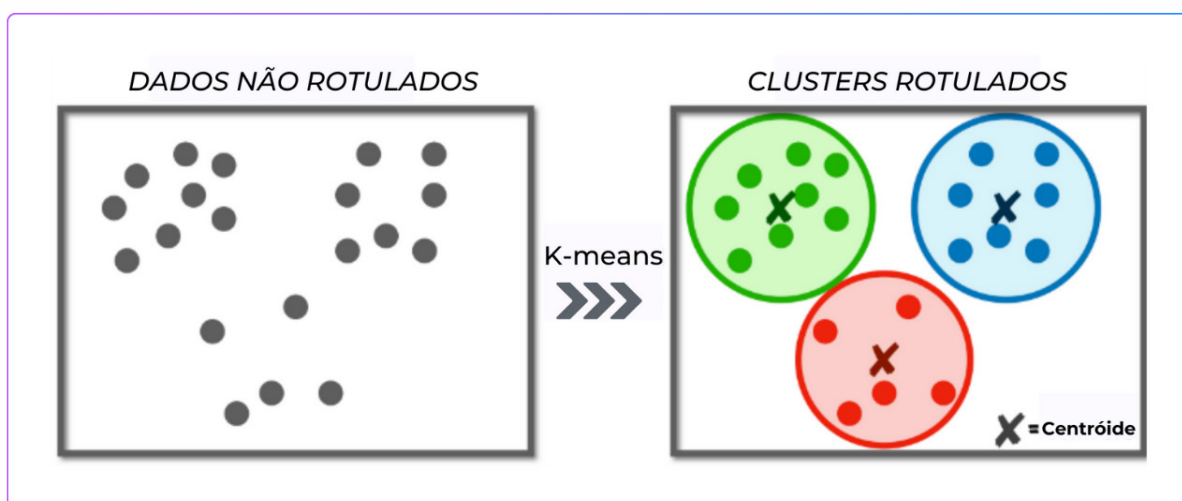


Fonte: autoria própria.

10.1 K-Means

O algoritmo de agrupamento mais comum é o *K-Means*. Ele pertence à categoria dos algoritmos de clusterização baseados em partição, cujo objetivo é dividir um conjunto de dados em K agrupamentos distintos, onde cada ponto de dados pertence ao agrupamento mais próximo do seu centroide. Um centroide é o ponto central do agrupamento e é usado para representar o “centro” do mesmo. Ele é calculado como a média aritmética das coordenadas de todos os pontos de dados que pertencem a esse agrupamento. O número de agrupamentos, K , é definido pelo usuário e ao fim da execução o esperado é ter K diferentes agrupamentos como demonstrado na Figura a seguir.

Figura 87 - Ilustração de um *K-Means* aplicado com três agrupamentos



Fonte: adaptada de [Murilo \(2023\)](#).

O algoritmo *K-Means* segue um processo iterativo que pode ser resumido nos seguintes passos:

1. Inicialização:

- » O usuário define o número de agrupamentos K que deseja encontrar.
- » Inicialmente, os centroides são escolhidos aleatoriamente. Esses centroides podem ser selecionados diretamente dos pontos de dados ou gerados aleatoriamente dentro do espaço de variáveis.

2. Atribuição de Agrupamentos :

- » Cada ponto de dados é atribuído ao agrupamento cujo centroide é o mais próximo. A proximidade é geralmente medida pela distância euclidiana.

3. Atualização dos Centroides:

- » Depois que todos os pontos foram atribuídos a seus respectivos agrupamentos, os centroides de cada agrupamento são recalculados.

O novo centroide é a média aritmética de todos os pontos dentro do agrupamento.

4. **Convergência:**

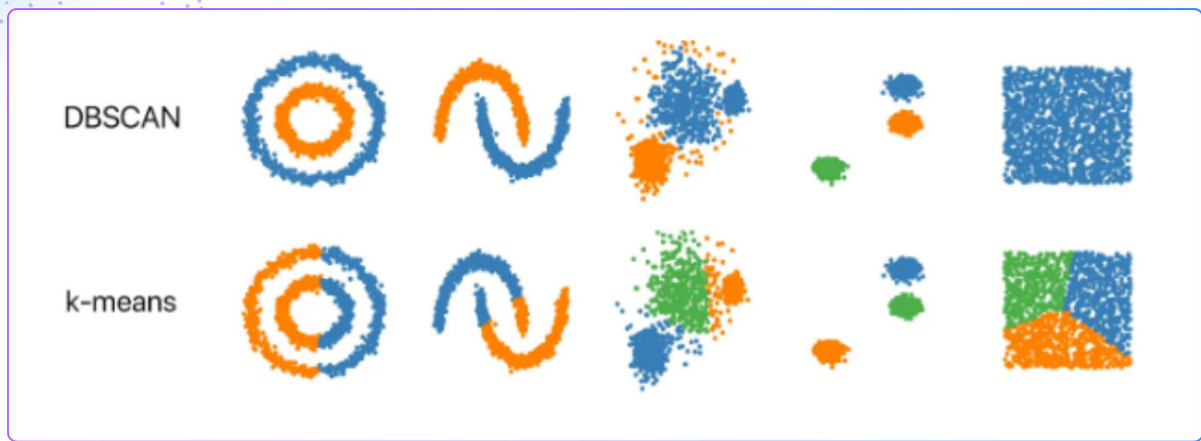
- » Os passos de atribuição de agrupamentos e atualização dos centroides são repetidos iterativamente até que os centroides não mudem mais significativamente ou até que um número máximo de iterações seja atingido. A convergência é geralmente alcançada quando a atribuição dos pontos aos agrupamentos não muda mais, ou seja, os centroides permanecem estáveis. Para calcular a estabilidade dos centroides é usado o cálculo da inércia que refere-se à soma das distâncias quadradas entre cada ponto de dados e o centroide.

O algoritmo *K-Means* é eficiente para uso em grandes bases de dados e possui uma convergência relativamente rápida. Por outro lado, o parâmetro *K* pode ser um desafio em sua aplicação, pois é necessário definir previamente a quantidade de agrupamentos a serem encontrados. Outro problema comum é a sensibilidade a *outliers*, que podem alterar significativamente a posição dos centroides. Algumas variações do *K-Means* propõem o uso de outras formas de calcular os centroides, como o *K-Medoids*, que utiliza um dado real do agrupamento como referência central, ou o *K-Medians*, que usa a mediana do agrupamento.

10.2 DBSCAN

Uma alternativa mais moderna ao *K-means* é o DBSCAN. O algoritmo possui duas principais vantagens, a capacidade de detectar *outliers* e excluir tais dados dos agrupamentos os identificando como ruído, e a definição automática do número de agrupamentos. Para alcançar tais vantagens o método funciona baseado em densidade considerando regiões de alta densidade como possíveis agrupamentos. Após definir agrupamentos iniciais ele considera as regiões próximas a partir do raio entre os pontos já pertencentes ao agrupamento, isso faz com que o método encontre agrupamentos de quaisquer formatos, o que é diferente do *K-means*, que por considerar apenas a distância para a condroide ele assuma que os agrupamentos sejam esféricos. Após encontrar as regiões de alta densidade únicas estas são consideradas agrupamentos únicos, possibilitando assim identificar quando agrupamentos existiram no conjunto de dados.

Figura 88 - Exemplos de aplicação do *K-means* e DBSCAN



Fonte: [Medium \(2020\)](#).



10.3 Avaliação de Agrupamentos

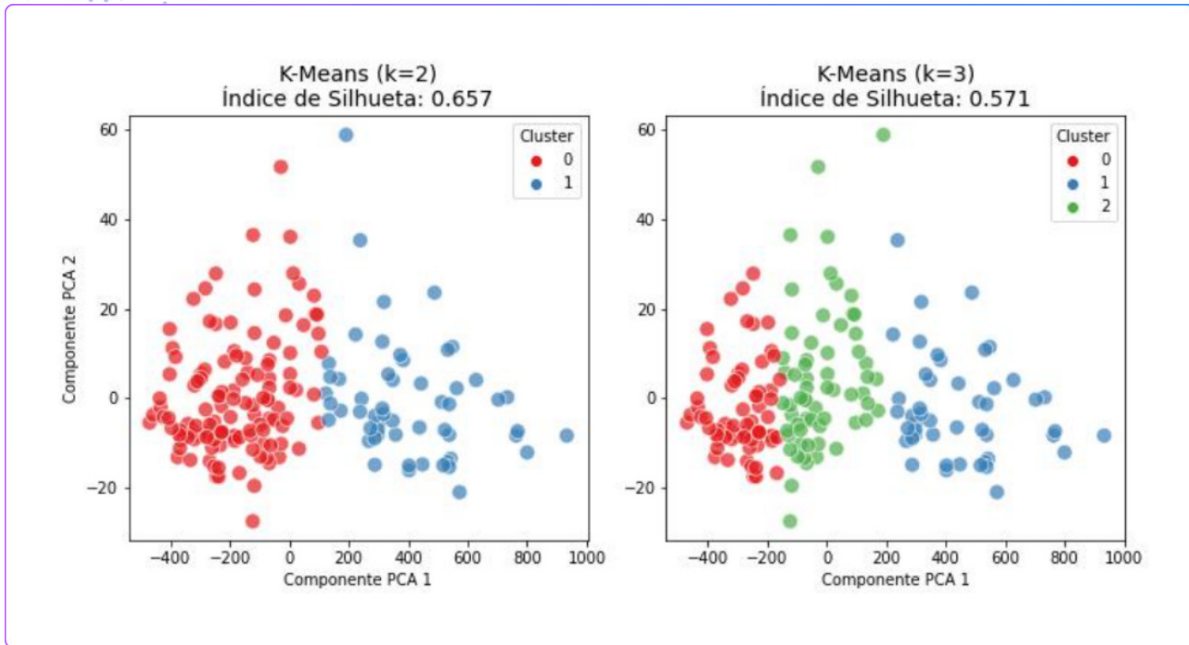
A avaliação de agrupamentos é um aspecto crucial na análise de dados, pois ajuda a determinar a qualidade e a utilidade dos clusters identificados por algoritmos de agrupamento. Diferente de tarefas de aprendizado supervisionado, onde as métricas de avaliação como acurácia, precisão e *recall* podem ser facilmente calculadas com base em rótulos conhecidos, o agrupamento geralmente é um problema de aprendizado não supervisionado, onde não se tem rótulos. Isso torna a avaliação dos agrupamentos mais desafiadora e subjetiva. Existem duas principais abordagens para avaliação de agrupamentos: avaliação interna e avaliação externa.

As métricas de avaliação interna avaliam a qualidade dos agrupamentos com base nas propriedades intrínsecas dos dados e dos agrupamentos formados, sem usar informações externas ou rótulos verdadeiros.

A métrica de avaliação interna mais comum é o Índice de Silhueta (*Silhouette Score*) que mede a coesão e a separação dos agrupamentos. Para cada ponto, a silhueta é calculada como a diferença entre a distância média ao próprio agrupamentos (coesão) e a distância média ao agrupamentos mais próximo (separação), normalizada pela maior dessas distâncias. O índice de silhueta varia de -1 a 1, onde valores próximos de 1 indicam que os pontos estão bem agrupados em seus próprios clusters, valores próximos de 0 indicam sobreposição de clusters, e valores negativos sugerem que os pontos podem estar mal agrupados.

Na figura a seguir podemos ver a avaliação do Índice de Silhueta do agrupamento do *dataset Wine* usando duas variáveis obtidas com PCA e o método *K-Means* com K igual a 2 e 3. Neste exemplo o resultado com dois agrupamentos possui maior índice e isso pode ser visualizado pela separação dos agrupamentos. Com três, o agrupamento vermelho e verde estão muito próximos, o que pode indicar que sejam apenas um e não dois agrupamentos separados.

Figura 89 - Exemplos de agrupamentos e índice de silhueta com uso do KNN com diferentes valores de k



Fonte: autoria própria.

Quando se tem ao menos alguns rótulos ou a possibilidade de rotular dados o agrupamento pode ser validado a partir destes rótulos, realizando assim a avaliação externa. A técnica mais comum para tal avaliação é o *Rand Index*, ou Índice de Rand, que calcula a porcentagem de agrupamentos corretos. O índice de Rand varia de -1 a 1, onde 1 indica um agrupamento perfeito em relação aos rótulos verdadeiros, 0 indica agrupamento aleatório e valores negativos indicam desacordo com os rótulos de referência.

10.4 Notebook Colab

No [notebook](#) a seguir realizaremos passo a passo o agrupamento do *dataset Wine* citado anteriormente com uso do algoritmo *K-Means*.

Objetivos de Aprendizagem

- » Compreender os conceitos fundamentais de clusterização e o funcionamento do algoritmo k-means;
- » Aplicar técnicas de redução de dimensionalidade e visualização de dados para interpretar resultados de clusterização;
- » Desenvolver competências na análise comparativa de resultados de clusterização com dados rotulados e não rotulados.

Clusterização com o K-Means

Neste notebook, vamos explorar o algoritmo de clusterização k-means aplicado ao

famoso dataset Iris. Este conjunto de dados contém informações sobre 150 amostras de flores de íris, divididas em três espécies diferentes. Embora o dataset seja rotulado, vamos utilizá-lo para aplicar o algoritmo k-means, que é um método não supervisionado, para comparar os resultados obtidos com as classes originais.

O que vamos fazer?

O objetivo aqui é aplicar o k-means para identificar possíveis agrupamentos nas amostras do dataset Iris, mesmo sabendo que os dados já possuem rótulos. Inicialmente, usaremos os rótulos para exibir um gráfico de dispersão dos dados originais. Em seguida, retiraremos a coluna que identifica as classes e aplicaremos o k-means, gerando um gráfico para comparar visualmente os resultados.

Montagem do Google Drive

Primeiro, precisamos acessar os arquivos armazenados no Google Drive. Usaremos o comando **drive.mount** para montar o Google Drive no ambiente Colab, permitindo acessar os arquivos diretamente.

```
from google.colab import drive
drive.mount('/content/drive') # Monta o drive do Google Drive
Mounted at /content/drive
```

O código acima monta o Google Drive no diretório [/content/drive](#), permitindo que você acesse seus arquivos diretamente no ambiente Colab.

Introdução à biblioteca Pandas

Pandas é uma poderosa biblioteca de Python para análise e manipulação de dados. Ela fornece estruturas de dados flexíveis, como DataFrames e Series, que facilitam o tratamento de dados tabulares (similar a tabelas em bancos de dados ou planilhas). Pandas é particularmente útil para carregar, limpar, transformar e analisar dados. Ele pode manipular vários tipos de dados, incluindo arquivos CSV, planilhas Excel, tabelas SQL, e muito mais.

Abaixo você encontra uma explicação detalhada cada uma delas:

DataFrame

- » O que é: Um DataFrame é uma estrutura de dados bidimensional (tabular), similar a uma tabela em uma base de dados ou a uma planilha em Excel. Ele é composto por linhas e colunas, onde cada coluna pode conter diferentes tipos de dados (números, strings, booleanos, etc.).
- » Características:
 - » Etiquetado: As linhas e colunas de um DataFrame são etiquetadas, o que

significa que você pode acessar os dados através de rótulos (ou índices) em vez de apenas por posição.

- » Flexível: Pode armazenar diferentes tipos de dados em colunas diferentes, por exemplo, inteiros em uma coluna, strings em outra, e assim por diante.
- » Manipulação e análise: É amplamente utilizado para operações de filtragem, agregação, pivotamento, e outras formas de manipulação de dados.
- » Exemplo: Imagine um DataFrame com as colunas "Nome", "Idade", e "Cidade". Cada linha desse DataFrame representa uma pessoa diferente, com os valores dessas três características.

Series

- » O que é: Uma Series é uma estrutura de dados unidimensional, similar a uma coluna de uma tabela ou a uma lista. É essencialmente uma lista de elementos com rótulos (índices), o que a torna uma versão mais poderosa de um array do NumPy.
- » Características:
 - » Etiquetado: Cada elemento de uma Series tem um rótulo de índice associado, o que facilita o acesso a dados específicos. Homogênea: Todos os elementos de uma Series são do mesmo tipo de dados (inteiros, strings, etc.).
 - » Parte de um DataFrame: Uma Series pode ser vista como uma coluna individual em um DataFrame, mas também pode existir independentemente.
- » Exemplo: A coluna "Nome" desse DataFrame pode ser extraída como uma Series, contendo apenas os nomes das pessoas, etiquetados pelos índices das linhas.

Antes de executar o código a seguir, [clique aqui](#) para baixar o arquivo **iris-dataset.csv** e salve-o na pasta raiz (fora que qualquer diretório) do Google Drive vinculado à conta que você está utilizando para acessar esse *notebook*.

```
import pandas as pd
filename = '/content/drive/My Drive/Colab Notebooks/iris-dataset.csv'
data = pd.read_csv(filename) # Lê o arquivo do dataset iris em um dataframe
```

O comando **pd.read_csv(filename)** lê o arquivo CSV e cria um DataFrame **data** contendo todas as amostras e características do dataset Iris. Esse DataFrame é essencial para manipulação e análise dos dados.

Explorando o dataset

Vamos explorar rapidamente o dataset visualizando os primeiros registros e listando as colunas do DataFrame. Isso nos ajuda a entender a estrutura dos dados que estamos manipulando.

```
data.head() # Mostra os primeiros registros do dataframe
```

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

```
list(data.columns) # Mostra os nomes das colunas do dataframe
```

```
['sepal.length', 'sepal.width', 'petal.length', 'petal.width', 'variety']
```

Os primeiros registros do DataFrame e a lista de colunas nos permitem verificar se os dados foram carregados corretamente. As colunas do dataset Iris geralmente incluem medidas como comprimento e largura das sépalas e pétalas, além da espécie da flor.

Preparação dos dados

Agora, vamos separar as características (atributos) que serão usadas para a clusterização. Neste exemplo, vamos utilizar as quatro primeiras colunas do DataFrame, que representam as medidas das flores.

```
X = data.iloc[:, 0:4].values # Separa somente as colunas das características
```

O código acima utiliza **iloc** para selecionar as quatro primeiras colunas do DataFrame **data** e armazena essas colunas na matriz **X**. Essa matriz será usada como entrada para o algoritmo de clusterização.

Redução de dimensionalidade com t-SNE

Antes de aplicar o algoritmo k-means, vamos reduzir a dimensionalidade dos dados usando t-SNE (t-Distributed Stochastic Neighbor Embedding). O t-SNE é uma técnica de redução de dimensionalidade particularmente útil para visualizar dados de alta dimensão. Ele transforma os dados de um espaço de características de alta dimensão para um espaço de 2 ou 3 dimensões, mantendo as relações de proximidade entre as amostras, o que é ideal para visualizar agrupamentos em um gráfico.

```
from sklearn.manifold import TSNE # Classe para redução da dimensionalidade
X_tsne = TSNE(n_components = 2).fit_transform(X) # Reduz de 4 para 2 características
```

set Iris para duas. A redução de dimensionalidade é especialmente útil quando queremos visualizar dados em um espaço 2D ou 3D sem perder muita informação sobre as relações entre as amostras.

Preparação para visualização

Agora que os dados foram reduzidos para duas dimensões, vamos organizá-los em um novo DataFrame, juntamente com as classes originais (espécies de íris) para facilitar a visualização.

```
data_tsne = pd.DataFrame(X_tsne) # Transforma as amostras obtidas pelo TSNE em dataframe
classes = data.iloc[:, 4] # Extrai a coluna dos rótulos das classes
data_tsne.insert(2, 'classe', classes) # Reúne as amostras e os rótulos das classes
data_tsne = data_tsne.rename(columns = {0: "carac_tsne_0", 1: "carac_tsne_1"}) #Altera os nomes das colunas
```

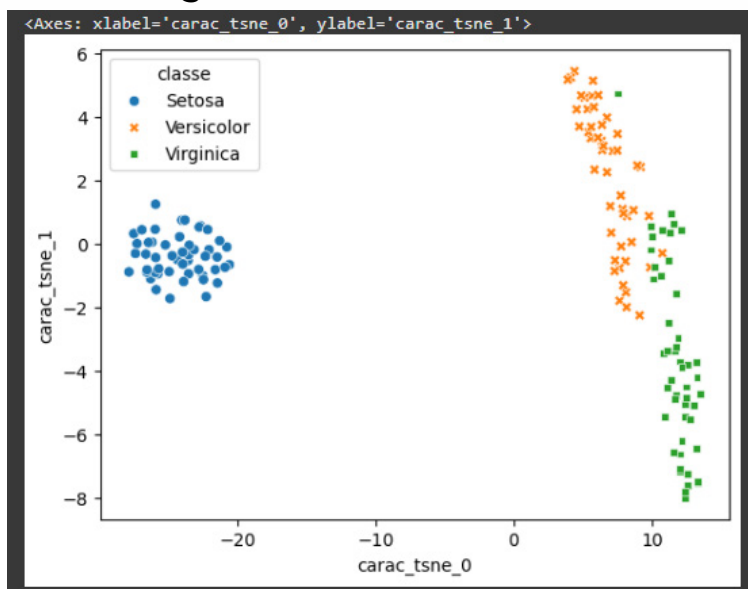
tantes do t-SNE e as classes das amostras. As colunas são renomeadas para refletir as novas características derivadas do t-SNE, facilitando a interpretação. Resta plotar esses dados em um gráfico de dispersão.

Introdução à biblioteca Seaborn

Seaborn é uma biblioteca Python baseada no Matplotlib que facilita a criação de gráficos estatísticos. Ela fornece uma interface de alto nível para desenhar gráficos atraentes e informativos. Seaborn integra bem com Pandas DataFrames, tornando muito simples a criação de gráficos a partir de dados tabulares. Seus recursos incluem gráficos de dispersão, gráficos de linha, gráficos de distribuição, gráficos de boxplot, entre outros.

```
import seaborn as sns
sns.scatterplot(x = data_tsne['carac_tsne_0'], y = data_tsne['carac_tsne_1'], hue = data_tsne.classe, style = data_tsne.classe)
```

Figura 90 - Gráfico - Seaborn



Fonte: autoria própria.

Utilizamos a função **scatterplot** para gerar o gráficos com os seguinte parâmetros:

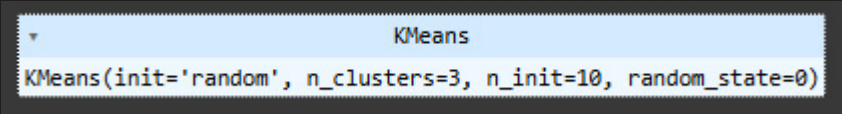
- » **x = data_tsne['carac_tsne_0']**: Define os valores para o eixo x do gráfico, que correspondem à primeira característica reduzida pelo t-SNE.
- » **y = data_tsne['carac_tsne_1']**: Define os valores para o eixo y do gráfico, que correspondem à segunda característica reduzida pelo t-SNE.
- » **hue = data_tsne.classe**: Determina a coloração dos pontos no gráfico com base na classe de cada amostra. Isso permite distinguir visualmente as diferentes classes (ou clusters) no gráfico.
- » **style = data_tsne.classe**: Diferencia as classes usando diferentes estilos de marcadores (como círculos, quadrados, etc.), proporcionando uma camada adicional de distinção visual entre as classes.

O gráfico de dispersão mostra como as amostras são distribuídas em um espaço bi-dimensional após a redução de dimensionalidade. As cores e os estilos indicam as diferentes classes, facilitando a visualização de possíveis agrupamentos naturais nos dados.

Aplicando o algoritmo K-Means

Agora que temos uma visualização inicial dos dados rotulados, vamos aplicar o algoritmo de clusterização k-means sem os rótulos das classes, ou seja, um aprendizado não supervisionado e verificar o resultado. Assim podemos comparar os 2 agrupamentos e, visualmente, checar a acurácia o k-means. K-means tenta dividir os dados em 'k' clusters, onde cada amostra pertence ao cluster mais próximo de seu centro (centroide). Após a aplicação do k-means, vamos visualizar os clusters obtidos e compará-los com as classes originais.

```
from sklearn.cluster import KMeans # Importação do algoritmo k - means
km = KMeans(n_clusters = 3, init = 'random', n_init = 10, max_iter = 300, tol = 1e-04, random_state = 0) # Parametrização do modelo
km.fit(X_tsne) # Computa os centroides dos clusters
```



The screenshot shows a Jupyter Notebook cell with the following code: `KMeans` and `KMeans(init='random', n_clusters=3, n_init=10, random_state=0)`. The output of the cell is a `KMeans` object with the same parameters.

No código acima, criamos um modelo k-means com os seguintes parâmetros:

- » **n_clusters**: Define o número de clusters a serem formados. Como sabemos que o dataset Iris tem três classes, usamos **n_clusters=3**. EM uma aplicação no mundo real esse número poderia ser escolhido com base no objetivo do agrupamento ou no deliamento experimental realizado por um especialista.
- » **init**: Método para inicialização dos centroides. Usamos **random** para escolher centroides iniciais aleatoriamente.
- » **n_init**: Número de vezes que o algoritmo k-means será executado com

diferentes centroides iniciais. O melhor resultado (com menor inércia) é retornado.

- » **max_iter**: Número máximo de iterações para uma única execução do k-means.
- » **tol**: Tolerância para declarar convergência, ou seja, o critério de parada baseado em pequenas variações na inércia.
- » **random_state**: Define a semente do gerador de números aleatórios para garantir reprodutibilidade.

Predição e atribuição aos Clusters

Após ajustar o modelo aos dados com **fit**, podemos prever a qual cluster cada amostra pertence usando **predict**. Essa etapa associa cada amostra ao cluster mais próximo, com base na distância ao centroide do cluster.

```
predicao = km.predict(X_tsne) # Associa os objetos aos índices da classe
```

O método **predict** retorna uma lista de rótulos de cluster, onde cada elemento indica o cluster ao qual a amostra correspondente pertence. Essa lista é usada para visualizar os clusters no espaço bidimensional.

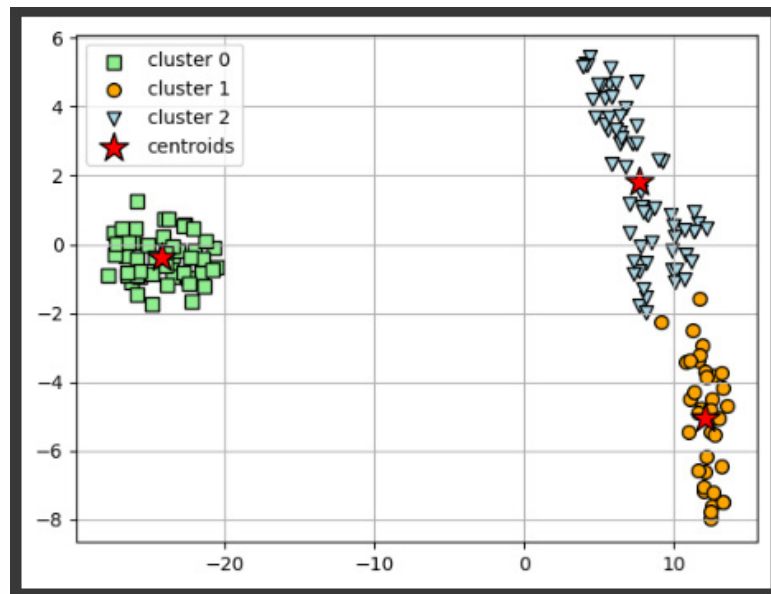
Visualizando os clusters

Vamos agora visualizar os clusters encontrados pelo k-means, colorindo cada um com uma cor diferente. Também vamos destacar os centroides dos clusters, que são os pontos centrais ao redor dos quais os clusters foram formados.

```
from matplotlib import pyplot as plt
plt.scatter(X_tsne[predicao == 0, 0], X_tsne[predicao == 0, 1], s = 50, c =
'lightgreen', marker = 's', edgecolor = 'black', label = 'cluster 0')
plt.scatter(X_tsne[predicao == 1, 0], X_tsne[predicao == 1, 1], s = 50, c = 'orange',
marker = 'o', edgecolor = 'black', label = 'cluster 1')
plt.scatter(X_tsne[predicao == 2, 0], X_tsne[predicao == 2, 1], s = 50, c =
'lightblue', marker = 'v', edgecolor = 'black', label = 'cluster 2')

# Plota os centroides de cada cluster
plt.scatter(km.cluster_centers_[:, 0], km.cluster_centers_[:, 1], s = 250, marker =
'*', c = 'red', edgecolor = 'black', label = 'centroids')
plt.legend(scatterpoints = 1)
plt.grid()
```

Figura 91 - Gráfico - Visualização de Cluster



Fonte: autoria própria.

No código acima, usamos a função **scatter** nativa do **matplotlib** para gerar o gráfico. Ao utilizar diretamente o **matplotlib**, apesar de serem necessários mais parâmetros, temos um controle maior da renderização. Nessa função:

- » **X_tsne[predicao == 0, 0]**: Combina a matriz **X_tsne** com a máscara booleana para selecionar todas as amostras (linhas) que pertencem ao cluster **0**. O **0** no final do índice indica que estamos interessados apenas na primeira coluna (a primeira dimensão) dessas amostras. Já **X_tsne[predicao == 0, 1]** fornece os valores das características da segunda coluna.
- » **s = 50**: Define o tamanho dos marcadores (pontos) no gráfico. Aqui, **s = 50** significa que cada ponto terá um tamanho moderado.
- » **c = 'lightgreen'**: Define a cor dos marcadores como verde claro (lightgreen), o que facilita a visualização e a distinção dos pontos pertencentes ao cluster 0.
- » **marker = 's'**: Define o formato dos marcadores como quadrados (s), diferenciando visualmente os pontos deste cluster dos de outros clusters que podem usar formatos diferentes.
- » **edgecolor = 'black'**: Define a cor da borda dos marcadores como preta, o que pode ajudar a destacar os marcadores no gráfico, especialmente se a cor de preenchimento for clara.
- » **label = 'cluster 0'**: Define o rótulo (label) deste conjunto de pontos como "cluster 0". Esse rótulo será utilizado na legenda do gráfico, permitindo identificar visualmente quais pontos pertencem ao cluster 0.

O gráfico resultante mostra os três clusters encontrados pelo k-means em diferentes cores, com os centroides destacados em vermelho. Essa visualização nos permite avaliar a separação dos clusters e a proximidade das amostras aos seus centroides.



PARA RELEMBRAR...

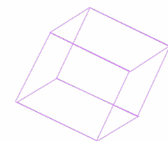
- » Apesar de semelhantes, classificação e agrupamento são técnicas distintas. Classificação temos **dados rotulados**, já sabemos de antemão qual é a resposta correta. Com métodos de agrupamento, os dados estão **sem rótulos** e buscamos identificar possíveis formas de agrupamento desses dados criando algumas classes.

SAIBA MAIS...

- » *k-means* - <https://www.youtube.com/watch?v=4b5d3muPQmA>
- » DBSCAN - <https://www.youtube.com/watch?v=RDZUdRSDOok>
- » Agrupamento hierárquico - <https://www.youtube.com/watch?v=7xHsRkOd-Vwo>
- » Coeficiente de silhueta - https://www.youtube.com/watch?v=a2Kg2_l3L8M
- » Método Cotovelo - https://www.youtube.com/watch?v=4E_DFMt60rc
- » Como analisar agrupamentos - <https://www.youtube.com/watch?v=MHnoWs-BJpeM>

Unidade XI

Redes Neurais



Unidade XI - Redes Neurais

As RNAs constituem um ramo do Aprendizado de Máquina, o qual, por sua vez, é um subcampo da IA. Inspiradas pela estrutura e função das redes neurais biológicas, as RNAs são sistemas de processamento de informação compostos por múltiplos elementos de processamento interconectados, chamados neurônios artificiais. Esses neurônios são organizados em camadas que transformam entradas em saídas, permitindo que a rede aprenda e faça inferências a partir de dados.

Dentro do espectro do ML, as RNAs são particularmente notáveis por sua capacidade de modelar relações complexas e não lineares. Elas são uma ferramenta poderosa para tarefas como classificação, regressão, e até mesmo para geração de conteúdo, como em Redes Generativas Adversárias (GANs). As RNAs se destacam principalmente quando se trata de aprender representações úteis de dados, o que é fundamental em tarefas de visão computacional, PLN e muitas outras áreas.

A história das RNAs é marcada por avanços e retrocessos, com períodos de entusiasmo seguidos por períodos de ceticismo. A ideia de simular a atividade cerebral surgiu na década de 1940, com trabalhos pioneiros como o de Warren McCulloch e Walter Pitts, que propuseram um modelo simples de neurônio artificial.

O desenvolvimento do Perceptron, na década de 1950 por Frank Rosenblatt, foi um marco importante, mas logo se mostrou limitado devido à sua incapacidade de resolver problemas não lineares. Na década de 1980, o algoritmo de retropropagação (*backpropagation*) foi popularizado, o que permitiu o treinamento eficiente de redes neurais multicamadas.

A virada do século XXI trouxe avanços significativos, impulsionados por melhorias em hardware, algoritmos e disponibilidade de dados. O surgimento das DL revolucionou campos como o reconhecimento de voz e a visão computacional. A partir daí, o interesse pelas RNAs cresceu exponencialmente, levando a aplicações práticas em diversos setores e contribuindo para o desenvolvimento de tecnologias inovadoras.

11.1 Neurônio Artificial

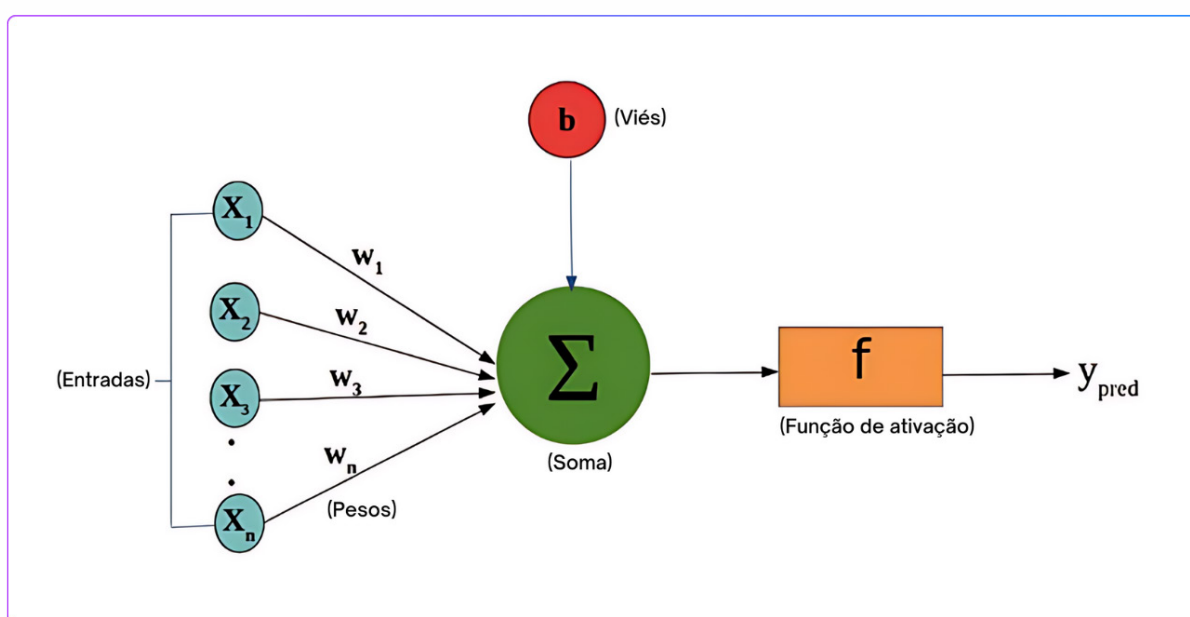
O neurônio biológico é a unidade básica de processamento no cérebro humano, caracterizado por sua capacidade de receber sinais elétricos, processá-los e transmiti-los a outros neurônios. Um neurônio artificial, por sua vez, é uma abstração simplificada desse modelo biológico. Ele recebe entradas (sinais), pondera cada uma delas com um

peso específico e as combina, geralmente adicionando um viés. A soma ponderada é então passada por uma função de ativação, que determina se e como o “neurônio” deve responder — ou seja, se deve “disparar”.

Embora a inspiração seja clara, é importante notar que a analogia não é perfeita. As RNAs são construções simplificadas que capturam apenas aspectos gerais da função neural biológica, sendo projetadas para otimizar a eficiência computacional e a capacidade de aprendizado de padrões, e não para replicar o cérebro humano com precisão.

O neurônio artificial, ilustrado na figura abaixo, recebe múltiplas entradas, que podem ser sinais de outros neurônios ou dados de entrada direta (X), cada uma com um peso (W) associado. Esses pesos representam a força das conexões entre os neurônios, similar às sinapses no cérebro humano.

Figura 92 - Esquema de um neurônio artificial - Perceptron



Fonte: adaptada de [Medium \(2024\)](#).

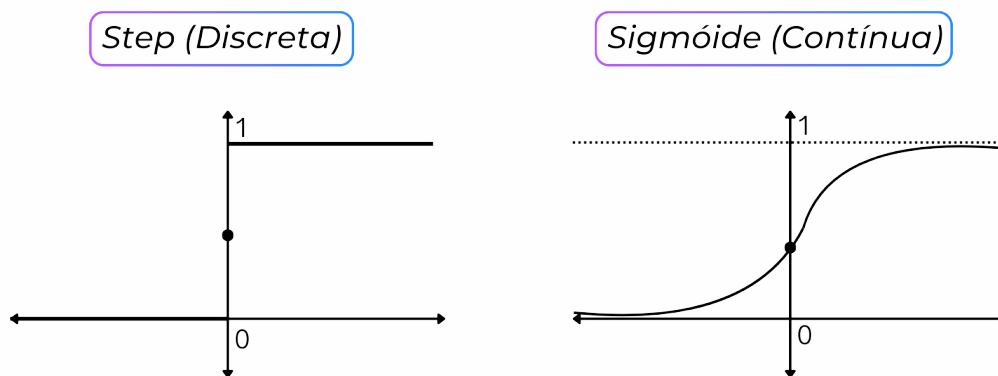
As entradas ponderadas são somadas, e um viés (b) pode ser adicionado a esta soma para ajustar o ponto de ativação do neurônio. O resultado é então passado através de uma função de ativação, que introduz não linearidade ao modelo. A função de ativação (f) determina a saída do neurônio, que pode ser enviada para outros neurônios em camadas subsequentes ou ser parte da saída final da rede.

Os pesos (W) em uma rede neural artificial são parâmetros ajustáveis que determinam a influência relativa das entradas na saída do neurônio. Durante o treinamento da rede, os pesos são modificados através do processo de aprendizado, permitindo que a rede ajuste suas respostas para melhor se adequar aos dados.

O viés, também conhecido como *bias*, é um parâmetro adicional que permite ajustar a resposta do neurônio de forma independente das entradas, funcionando como uma tentativa de representar o viés presente no problema.

A função de ativação tem a função de transformar o resultado do somatório em uma saída coerente. Inicialmente, foi utilizada uma função *step*, que respondia com 0 quando o somatório resultava em um valor negativo e com 1 quando o valor era positivo. Posteriormente, começaram a ser usadas funções diferentes, como a sigmoide, que mapeia a saída em valores contínuos entre 0 e 1, como ilustrado na figura a seguir.

Figura 93 - Funções de ativação *step* e sigmoide



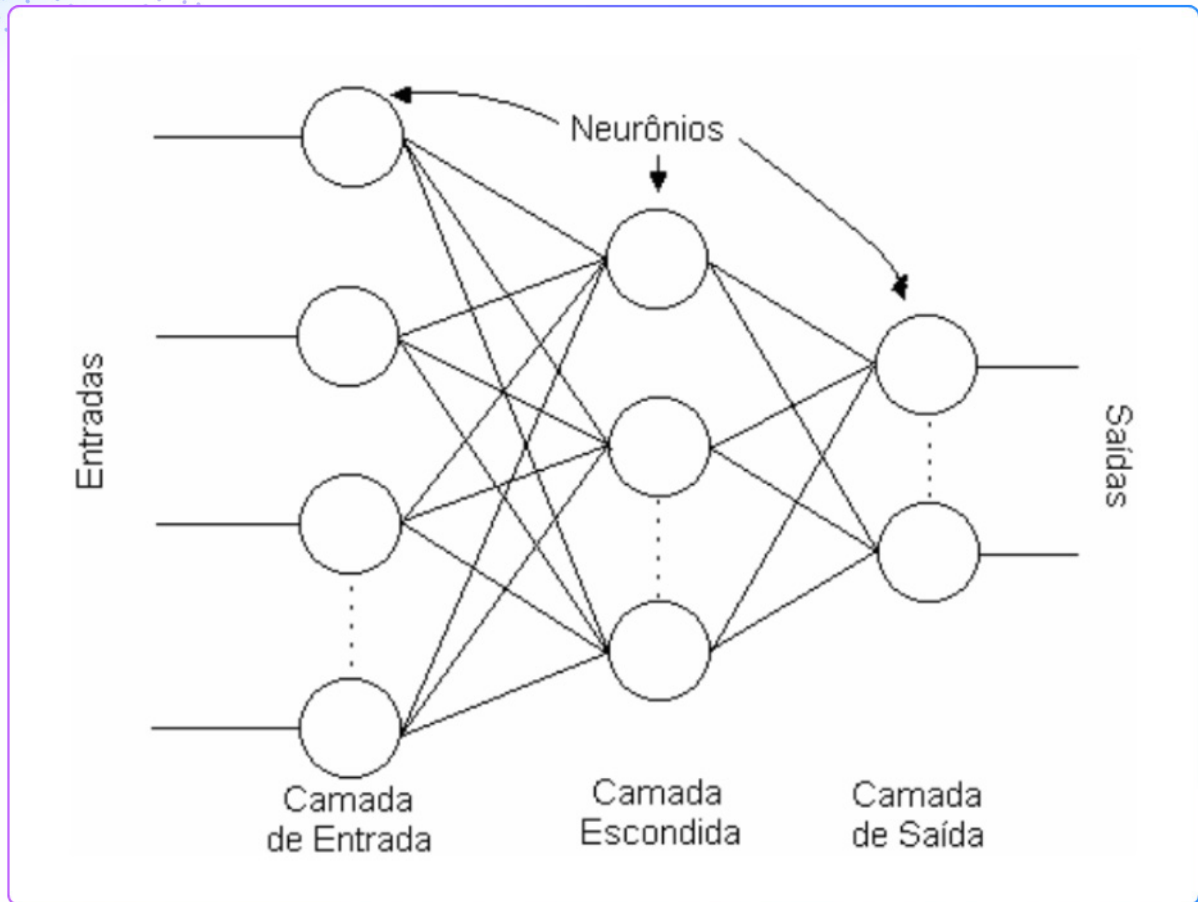
Fonte: autoria própria.

Este modelo de neurônio foi inicialmente proposto como uma nova abordagem de Aprendizado de Máquina, capaz de resolver múltiplas tarefas. No entanto, devido à sua simplicidade, não conseguiu obter resultados significativos. O avanço das RNAs ocorreu com a introdução do algoritmo de *backpropagation*, que possibilitou a criação de redes com múltiplas camadas, conhecidas como *Multi-Layer Perceptron* (MLP).

11.2 Multi-Layer Perceptron

O cérebro humano é composto por uma complexa rede de neurônios biológicos interligados. De maneira similar, às pesquisas em RNAs buscaram formas de combinar vários neurônios artificiais para formar uma rede mais complexa. A solução surgiu com o algoritmo de *backpropagation*, que deu origem às redes chamadas MLPs. Como o nome sugere, essas redes organizam os perceptrons, o neurônio artificial proposto anteriormente, em forma de camadas, como ilustrado na figura a seguir.

Figura 94 - Esquema básico de um *Multi-Layer Perceptron*



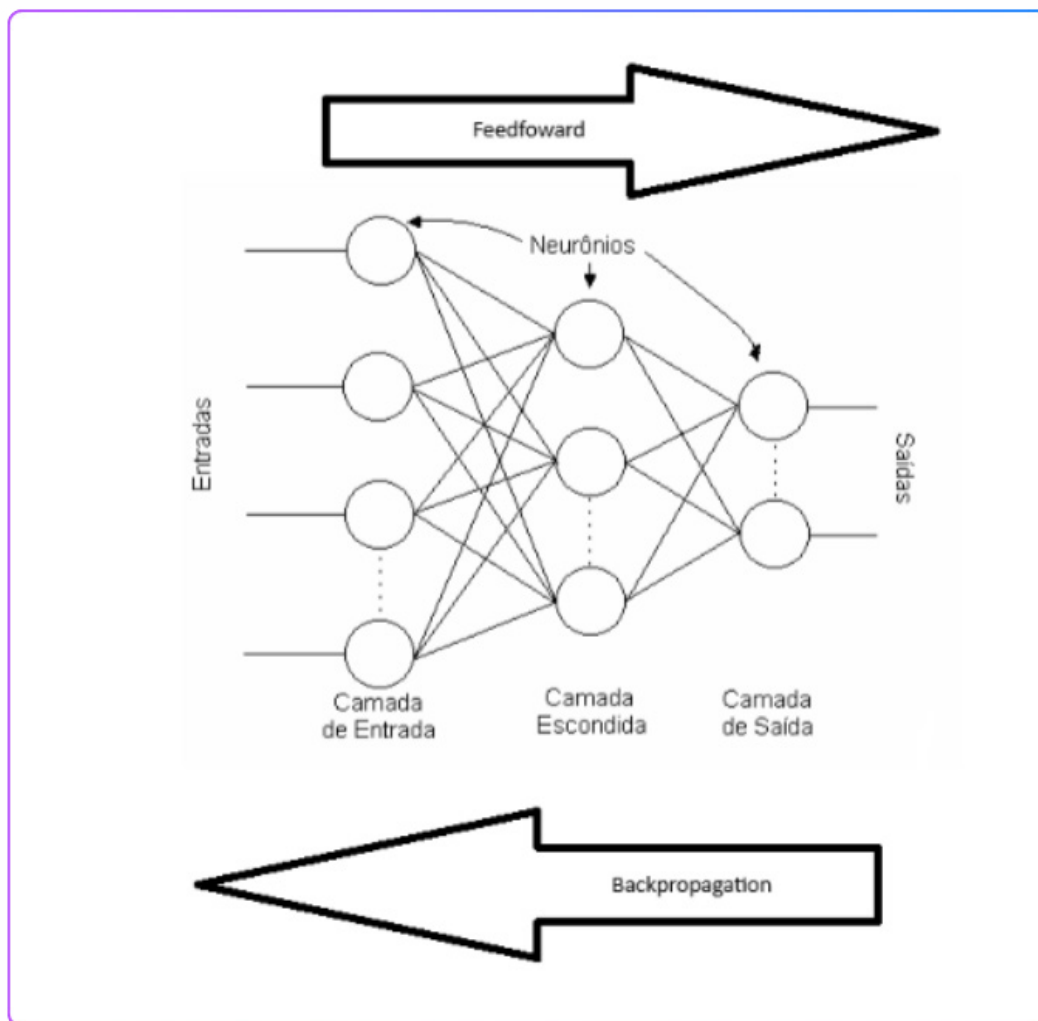
Fonte: adaptada de [Xiang \(2024\)](#).

Em um MLP temos três principais estruturas:

- » **Camada de Entrada:** A camada de entrada recebe as variáveis do problema a ser tratado. Os neurônios desta camada multiplicam seus pesos pelas variáveis, com cada variável tendo um peso independente. Por conta disso, o número de neurônios é dado pelo número de entradas.
- » **Camada Escondida:** A camada escondida recebe as saídas da camada anterior e gera uma nova saída. O conceito por trás da camada escondida é proporcionar uma interpretação mais complexa dos dados por meio da combinação de camadas. Um MLP básico possui ao menos uma camada escondida, mas o mesmo processo de treinamento pode ser aplicado a várias camadas, o que pode proporcionar uma compreensão mais profunda do problema. O número de neurônios na camada escondida é arbitrário e quanto mais, maior a capacidade da rede.
- » **Camada de Saída:** A camada de saída é responsável por fornecer a resposta ao problema, seja uma classificação ou regressão. Ela transforma a abstração gerada pela camada escondida em saídas específicas para o problema tratado. Em um problema de classificação com duas classes, por exemplo, é comum termos dois neurônios de saída, cada um representando a probabilidade da entrada pertencer a uma das classes.

O *backpropagation* é o algoritmo responsável por treinar o MLP. Em um treinamento de uma RNA temos dois estágios: o *feedforward* e logo após o *backpropagation*, como na figura a seguir. No *feedforward* os diversos exemplos de um *dataset* são passados pela rede em direção a saída, isso faz com que tenhamos respostas para um dos exemplos e com base nessas respostas é calculado o erro do MLP.

Figura 95 - Esquema básico de um *Multi-Layer Perceptron* com indicação de fluxo de dados no *feedforward* e *backpropagation*



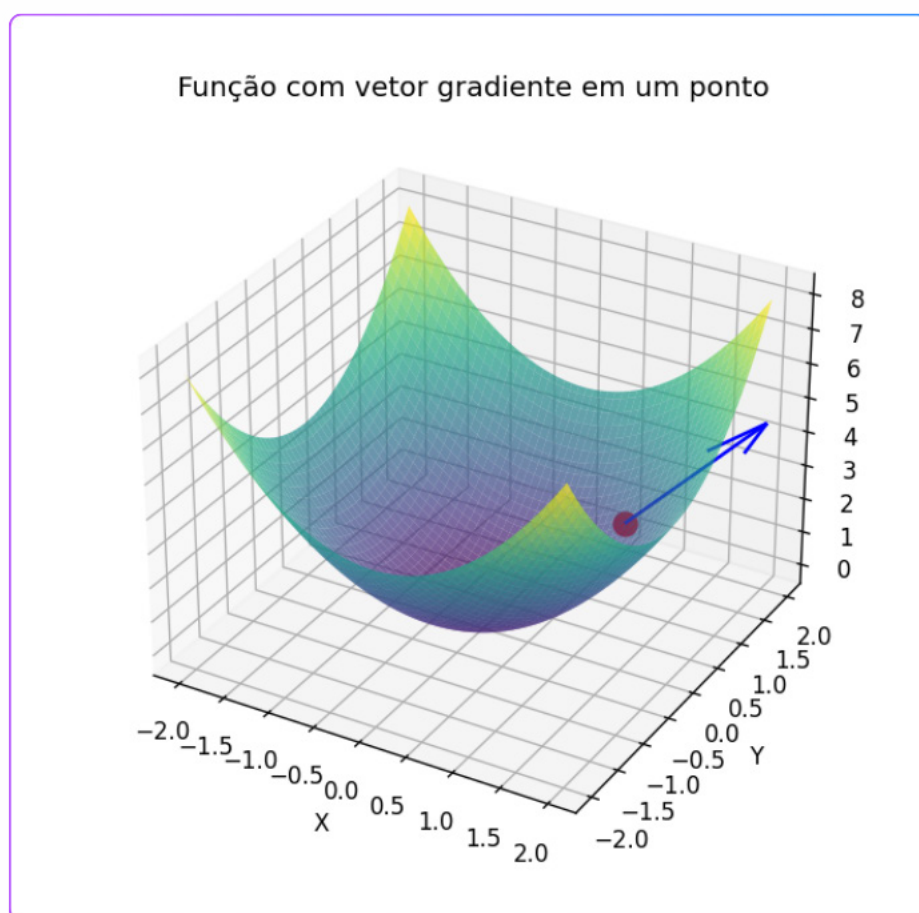
Fonte: adaptada de [Xiang, 2024](#).

O erro é mensurado pela chamada função de custo, também chamada de função de perda ou *loss function* em inglês. A função de custo é responsável por guiar o treinamento para que o MLP reduza o erro a cada tentativa. Uma função de custo comum e que foi usada inicialmente para treinar MLPs é o MSE. A função consiste em calcular a média do erro quadrático de cada amostra como mostra a equação:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y - \hat{y})^2$$

Na equação o n se refere as n amostras do conjunto de treino, o y é o rótulo, resposta esperada, e $\hat{y}$ a resposta calculada pela rede. Existem outras funções de perda como a Entropia Cruzada, Divergência KL e Erro Absoluto Médio, cada uma possui suas características e são usadas conforme o tipo de saída da rede neural. A partir do erro é calculado o vetor gradiente em cada neurônio da camada de saída e logo em seguida ajustados os pesos e o viés. O vetor gradiente é um operador no cálculo vetorial que indica a direção na qual uma função cresce mais rapidamente. Na figura a seguir, podemos ver um exemplo com a superfície de uma função e o vetor gradiente calculado em um ponto específico.

Figura 96 - Exemplo de vetor gradiente em uma função com duas variáveis



Fonte: autoria própria.

Como a função de perda mensura o erro do modelo, ao movimentarmos os pesos na direção oposta ao gradiente, obteremos novos pesos que reduzirão o erro calculado. Esse processo de otimização é chamado de Gradiente Descendente, e a repetição iterativa deste processo realiza o treinamento do MLP, reduzindo progressivamente o erro em relação às amostras de treinamento.

Para corrigir as camadas escondidas e de entrada de um MLP o cálculo do gradiente é realizado novamente a cada camada, gerando assim uma série de cálculos que propagam o erro, daí o nome *backpropagation*. A retropropagação do erro garante que a rede

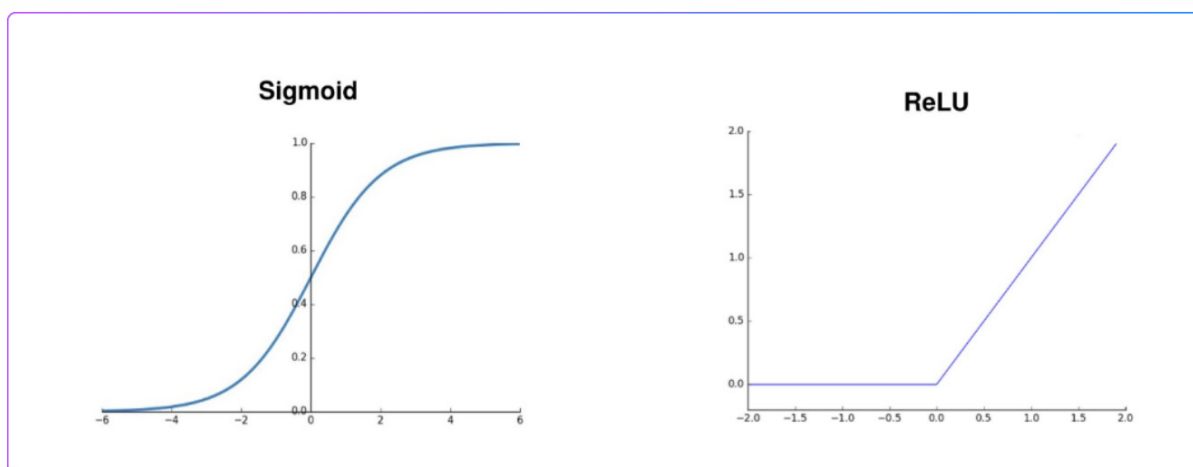
seja ajustada de forma que as camadas trabalhem em conjunto para minimizar o erro e assim gerar respostas corretas.

O MLP oferece uma capacidade muito maior do que um neurônio artificial individual, pois, com várias camadas, é capaz de construir funções discriminantes complexas. No entanto, com a introdução de várias camadas, surge o problema do *vanishing gradient*, ou “sumiço do gradiente” em português. Ao retropropagar o gradiente para camadas anteriores, temos um produto de múltiplos gradientes, o que se torna problemático devido à função de ativação sigmoide, que gera gradientes pequenos e sempre menores que 1. Como resultado, quanto mais camadas escondidas, menor o gradiente nas primeiras camadas, tornando-as mais difíceis de serem treinadas. Este problema conteve o avanço das redes neurais por algumas décadas e a solução para este problema deu origem às chamadas DL.

11.3 Deep Learning

O estudo das redes neurais avançou significativamente após a criação do *backpropagation*, resultando em diversos aprimoramentos em relação ao MLP clássico. Um dos primeiros avanços foi a introdução da função *Rectified Linear Unit* (ReLU), que substituiu a função sigmoide e ajudou a mitigar o problema do *vanishing gradient*. Como mostrado na figura a seguir, a ReLU funciona de maneira simples, permitindo a passagem de sinais positivos e anulando valores negativos, substituindo-os por 0. Esse funcionamento é suficiente para permitir que os neurônios mapeiem suas saídas, ao mesmo tempo em que gera gradientes melhores, pois a derivada (inclinação) da função ReLU é 1 e constante. Com isso, multiplicar seus gradientes através de várias camadas não apresenta o mesmo problema que ocorre com a sigmoide. Com uso da ReLU surgiram as primeiras redes neurais com muitas camadas, caminhando assim para as redes profundas.

Figura 97 - Funções de ativação sigmoide e ReLU



Fonte: adaptada de [Locke \(2023\)](#).

Outros pontos avançaram para o surgimento das redes profundas, sendo os principais:

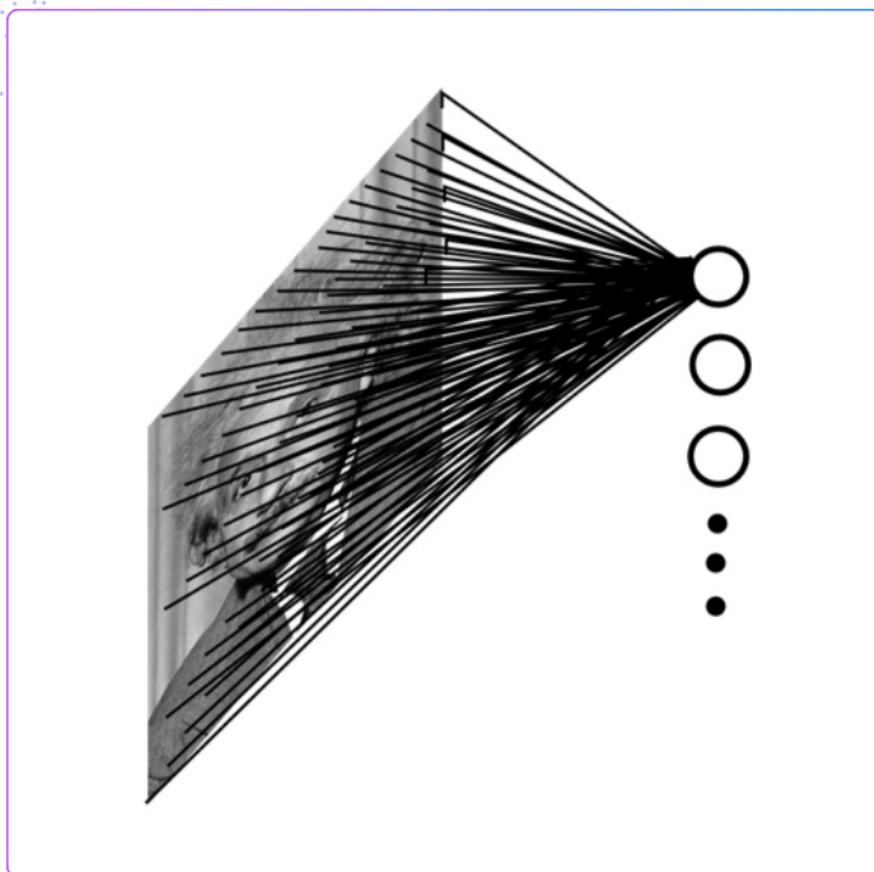
- » **Otimizadores:** Novos algoritmos para a aplicação do gradiente nos pesos, também conhecidos como otimizadores, foram propostos para otimizar as redes de maneira mais rápida e eficiente.
- » **Inicialização dos pesos:** Toda rede neural precisa iniciar com alguns pesos aleatórios para dar início ao processo de *feedforward*. Estudos descobriram formas melhores de definir esses pesos iniciais, melhorando o desempenho do treinamento.
- » **Regularização:** O sobreajuste dos dados sempre foi um problema nas RNAs, pois com muitas camadas e neurônios, o modelo passa a ter alta capacidade, o que facilita o sobreajuste aos dados de treinamento. Para evitar esse problema, foram propostas técnicas como *Dropout* e *Batch Normalization*.
- » **Infraestrutura computacional:** O surgimento de hardware especializado contribuiu enormemente para a evolução das RNAs. Unidades de Processamento Gráfico (GPUs) e Unidades de Processamento de Tensor (TPUs) viabilizaram o treinamento de redes muito maiores do que se tinha anteriormente.

Além dos pontos apontados acima, tivemos também o desenvolvimento de arquiteturas alternativas ao MLP. Sendo a principal a CNN.

11.3.1 CNNs: Redes Neurais Convolucionais

As CNNs propostas nos anos 90 substituem o neurônio artificial, como o perceptron, por uma operação de convolução que pode ser treinada. A convolução é uma operação entre sinais que tem como objetivo buscar por um padrão. A ideia de utilizar uma convolução vem da dificuldade do MLP em lidar com imagens. Uma imagem digital é um mapa de pixel e para usar uma imagem como entrada em um MLP seria necessário um peso para cada pixel. Esse arranjo gera uma entrada com altíssima dimensão, com uma imagem de 200x200 seriam 40 mil entradas, e os pesos não possuem compreensão de que pixels podem ser vizinhos.

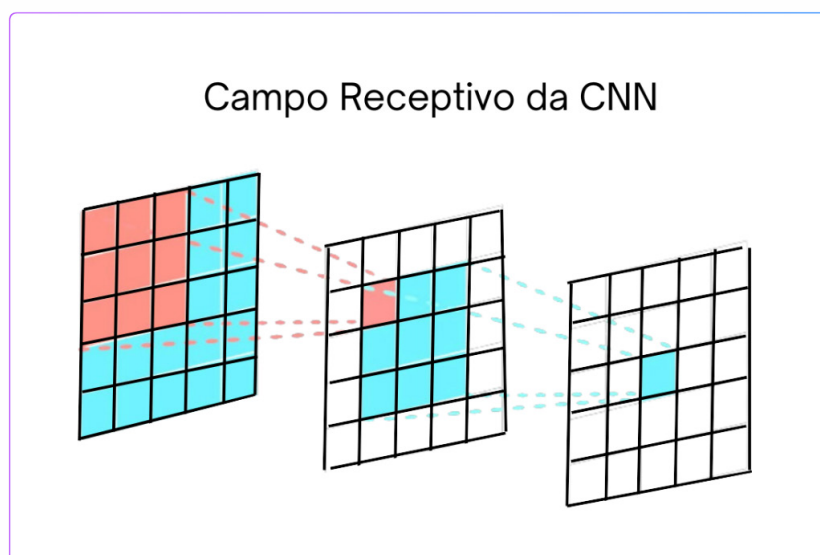
Figura 98 - Esquema básico de um *Multi-Layer Perceptron* ao receber uma imagem



Fonte: [Anderson \(2024\)](#).

A convolução, por outro lado, oferece uma operação que permite compreender a vizinhança dos pixels e reduz a necessidade de tantos pesos, pois funciona como um filtro que captura uma região da imagem, gerando uma interpretação dessa região como saída. A figura a seguir mostra de forma simplificada uma sequência de duas convoluções sendo aplicadas.

Figura 99 - Esquema de um campo receptivo de duas camadas convolucionais



Fonte: Adaptada de [Deepia \(2024\)](#).

Figura 101 - Exemplo de imagens do ImageNet

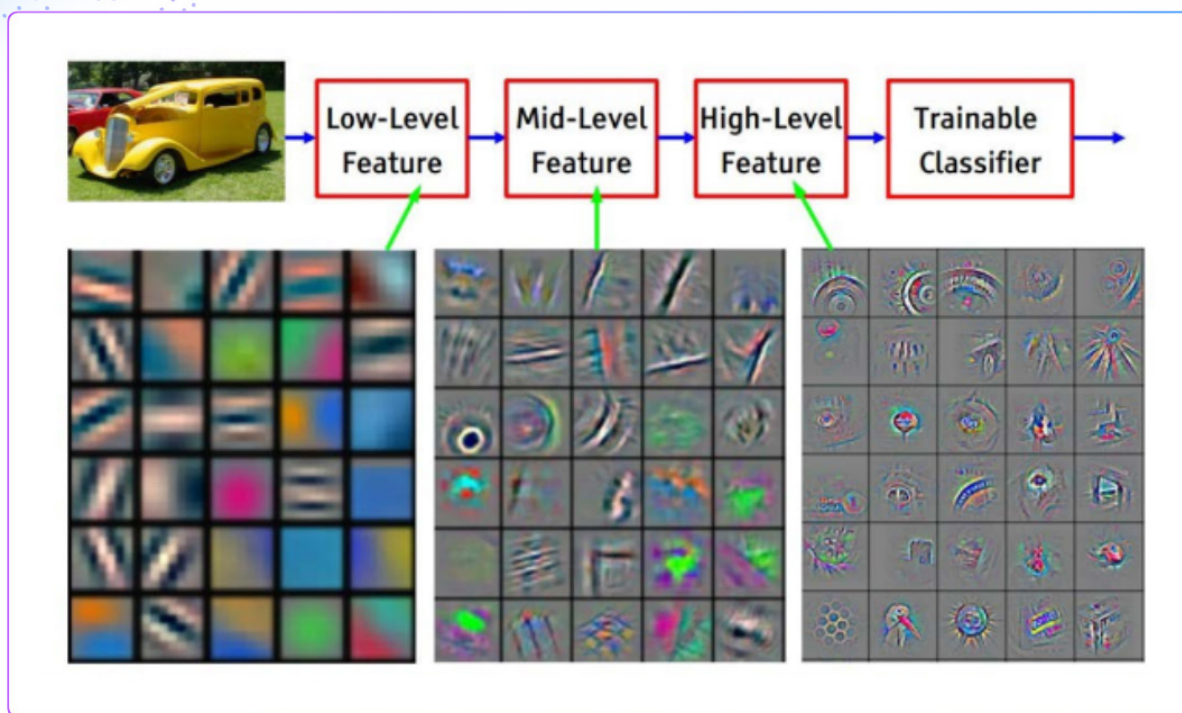


Fonte: [Gluon \(2012\)](#).

O resultado da AlexNet trouxe grande atenção às redes neurais a partir de 2012, levando ao surgimento de várias outras arquiteturas e avanços nas RNAs. Na visão computacional, surgiram soluções revolucionárias para tarefas como segmentação de imagens, detecção de objetos e descrição de imagens. Grande parte desse avanço se deve à introdução do conceito de Transferência de Aprendizado, conhecido como *transfer learning* em inglês.

A transferência de aprendizado ocorre ao treinar uma CNN em um grande conjunto de dados, como o ImageNet, em uma tarefa mais simples — no caso do ImageNet, a classificação de imagens — e, em seguida, usar os pesos resultantes para iniciar outro treinamento em uma tarefa semelhante, mas com um menor volume de dados disponível. Assim, o primeiro treino passou a ser chamado de pré-treino, enquanto o segundo treinamento é conhecido como ajuste fino, ou *fine tuning*. Ao retrainar a rede, ela é capaz de trazer consigo o entendimento básico dos objetos do *dataset* do seu treino inicial, como exemplificado na figura a seguir, e adaptar esse conhecimento prévio para uma nova tarefa.

Figura 102 - Visualização das convoluções obtidas ao treinar uma CNN



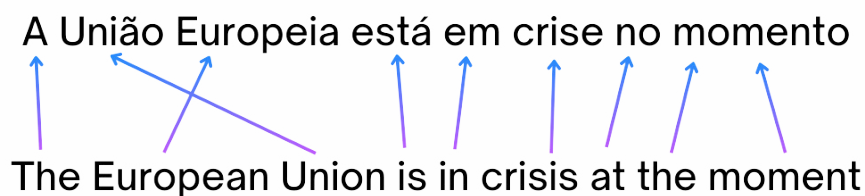
Fonte: adaptada de [Zeiler, Fergus, 2013.](#)

Essa abordagem permitiu o uso de DL em diversos problemas que anteriormente não eram abordados com RNAs devido à falta de volume de dados. No entanto, as CNNs tratavam majoritariamente de problemas que envolviam imagens como entradas. Em outros campos, como Linguagem Natural e Processamento de Áudio, as CNNs e o *transfer learning* ainda enfrentavam dificuldades para obter bons resultados. A mudança nessas áreas ocorreu com o surgimento da arquitetura Transformer, que introduziu uma operação diferente tanto do MLP quanto da convolução, permitindo o *transfer learning* com textos e áudios.

11.3.2 Transformer

Em 2017 foi introduzida pela equipe da Google a arquitetura chamada Transformer. Inicialmente proposta para tradução de textos, a arquitetura usa de uma operação chamada “mecanismo de atenção” para processar suas entradas. O mecanismo de atenção é uma técnica que permite que o modelo foque em partes específicas da entrada enquanto realiza uma tarefa. Ele atribui diferentes pesos a diferentes partes da entrada, destacando informações mais relevantes, como no exemplo da figura a seguir, melhorando assim a performance em tarefas sequenciais e de PLN.

Figura 103 - Visualização da atenção em um problema de tradução



Fonte: autoria própria.

Com o uso dos *Transformers* (transformadores), foi possível realizar um pré-treino eficiente tanto em Linguagem Natural quanto em Áudio. Utilizando a arquitetura pré-treinada na tarefa de modelagem de linguagem, que consiste em prever a próxima palavra de um texto, surgiram modelos como BERT, GPT e Llama, que revolucionaram o PLN ao resolverem tarefas anteriormente consideradas complexas. O impacto foi semelhante ao das primeiras CNNs treinadas com o ImageNet, que, no caso do PLN, introduziu o DL em várias tarefas anteriormente tratadas com métodos clássicos de PLN. No contexto do processamento de áudio, modelos baseados em *Transformers* como Whisper, Wav2Vec, HuBERT e Tacotron viabilizaram tarefas complexas, como reconhecimento, transcrição e sintetização de fala.

11.3.3 Rede Neurais Generativas

RNAs atualmente passaram a dominar a maior parte do campo da IA devido ao ótimo desempenho em tarefas complexas. A possibilidade de explorar os grandes conjuntos de dados que se formaram a partir da digitalização de processo e do avanço da internet age como combustível em soluções baseadas em RNAs. Com o desenvolvimento das redes neurais generativas e de modelos, o campo testemunhou uma explosão na criação de conteúdo sintético, desde imagens hiper-realistas até música e vídeo. Essas inovações não só melhoraram o desempenho em tarefas tradicionais, mas também abriram novas aplicações e indústrias, como arte digital, realidade aumentada e IA criativa. O contexto atual das RNAs é, portanto, marcado por um ciclo virtuoso de inovação, onde grandes dados alimentam redes mais poderosas, que, por sua vez, geram novos dados e tendências, continuando a impulsionar o progresso tecnológico.

11.3.4 Frameworks para DL

Com o avanço das RNAs e o surgimento do DL, foram criados *frameworks* para viabilizar a implementação de novas arquiteturas neurais. Inicialmente, as implementações eram feitas usando Nvidia *Compute Unified Device Architecture* (CUDA), que permite a programação de GPUs de forma geral, e não apenas para a geração de gráficos. No entanto, devido à complexidade da programação com CUDA, foram propostos *frameworks* que servem como interface para o uso das GPUs no treinamento de redes neurais.

Entre os vários *frameworks* que surgiram, os principais são o TensorFlow e o PyTorch. O TensorFlow foi inicialmente desenvolvido pela Google para uso interno e, em 2015, foi disponibilizado como uma biblioteca de código aberto. O PyTorch foi lançado pela Meta AI (antigo Facebook) em 2016 e foi desenvolvido a partir do framework Torch. Ambos os *frameworks* são baseados principalmente em Python e atuam como uma interface para operações implementadas em CUDA. Essas bibliotecas foram fundamentais para popularizar a implementação e a criação de novos modelos de DL, facilitando o processo de codificação do treinamento e da inferência.



SAIBA MAIS...

- » Perceptron - <https://youtu.be/6yYUc6nU3Cw?si=LZGZbtV-Ph0d4IJJo>
- » Redes Neurais - <https://www.youtube.com/watch?v=CqOfi41LfDw>
- » Redes Neurais - <https://youtu.be/XxZ0BibMTjw?si=vtO6zjtguK1a95nX>
- » Gradiente Descendente -
 - » https://youtu.be/31w-xQX0Z_8?si=dkeqa39mdwSTYzhB
- » Propagação reversa - <https://www.youtube.com/watch?v=IN2XmBhILt4>
- » Propagação reversa - <https://youtu.be/DGNbd2FGw2s?si=HJMNSUrcfyBsQcG4>
- » ReLU - <https://www.youtube.com/watch?v=68BZ5f7P94E>
- » RNN - <https://www.youtube.com/watch?v=AsNTP8Kwu80>
- » LSTM - <https://www.youtube.com/watch?v=YCzL96nL7j0>
- » Transformers - <https://www.youtube.com/watch?v=zxQyTK8quyY>
- » CNN - https://youtu.be/n4rmrZg1_58?si=NxSTmjKr9Ax0lxaH
- » AlexNet and ImageNet: The Birth of *Deep Learning*: -
 - » <https://www.pinecone.io/learn/series/image-search/imagenet/>
- » Tutorial de Pytorch: <https://pytorch.org/tutorials/beginner/basics/intro.html>
- » Tutorial de TensorFlow: <https://www.tensorflow.org/learn?hl=pt-br>

PARA RELEMBRAR...

Estrutura das Redes Neurais:

- » Camada de Entrada: recebe dados brutos
- » Camadas Ocultas: processam e capturam complexidade dos dados
- » Camada de Saída: produz a decisão final ou inferência

Fluxo de Dados e Treinamento:

- » *Feedforward*: propagação direta dos dados da entrada para a saída
- » *Backpropagation*: ajuste de pesos baseado no erro entre saída desejada e produzida

Gradient Descent: algoritmo para minimizar a função de custo

Tipos de Redes Neurais:

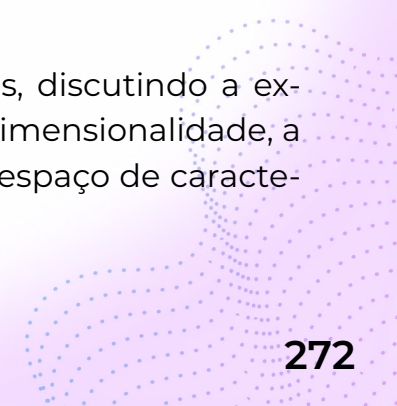
- » *Perceptrons*: neurônio artificial simples
- » MLP: múltiplas camadas de *Perceptrons*
- » CNNs: para processamento de imagens
- » RNNs: para dados sequenciais (LSTM, GRU)
- » *Autoencoders*: para compressão e geração de dados
- » GANs: para geração de dados realistas

Unidade XII

Encerramento



Unidade XII - Encerramento



12.1 Considerações Finais

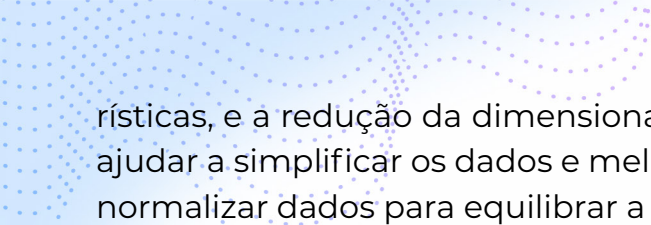
A jornada pelas técnicas de ML e redes neurais destaca a complexidade e a interconexão das diversas etapas envolvidas no desenvolvimento de modelos eficazes. Desde a definição clara e a aplicação prática dos conceitos até a implementação de algoritmos avançados, cada passo é crucial para alcançar resultados precisos e úteis. A importância de uma boa prática de pré-processamento, incluindo a extração, seleção e normalização de características, não pode ser subestimada, pois essas etapas formam a base sobre a qual modelos robustos são construídos. Além disso, a compreensão dos desafios como a maldição da dimensionalidade e o *overfitting* (sobreajuste) é vital para criar modelos que não apenas aprendam bem com os dados de treinamento, mas também se generalizem bem para novos dados.

Ao longo dessa unidade, discutimos uma série de tópicos cruciais no campo do ML, onde exploramos sua definição, suas aplicações, e as diferenças entre IA, ML e DL. Entendemos que ML é a prática de treinar algoritmos para aprender com dados e fazer previsões ou decisões com base nesses dados. Exploramos como diferentes tipos de ML, como aprendizado supervisionado e não supervisionado, são aplicados em áreas diversas, desde reconhecimento de imagem até previsão de mercado.

Voltamos nossa atenção para conceitos como detecção de padrões e anomalias, e as tarefas de detecção, reconhecimento e previsão. Falamos sobre como a detecção de padrões e anomalias pode ser usada para identificar comportamentos esperados e inesperados nos dados, enquanto a detecção, reconhecimento e previsão são fundamentais para a construção de sistemas inteligentes capazes de interpretar dados e antecipar eventos futuros.

Foram discutidos os diferentes tipos de aprendizagem: supervisionada, não supervisionada, semi-supervisionada e por reforço. Cada tipo foi explicado tecnicamente, complementado com metáforas para facilitar a compreensão e exemplos práticos para ilustrar sua aplicação. Discutimos a importância de entender os diferentes métodos de aprendizagem de máquina para escolher a abordagem mais adequada para cada situação.

Abordamos o processamento e transformação de características, discutindo a extração e seleção de características, a relação entre características e dimensionalidade, a maldição da dimensionalidade, a normalização e transformação do espaço de caracte-



rísticas, e a redução da dimensionalidade. Vimos como técnicas bem aplicadas podem ajudar a simplificar os dados e melhorar a eficiência computacional, e a importância de normalizar dados para equilibrar a contribuição de diferentes características.

Exploramos algumas tarefas clássicas como Regressão, Classificação e Agrupamento, passando por diferentes abordagens e modelos para cada problema. Por fim, estudamos as RNAs que tanto estão em destaque atualmente e o processo de surgimento do DL.

Este curso tem como objetivo apresentar os conceitos e ferramentas que fundamentam o ML. Ao passar por todos esses tópicos, esperamos que você tenha construído uma base sólida de conhecimento para navegar pelo universo dos problemas orientados a dados e criar novas soluções.

Referências

- AEBERHARD, Stefan; COOMANS, Danny; DE VEL, Olivier. Comparative analysis of statistical pattern recognition methods in high dimensional settings. **Pattern Recognition**, v. 27, n. 8, p. 1065-1077, 1994.
- BELLMAN R.E. Adaptive Control Processes. **Princeton: Princeton University Press**, 1961.
- BISHOP, C. M. Pattern Recognition and *Machine Learning*. New York: **Springer**, 2006.
- CHAPELLE, O.; SCHOLKOPF, B.; ZIEN, A. *Semi-Supervised Learning*. MIT Press, 2006.
- CHEN, Tianqi *et al.* Package 'xgboost'. **R version**, v. 90, n. 1-66, p. 40, 2019.
- ESTER, Martin *et al.* **A density-based algorithm for discovering clusters in large spatial databases with noise**. In: kdd. 1996. p. 226-231.
- FISHER, Ronald A. The use of multiple measurements in taxonomic problems. **Annals of eugenics**, v. 7, n. 2, p. 179-188, 1936.
- FIX, Evelyn. Discriminatory analysis: nonparametric discrimination, consistency properties. **USAF school of Aviation Medicine**, 1985.
- GÉRON, A. Hands-On *Machine Learning* with *Scikit-Learn*, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems. **O'Reilly Media**, 2019.
- GUYON, I.; ELISSEEFF, A. An Introduction to Variable and Feature Selection. **Journal of Machine Learning Research**, 2003.
- HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. The Elements of Statistical Learning: Data Mining, Inference, and Prediction. **Springer Series in Statistics**, 2009.
- HYNDMAN, R.J.; ATHANASOPOULOS, G. Forecasting: principles and practice. Melbourne, Austrália: **O Texts**, 2021.
- KE, Guolin *et al.* *Lightgbm*: A highly efficient *gradient boosting* decision tree. **Advances in neural information processing systems**, v. 30, 2017.
- MCKINNEY, W. Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython. **O'Reilly Media**, 2017.
- MITCHELL, T. M. *Machine Learning*. **McGraw-Hill**, 1997.
- NORVIG, Peter; RUSSELL, Stuart. Inteligência artificial. Rio de Janeiro: **Grupo GEN**, 2013.
- STARMER, J. The StatQuest Illustrated Guide to *Machine Learning!!!* **Packt Publishing**, 2022.
- THEODORIDIS, S.; KOUTROUMBAS, K. *Nonlinear classifiers*. **Academic Press**, p. 151-260, 2009.
- VANDERPLAS, J. **Python Data Science Handbook: Essential Tools for Working with Data**. O'Reilly Media, 2016.



CEIQ
CENTRO DE EXCELÊNCIA EM
INTELIGÊNCIA ARTIFICIAL

GOV. DE
GOIÁS
O ESTADO QUE DÁ CERTO



INF
INSTITUTO DE
INFORMÁTICA

PRPI
PRÓ-REITORIA DE
PESQUISA E INOVAÇÃO



UFG
UNIVERSIDADE
FEDERAL DE GOIÁS

SOBRE O E-BOOK

Tipografia: Montserrat

Publicação: Cegraf UFG

Câmpus Samambaia, Goiânia -

Goiás, Brasil. CEP 74690-900

Fone: (62) 3521-1358

<https://cegraf.ufg.br>